

PULSE SEQUENCE PROGRAM MANUAL

Abstract

This guide will show you how to program sequences on your system.

Document history

Revision	Date	Comment
1.0	15/11/11	Creation
1.1	17/01/23	New template

Document Ref: MRS_DOCUMENT_ID
Date completed: 21-06-2019
Prepared by: Jerome Burnet
Technical Consultant / Expert: update
Checked by: update

Please note: This document's version may not be the latest.

You can check and download latest version at the following link:

<http://mrsolutions-users.com/>

The above link requires a registration to MR Solutions' forum where users can share & interact about their work.

Notices

Disclaimer

MR Solutions Ltd (MRS) reserves the right to make changes to any hardware, software or documentation without further notice. The information contained herein is believed to be reliable; however, **MRS** assumes no responsibility for possible inaccuracies or omissions. No patent rights or licences to any of the designs described herein are implied or granted to any third party.

The User must not modify, add to or connect anything to the **MRS** system without the prior written permission of **MRS**. This is particularly important in respect of the earth wiring as it may result in a hazardous condition.

Warranty

MRS warrants products and spare parts of its own manufacture against faulty materials and workmanship for a period of 12 months from the date of purchase or on-site acceptance, if installed by **MRS** authorised personnel.

The warranty excludes consumable materials and parts with an inherently limited life and excludes faults arising from misuse, neglect or vandalism.

MRS will replace or repair at its option and at no cost to the purchaser any faulty items reported to it within the warranty period.

Service

Service of this equipment by replacement of parts, on site repair or routine maintenance inspections is available for this equipment. Please contact **MRS** Support for a quotation.

Technical Support

Please refer to the last chapter of this document.

Design Changes

MRS reserves the right to change the design of any product from time to time without notice and with no obligation to make corresponding changes in products previously manufactured.

Copyright

Copyright © 2011 MR Solutions Ltd. The copyright in this document which contains proprietary information is vested in **MRS** Limited. All rights reserved. No part of this publication may be reproduced, photocopied, stored in a retrieval system, transmitted, or translated into any language without the prior written permission of **MRS**.

Trademarks

MRS is the registered trademark of MR Solutions Ltd.

Contents

Chapter 1.	Introduction.....	10
Chapter 2.	The paramlist headers.....	11
2.1	List of keywords.....	12
2.1.1	Generic Controls	12
2.1.2	Basic 1D Transmission/Acquisition.....	12
2.1.3	Optional 1D Acquisition.....	12
2.1.4	Basic Imaging	13
2.1.5	Basic Multi-Dimensional Acquisition	13
2.1.6	Extended Multi-Dimensional Acquisition.....	13
2.1.7	Extended Imaging	14
2.1.8	Multiple Receivers	14
2.2	Keyword Reference	14
2.2.1	Generic Controls	14
2.2.2	Basic 1D transmission/Acquisition	16
2.2.3	Optional 1D Acquisition.....	18
2.2.4	Basic Imaging	23
2.2.5	Basic Multi-Dimensional Acquisition	30
2.2.6	Extended Multi-Dimensional Acquisition.....	32
2.2.7	Extended imaging.....	38
2.2.8	Multiple Receivers	39
2.3	Common int data type	41
2.4	Templates.....	43
2.5	Appendix.....	44
Chapter 3.	Library functions.....	46
3.1	#include directive	46
3.2	List of library functions	47
3.2.1	Fast RF control functions	47
3.2.2	Instrument control RF functions.....	48
3.2.3	RF control functions.....	49

3.2.4	Gradient Control Functions.....	49
3.3	PPL library hardware control function reference	50
3.3.1	Fast RF control	50
3.3.2	Summary of library function timing for RF controller	60
3.3.3	Support RF functions - slow control of instrument parameters	62
3.4	RF control functions.....	64
3.4.1	MR3031_setup.....	65
3.4.2	Append_Boards_Ready	66
3.4.3	Disable3031	66
3.4.4	Enable3031	66
3.4.5	Get3031Status.....	66
3.4.6	HaltEnabled3031	67
3.4.7	MR3031_command.....	67
3.4.8	MR3031_go	67
3.4.9	MR3031_reset.....	68
3.4.10	MR3031_stop.....	68
3.4.11	RunEnabled3031	68
3.4.12	Set3031Address	69
3.4.13	Set3031LoopMode	69
3.4.14	Set3031Waits.....	69
3.4.15	Set_Addr_And_Waits.....	70
3.4.16	Set_Board_Multipliers	70
3.5	Gradient Control Functions.....	70
3.5.1	MR3040_SetListAddress	70
3.5.2	MR3040_InitList	70
3.5.3	MR3040_Output.....	71
3.5.4	MR3040_Hold	71
3.5.5	MR3040_LoopCount.....	72
3.5.6	MR3040_SetList	72
3.5.7	MR3040_Start	72

3.5.8	MR3040_Continue	72
3.5.9	MR3040_Break	72
3.5.10	MR3040_Stop	73
3.5.11	MR3040_Control	73
3.5.12	MR3040_RunChannels	73
3.5.13	MR3040_Clock	74
3.5.14	MR3040_SeqStatus	74
3.5.15	MR3040_CreateBaseMatrix	74
3.5.16	MR3040_CreateMatrix	75
3.5.17	MR3040_ModifyMatrix.....	75
3.5.18	MR3040_MatrixStatus	75
3.5.19	MR3040_SelectMatrix.....	76
3.6	Instrument parameters (WIN.INI)	76
Chapter 4.	Introduction to ppl.....	77
4.1	Console input and output	77
4.2	Numbers and constants	78
4.3	Expressions.....	80
4.4	Program flow control (loops and branches)	82
4.5	Input and Output to the User Port	85
4.6	Delays and Timing functions	87
4.7	Pulse Generation.....	91
4.8	Pulse Programming Language (PPL) reference.....	94
4.8.1	Comments.	94
4.8.2	Variables and constants.....	95
4.8.3	Declaration of constants.....	95
4.8.4	Declaration of int variables.....	96
4.8.5	Declaration of arrays of type int.....	96
4.8.6	4.8.6 Declaration of common int variables	96
4.8.7	32 bit data type.....	97
4.8.8	Declaration of library functions	98

4.8.9	Library functions	100
4.8.10	Operators	104
4.8.11	Operator precedence	110
4.8.12	Scale function	111
4.8.13	Console I/O functions	111
4.8.14	Expressions	112
4.8.15	Control structures.	114
4.8.16	Looping	115
4.8.17	Statements	116
4.8.18	Program syntax	116
4.9	Execution times summary	117
4.9.1	Delay timings.	117
4.9.2	Expression timings.	117
4.9.3	Examples of expression timing calculations:	119
4.9.4	Mixed type expression timing	119
4.9.5	Scale function timing	121
4.9.6	Output timings.	122
4.9.7	Input timings	122
4.9.8	Pulse function timings	122
4.9.9	For loop timings.	123
4.9.10	do - until loop timing	124
4.9.11	If - else timings	124
4.9.12	Goto timing	126
4.9.13	Printf and Read timing	126
4.9.14	Function calls	126
Chapter 5.	Pulse sequence writing guide	128
5.1	Introduction	128
5.2	Software Overview	128
5.3	Structure of MRI data	128
5.4	Introduction to pulse program layout	130

5.4.1	Program syntax.....	130
5.4.2	#USE and # INCLUDE statements	132
5.4.3	PARAMLIST HEADER	132
5.4.4	CONSTS, INTS and COMMON INTS.....	133
5.4.5	Sequence setup, error messages, etc.	133
5.4.6	Pulse sequence output.....	134
5.5	Gradient Controller	134
5.5.1	Naming Convention	134
5.5.2	Parsetup Keyword Definitions	135
5.5.3	OBLIQUE_ORIENTATION.....	135
5.5.4	GRADIENT_STRENGTH.....	136
5.5.5	Oblique Angle Transformations	136
5.6	Appendix A: #include File	138
5.6.1	M3040_15.PPH.....	139
5.6.2	M3031_15.PPH.....	142
5.6.3	OFFST_15.PPH	143
5.6.4	VAR_15.PPH.....	144
5.7	Appendix B: Coding Pilot Capability into PPL Programs	146
5.8	Appendix C: Timings of Gradient Controller Functions	147
5.8.1	MR3040_SetListAddress.	147
5.8.2	MR3040_InitList().....	147
5.8.3	MR3040_Output(int loop_type, int frame_address, int frame_points, int frame_rate)	147
5.8.4	MR3040_Hold(int loop_type)	147
5.8.5	MR3040_LoopCount(int loop_count)	147
5.8.6	MR3040_SetList(int address, int channel_id)	147
5.8.7	MR3040_Start(int channel_id)	147
5.8.8	MR3040_Continue(int channel_id)	147
5.8.9	MR3040_Break(int channel_id)	148
5.8.10	MR3040_Stop(int channel_id)	148

5.8.11	MR3040_Control(int control_bits)	148
5.8.12	MR3040_RunChannels(int address3, int address2, int address1)	148
5.8.13	MR3040_Clock(int duration)	148
5.8.14	MR3040_SeqStatus(int mask)	148
5.8.15	MR3040_CreateBaseMatrix(int z_angle, int y_angle, int x_angle, int div_z, int mul_z, int div_y, int mul_y, int div_x, int mul_x)	148
5.8.16	MR3040_CreateMatrix(int matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle)	148
5.8.17	MR3040_ModifyMatrix(int_dest_matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle, int source_matrix_id)	149
5.8.18	MR3040_MatrixStatus(int mask)	149
5.8.19	MR3040_SelectMatrix(int matrix_id)	149
5.9	Appendix D: File format specification	149
5.9.1	MRD: Raw spectrometer data files	149
5.9.2	SUR: Reconstructed image data files.....	151
5.9.3	PPR: sequence parameter files.....	153
Chapter 6.	Compiler warnings and error messages.....	157
6.1	Warnings.....	157
6.2	Error messages	157
Chapter 7.	Pulse Programs Examples.....	165
Chapter 8.	Final Words	221
Chapter 9.	Quick procedure (for printing)	Error! Bookmark not defined.

Chapter 1. INTRODUCTION

The chapters in this manual are organised as follow:

- Chapter 2 describes the PARAMLIST HEADERS which are used in order to define the user interface at run-time for the particular sequence
- Chapter 3 introduces the library of pulse programming functions which have been written specifically for controlling the hardware in the console, mainly related to the RF unit and the gradient control unit
- Chapter 4 provides the structures of the pulse programming language (PPL)
- Chapter 5 introduces guidance for users to write pulse sequence program
- Chapter 6 provides a list of PPL sequence compiler warnings and error messages
- Chapter 7 guides through PPL pulse sequence examples

Chapter 2. THE PARAMLIST HEADERS

The windows program **PARAMETER EDITOR** allows the users to set up MR parameters by editing the values displayed on the screen. A header section (`/*PARAMLIST ~ END*/`) in the Pulse Programming Language (PPL) program determines these items. Writing programs in this way means that the author of the PPL program can protect the user from the intricacies of the pulse sequence program but still permit control over the important parameters.

The function of the header is to describe a series of PPL variables for users via the parameter editor. The variables in the header section are initialised just before the program begins to execute. Functionally this is the same as compile initialisation time.

The PARAMLIST header in the .PPL file has the following general format:

```
/* PARAMLIST
KEYWORD [pre-prompt,] [post-prompt,] [min,] [max,] default, variable, [scale,] [further parameters, ..];
...
END
*/
```

Terms in [] are optional and whether or not they are required depends upon the particular keyword. There must be at least one white space character before and after the PARAMLIST and END keywords. Between these limits are a number of lines consisting of a keyword followed by one or more items separated by commas, and terminated with a semi-colon. Up to 100 lines may be included in this header section. The entire section would be handled as a comment as far as the PPL compiler is concerned.

Items in the PPL program, including all those in the header section, are expressed in units. In particular, variables included within the header section may only be of 16-bit integer type. Items viewed and entered by the user within the PARAMETER EDITOR should be in sensible user units, e.g. cm, ms or kHz; these may also include values with a decimal fraction. A scale factor and a format descriptor are included in the syntax of the PARAMLIST keyword definitions to enable inter-conversion of PPL units and user units. It is the responsibility of the program author to ensure that these scale factors are set up for correct conversion.

2.1 List of keywords

Keywords fall into several classes:

- Generic Controls
- Basic 1D Transmission/Acquisition
- Optional 1D Acquisition
- Basic Imaging
- Basic Multi-Dimensional Acquisition
- Extended Multi-Dimensional Acquisition
- Extended Imaging
- Multiple Receivers

2.1.1 **Generic Controls**

- EDITTEXT "pre-prompt","post-prompt","format", min, max, default, scale, var_name;
- SCROLLBAR "pre-prompt","post-prompt","format", min, max, default, scale, var_name;
- VAR_ARRAY "pre-prompt","post-prompt","format", min, max, scale, array_name, no_elements [, default];

2.1.2 **Basic 1D Transmission/Acquisition**

- OBSERVE FREQUENCY "nucleus", min, max, default, ppl_var_MHz, ppl_var_kHz, ppl_var_Hz, ppl_var_rx1MHz;
- DECOUPLE_FREQUENCY "nucleus", min, max, default, ppl_var_MHz, ppl_var_kHz, ppl_var_Hz, ppl_var_rx1MHz;
- NO_SAMPLES min, max, default, var_name;
- SPECTRAL_WIDTH min, max, default, var_name;

2.1.3 **Optional 1D Acquisition**

- BUFFER_SIZE min, max, default;
- DATA_TYPE default format code, e.g. 0x13;
- DISCARD min, max, default, ppl_var_name ;

- DSP_ROUTINE "dsp_program, e.g. c:\smis\dsp";
- NO_AVERAGES min, max, default, var_name;
- PHASE_CYCLE min, max, default, var_name;

2.1.4 Basic Imaging

- GRADIENT_STRENGTH grad_var;
- FOV read_seq, phase_seq, phase_time, read_var, phase_var ;
- FOV_READ_OFF min, max, default, scale, var_name ;
- FOV_PHASE_OFF min, max, default, scale, var_name ;
- FOV_SLICE_OFF min, max, default, scale, var_name ;
- SLICE_FOV phase_seq, phase_time, phase_var ;
- SLICE_THICKNESS slice_val, rf_bandwidth, slice_var;
- SLICE_SEPARATION min, max, default, var_Hz;
- ORIENTATION min, max, default, var_name ;
- OBLIQUE_ORIENTATION min, max, default, phase_var, x_angle, y_angle, z_angle;
- MULTI_ORIENTATION min, max, default, phase_var, x_angle_array, y_angle_array, z_angle_array ;

2.1.5 Basic Multi-Dimensional Acquisition

- NO_ECHOES min, max, default, var_name ;
- NO_SLICES min, max, default, var_name;
- NO_VIEWS min, max, default, var_name ;
- NO_VIEWS_2 min, max, default, var_name ;
- EXPERIMENT_ARRAY min, max, default, var_name;

2.1.6 Extended Multi-Dimensional Acquisition

- VIEWS_PER_SEGMENT min, max, default, var_name ;
- SLICE_BLOCK min, max, default, var_name;
- VIEW_BLOCK min, max, default, var_name;

- BATCH_EXPERIMENTS min, max, default, var_name;
- BATCH_SLICES min, max, default, var_name;
- SLICE_INTERLEAVE min, max, default, var_name;
- BATCH_INTERLEAVE min, max, default, var_name;
- SAMPLE_PERIOD_2 min, max, default, divisor, var_name;
- SWEEP_WIDTH_2 min, max, default, divisor, var_name;

2.1.7 Extended Imaging

- READ_SELECTION read_val, rf_bandwidth, read_var;
- PHASE_SELECTION phase_val, rf_bandwidth, phase_var;
- SLICE_SELECTION slice_val, rf_bandwidth, slice_var;

2.1.8 Multiple Receivers

- NO_RECEIVERS min, max, default, var_name;
- RECEIVER_MASK min, max, default, var_name;

2.2 Keyword Reference

The keywords' argument-list nomenclature is defined in Appendix (2.5), at the end of Chapter 2.

2.2.1 Generic Controls

2.2.1.1 EDITTEXT "pre-prompt", "post-prompt", "format", min, max, default, scale, var_name;

This keyword creates an edit box to be drawn in the PARAMETER EDITOR window. The box and its contents are formatted according to the items in the EDITTEXT keyword. "pre-prompt" and "post-prompt" are quoted strings which are displayed before and after the edit box created by PARSETUP on the screen. The "format" quoted string argument controls how the value is presented to the user in the edit box, and must be one of:

"%d" or "%i"	signed integer
"%U"	unsigned integer

"%o"	unsigned octal integer
"%x" or "%X"	unsigned hexadecimal integer
"%f"	fixed point float
"%e" or "%E"	scientific notation
"%g" or "%G"	whichever is the shorter of "%f" and "%e"

The min, max and default values should all be integers in PPL units. "min" and "max" limits the values entered by the user. If the user attempts to enter a value less than "min" into an edit box a warning message appears reporting the error and the user is required to re-enter a valid number. If the user attempts to enter a value greater than "max" into an edit box a warning message reporting the error appears and the use is required to re-enter a valid number. "default" generates the initial edit box value when the .PPL file is read in for the first time.

"scale" is a floating point value (which can be expressed either in fixed point or scientific notation, e.g., 1.234 or 1234E-3), which converts between the "user units" displayed in the edit box, and the "PPL units", which will eventually be used to initialize the PPL variable. The formula, PPL unit = edit box value X scale, is used for the conversion. This is employed to check that the PPL unit corresponding to the current edit box value lies between the min and max values. Any fractional part of the product above is lost when converting between floating point and integer.

"var_name" is the name of PPL variable which will be used to set to the PPL unit value corresponding to the edit box value.

2.2.1.2 SCROLLBAR "pre-prompt", "post-prompt", "format", min, max, default, scale, var_name;

This keyword creates a scroll bar in the PARAMETER EDITOR window, formatted according to the items following the keyword. The use of a scroll bar is particularly suitable to input the values, which may sensibly be thought of as analogue controls, and also prevents the user from entering invalid values. The keyboard's PgUp and PgDn keys move the scroll bar thumb over one tenth of its range. The left and right arrows move the thumb the equivalent of one PPL unit, so retaining fine control. The "pre-prompt" string appears to the left of the scrollbar, and the "post-prompt" string appears to the right. The format, var_name, min, max, default and scale values are all treated as shown in the EDITTEXT keyword.

2.2.1.3 VAR_ARRAY "pre-prompt", "post-prompt", "format", min, max, scale, array_name, no_elements [, default];

This keyword builds a button labelled "Array..." flanked by the "pre-prompt" and "post-prompt" strings. Activating this button, either by clicking upon it with the mouse cursor or by moving input focus to it and pressing the spacebar, causes a dialogue box to be displayed.

The dialogue box contains a list-box, which indicates each index and its corresponding value for each array element, and an edit-box, which allows any of the array values to be modified. The dialogue box also contains an "OK" button and a "Cancel" button. Pressing the OK button causes the (possibly modified) array values shown in the list-box to be stored by PARAMETER EDITOR in its internal data area, and then closes the dialogue box. Pressing the Cancel button causes the dialogue box to be closed without updating the array values stored by PARAMETER EDITOR in its internal data area; so that next time the "Array..." button is activated the old values will be displayed. When a parameter file is saved, the array values, which are written into the parameter file, are those that are stored by PARAMETER EDITOR in its internal data area.

The "pre-prompt" string appears to the left of the ARRAY ... button and describes the meaning of the parameter to the user, e.g. "Echo time"; the "post-prompt" string appears to the right of the ARRAY ... button, normally containing user units, e.g. "ms" or "cm".

The format, var_name, min, max, default and scale values are also treated as shown in the EDITTEXT keyword.

"array_name" defines the array variable as declared in the PPL program. The first no_elements of the array are initialised before the program runs. The elements are initialised to the values edited by the user, converted to PPL units by the factor scale. "no_elements" defines the number of elements of the array specified in the VAR_ARRAY argument list, which will be edited: the array declared in the PPL program must have at least no_elements elements.

[, default] allows to set up an optional list of default array values in PPL units. The first default value is assigned to array_name[0], the next to array_name[1] and so on. Those array elements, which do not have explicit default values defined, are set initially to the min value. The default value is used to generate the initial scrollbar thumb position when the .PPL file is read in for the first time.

Please note: VAR_ARRAY controls are not enabled in interactive setup mode, as PPL does not support arrays of the common int type.

2.2.2 Basic 1D transmission/Acquisition

2.2.2.1 **OBSERVE FREQUENCY "nucleus", min, max, default, ppl_var_MHz, ppl_var_kHz, ppl_var_Hz, ppl_var_rx1MHz;**

This keyword creates two adjacent boxes. One is a list-box containing text strings that are used to look up the centre frequency for that nucleus (Hz) in a pre-defined table (c:\windows\partfilio.ini). The second is an edit box into which an offset frequency (Hz) can manually be entered by users. The offset frequency is limited by "min" and "max", and initially set to "default". Each of these three values may be specified to 0.1 Hz resolution. The PPL variables ppl_var_MHz, ppl_var_kHz, ppl_var_Hz are sent the MHz, kHz, and Hz parts of the frequency information by SCAN.

The value in the "nucleus" quoted string must exactly match one of the text strings in the [nuclei] section of partfilio.ini, as described: Identifier = "Nucleus", Frequency, Gyromagnetic Ratio. The Gyromagnetic (GM) Ratio for a nucleus is defined as a fraction of the GM ratio for protons. If a GM ratio is not defined for any nucleus, it is assumed to be 1.0.

For example, the [nuclei] section of partfilio.ini:

```
[nuclei]
Nucleus1="H 1", 200000000, 1.0
Nucleus2="P 31", 80961000, 0.4051
```

Multi-receiver versions of the OBSERVE_FREQUENCY keyword exist. See the 'Multiple Receivers' topic for further details 2.2.8).

2.2.2.2 **DECOUPLE FREQUENCY "nucleus", min, max, default, ppl_var_MHz, ppl_var_kHz, ppl_var_Hz, ppl_var_rx1MHz;**

See the OBSERVE_FREQUENCY keyword.

2.2.2.3 **NO_SAMPLES min, max, default, var_name;**

This keyword creates an edit box with the pre-prompt "Number of Samples". The user may edit the initial value set by the variable "default", and the edited value is passed to the PPL variable "var_name", which must be used as the loop counter for the number of samples digitised. The number of samples is also used by the acquisition program.

Please note: the NO_SAMPLES control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running.

Multi-receiver versions of the NO_SAMPLES keyword exist. See the 'Multiple Receivers' help topic for further details.

2.2.2.4 **SPECTRAL_WIDTH min, max, default, var_name;**

This keyword produces a list-box with the pre-prompt "Spectral Width". The list-box contains the values defined within the [specwidth] section of the initialisation file (c:\windows\dsp.ini). The selected sample period is passed to the variable "var_name". The initialisation file (dsp.ini) has the following format:

```
[specwidth]
SW1="<label 1>", sample period 1
SW2="<label 2>", sample period 2
...
```

where the label is that displayed in the "Spectral Width" list-box in the parameter editor and the sample periods are defined in 100 ns units (10^7 /selected spectral width), for example, SW1="250 Hz 4.0 ms", 40000.

The initial list-box value selected is the one that matches the sample period within the list definition (in dsp.ini). If the list does not contain the default value, the default value is appended to the list-box, but may not be saved to the PPR file. The sample period is assigned to the variable "var_name" and passed to the PPL program. The sample period is used for the reconstruction and analysis programs to define the scale of the frequency axis, and for the calculation of field-of-view (FOV).

The SPECTRAL_WIDTH keyword also selects the appropriate digital filter for the selected spectral width, defined within the [dspnames] section in c:\windows\parfilio.ini (see DSP_ROUTINE keyword in 2.2.3.4 for further details).

Please note: the SPECTRAL_WIDTH cannot be controlled in interactive setup mode as its value is used by the acquisition routine, and in which the acquisition routine is running. Multi-receiver versions of the SPECTRAL_WIDTH keyword exist. See the 'Multiple Receivers' help topic for further details.

2.2.3 Optional 1D Acquisition

2.2.3.1 BUFFER_SIZE min, max, default;

This keyword creates an edit box with the pre-prompt "Buffer Size" and post-prompt "bytes". If the acquired data arrives at the DSP in the same order in time as that in which it appears in the data file (looping for multiple averages excluded), successive traces may be hatched together for transfer to PC and disc, up to a maximum value of the **BUFFER_SIZE** entry. Batching together traces into buffers of up to 16000 bytes will normally allow a higher overall throughput by reducing data transfer overheads; little benefit is achieved in batching traces beyond this point.

The size of the **BUFFER_SIZE** is an upper limit to the buffer length; and will be rounded down to the nearest integral multiple of the trace length. If the buffer size is less than the length of a trace, or if the keyword is not defined, batching will be disabled, and transfers will take place a single trace at a time.

For example, assume that the acquisition contains 64 views of 128 complex ints. Each trace will occupy 512 bytes. For buffer sizes between 0 and 1023 inclusive, each trace will be uploaded and written to disc as an individual unit. For buffer sizes of 1024 up to 1535 inclusive, data will be uploaded as 64 pairs of traces, occupying 1024 byte long buffers. For buffer sizes of 1536 to 2047 inclusive, data will be uploaded in 42 groups of 3 traces, occupying 1536 byte long buffers, followed by a single trace occupying 512 bytes of a 1536 byte long buffer. The entire image may be uploaded in a single buffer by declaring a buffer size of 32000 bytes, at the expense of requiring a larger amount of DSP memory to hold the buffer.

Batching is not implemented for multislice acquisitions, and will only operate in single view multi-echo acquisitions, due to the interleaving in the data file of data from different views between successive echoes.

2.2.3.2 DATA_TYPE default format code;

This keyword creates a list-box with the pre-prompt "Data Type", containing the allowed formats for data in the output file. The default data format is complex float (0x15) if this keyword is not defined. Format codes are as defined for the data file format, i.e.

<u>Code</u>	<u>Description</u>
0x15 =	complex float (2x4 bytes per data point)
0x13 =	complex int (2x2 bytes per data point)
0x11 =	complex char (2x1 bytes per data point)
0x05 =	real float (1x4 bytes per data point)
0x03 =	real int (1x2 bytes per data point)
0x01 =	real char (1x1 bytes per data point)

All data accumulation is performed internally within the acquisition routine in complex floating point format; the conversion to the output format only being executed immediately prior to offloading to the PC and disc file. Char data types take the MOST significant 8 bits of the 16 bit input data value.

Please note: the DATA_TYPE control is not enabled in interactive setup mode as its value is used by the acquisition routine, and cannot be changed while the acquisition routine is running.

2.2.3.3 *DISCARD min, max, default, ppl_var_name;*

The DISCARD keyword generates a scrollbar with limits “min” and “max”, initial scrollbar thumb position “default”, pre-prompt “Digital Filter Discard”, post-prompt “points”.

The DISCARD keyword provides a value, which is used by the DSP acquisition routine to determine how many data points from the start of the sample to ignore, therefore it cannot be defined as a common int, i.e. it cannot be adjusted interactively during set-up mode.

If the keyword DISCARD is not present, the DISCARD value defaults to **2** (Note this is different to all other defaults which are 1). If it is present, the user-selected value is passed to the DSP acquisition program. The DSP acquisition program always discards the first two data points, which will be buffered in the interface board, before digitally filtering the rest of the data (any users who write their own DSP program will also have to remove these two points). It then takes the **DISCARD** argument (passed via SCAN), and discards that number of points after digital filtering, i.e. the number of points discarded is the number of output points from the digital filter, not the number of input points. Users writing their own programs may or may not include this operation.

Please note: the default will be to discard two points before digital filtering, and two points after. The user has the option to either increase or decrease the number of points to be discarded after filtering, but has no control over the points discarded before filtering.

2.2.3.4 *DSP_ROUTINE "dsp_program";*

This keyword produces a list box, with pre-prompt "DSP Routine", containing a list of the dsp routines defined in the initialisation file (c:\windows\parfilio.ini). The list must be defined within a [dspnames] section as follows:

```
[dspnames]
```

```
dsp1="dsp program 1", "initialisation filename 1"
```

```
dsp2="dsp program 2", "initialisation filename 2"
```

```
...
```

where “dsp program” is the pathname of the acquisition program that runs on the DSP board and that is to be used with the PPL program, and “initialisation filename” is the pathname (e.g., dsp.ini) of the file containing the spectral width definitions (see 2.2.2.4 for SPECTRAL_WIDTH).

The default list-box selection is that whose program name matches the “dsp_program” string within the keyword definition. Note that if the given dsp_program cannot be found in the list of program names, a match will be attempted on the base filenames, i.e. without the full pathname. If the defined dsp_program still cannot be found, the defined name is appended to the list-box and selected by default. This value may not then be saved to the PPR file.

Please note: this control is not enabled in interactive setup mode, as the acquisition program cannot be changed whilst it is running.

The acquisition file DSP implements 6-pole Butterworth filters (as previously), with a 1dB (10% signal amplitude) roll-off at the Nyquist frequency, falling to -3dB at 123% and -20dB at 164% of the Nyquist frequency. The available filters for this routine are defined in the file DSP.INI.

The acquisition file DSPB8 implements 8-pole Butterworth filters, with a -1dB (10% signal amplitude) roll-off at the Nyquist frequency, falling to -3dB at 116% and -20dB at 144% of the Nyquist frequency. The available filters for this routine are defined in the file DSP.INI.

The acquisition file DSPB8x2 implements 8-pole Butterworth filters, but with a reduced bandwidth optimised for x2 oversampling. The Nyquist and filter bandwidths have been chosen such that the roll-off at half the Nyquist frequency is less than -0.05dB (0.5% signal amplitude) and the attenuation at three-halves of Nyquist (at which aliasing would become relevant) is greater than -46dB (99.5% signal amplitude). The acquisition file DSPB6x2 implements the corresponding filter and Nyquist frequencies using 6-pole Butterworth filters. The available filters for these routines are defined in the file DSPBX2.INI.

The acquisition file DSPE8 implements 8-pole Elliptic filters, with 3dB (50% signal amplitude) roll-off at the Nyquist frequency. The transition band has been narrowed such that the roll-off is less than -0.05dB (0.5% of signal amplitude) within 95% of Nyquist, and the attenuation exceeds -56dB (99.84% of signal amplitude) outside 115% of Nyquist (attenuation is -20dB at 105% of Nyquist, and -36dB at 110% of Nyquist). The available filters for this routine are described in the file DSPE.INI.

2.2.3.5 NO_AVERAGES min, max, default, var_name;

This keyword creates an edit box with the pre-prompt “Number of Averages”. The user may be able to edit the initial value “default”, and the edited value is passed to the PPL variable “var_name”, which must be used as the loop counter for the number of averages performed. The number of averages is also used by the acquisition program. In the absence of this keyword, the acquisition program uses a default value of 1.

In experiments where the keywords VIEW_BLOCK or SLICE_BLOCK (see 2.2.6.2 and 2.2.6.3) are used, and the NO_AVERAGES is not an exact multiple of both block sizes, the standard acquisition routine will expect more than the requested number of averages. The total number acquired is the smallest multiple of SLICE_BLOCK, which is not less than the smallest multiple of VIEW_BLOCK, which is not less than NO_AVERAGES. Consider the following two examples. If an experiment were to be conducted with:

NO_AVERAGES	8;
VIEW_BLOCK	4;
SLICE_BLOCK	2;

A total of 8 averages would be performed; 8 being the smallest multiple of 2 (SLICE_BLOCK) which is not less than 8, which in turns is the smallest multiple of 4 (VIEW_BLOCK) not less than 8 (NO_AVERAGES).

If an experiment were to be conducted with:

NO_AVERAGES	11;
VIEW_BLOCK	5;
SLICE_BLOCK	4;

A total of 16 averages would be performed; 16 being the smallest multiple of 4 (SLICE_BLOCK) which is not less than 15, which is the smallest multiple of 5 (VIEW_BLOCK) not less than 11 (NO_AVERAGES). On saving data to the output file the number of averages actually performed is recorded after the keyword: COMPLETED_AVERAGES.

Please note: the NO_AVERAGES control is not enabled in interactive setup mode as its value is used by the acquisition routine, and cannot be changed while the acquisition routine is running. Multi-receiver versions of the NO_AVERAGES keyword exist. See the 'Multiple Receivers' help topic (See 2.2.8) for further details.

2.2.3.6 PHASE_CYCLE min, max, default, var_name;

This keyword creates a list-box, from which the user would choose "None", "DC correct", or "CYCLOPS". Legal values for min, max and default and their corresponding actions are listed below:

"None"	1
"DC correct"	2
"Cyclops"	4

If the min, max and default values are all set to the same value, the user is forced to accept the choice of phase cycling determined by the author of the PPL program. The PPL variable "var_name" must be used to control the phase cycle of the transmitted rf. The acquisition program re-directs the received signal as follows:

"None"	Successive FIDs are simply accumulated.
"DC correct"	Successive FIDs alternately added and subtracted. If the phase of the transmitted rf is arranged also to alternate then the signal will accumulate whilst any DC offset will cancel out.
"CYCLOPS"	The FID data is rotated according to the CYCLOPS scheme repeated continuously.

	Accumulated data array	
	Real	Imaginary
Scan 1	+ Channel 1	+ Channel 2
Scan 2	+ Channel 2	- Channel 1
Scan 3	- Channel 1	- Channel 2
Scan 4	- Channel 2	+ Channel 1

If the phase of the transmitted rf is rotated through the four angles 0°, 90°, 180°, and 270° in a simple pulse-and-collect sequence, the signal accumulates whilst DC offsets and other artefacts may cancel out.

If this keyword is not present the phase cycle "None" is used by the acquisition program. Note that the phase cycling control is never enabled in interactive setup mode, as the phase cycling mode of the acquisition routine is determined at start-up time and cannot be changed once the sequence is running. Multi-receiver versions of the PHASE_CYCLE keyword exist. See the 'Multiple Receivers' help topic (See 2.2.8) for further details.

2.2.4 Basic Imaging

2.2.4.1 GRADIENT_STRENGTH *grad_var*;

This keyword enables the gradient strengths used at the time of the experiment to be saved to the output file. "grad_var" is a four-element integer array PPL variable, which will be initialized with the minimum value of the physical gradient strengths defined in c:\windows\parfilio.ini, e.g., GXMax, GYMax, GZMax.

If the OBSERVE_FREQUENCY keyword is defined, "grad_var" contains the minimum defined field strength multiplied by the gyromagnetic ratio for the currently selected nucleus. The unit of "grad_var" are therefore Hz mm⁻¹ (See 2.2.2.1 for further details on the OBSERVE_FREQUENCY keyword).

The gradient strengths are assumed to be isotropic, and the minimum value is used in calculating the limits of FOV, SLICE_THICKNESS and FOV_READ/PHASE/SLICE_OFF. This keyword does not create a visible control in PARSETUP.

2.2.4.2 *FOV read_seq, phase_seq, phase_time, read_var, phase_var ;*

This keyword initiates the field of view calculations for the read and phase encoding directions. The field of view is assumed to be square. PARAMETER EDITOR draws an edit box with the pre-prompt "Field of View" and the post-prompt "mm". The gradients will be adjusted as necessary to produce an image of the specified size. An attempt to set a field of view, which the instrument cannot achieve, will be detected, and a correct number must be entered.

"read_seq" is the numerical value of the read gradient stored in the waveform storage board at the time of the data acquisition. Similarly "phase_seq" is the numerical value of the phase encoding gradient at the time of the phase encoding delay. "phase_time" is the effective duration of the phase encoding delay. This may not be exactly equal to the delay time, depending on gradient response times and eddy current effects.

"read_var" is a PPL variable which will contain a number calculated according to the field of view, and must be used as the read gradient multiplying DAC value. Similarly the PPL variable here illustrated by "phase_var" must be used as the multiplying DAC value for the phase encode gradient.

Both "read_var" and "phase_var" must be declared as common ints for the interactive FOV control to be enabled. In addition, the variables associated the FOV_READ_OFF, FOV_PHASE_OFF, READ_SELECTION and PHASE_SELECTION keywords (if defined) must also be declared as common ints. It might be included in c:\smis\include\var_15.pph.

For use in volume-selection experiments, this keyword may be defined with values of 0 for either "read_seq" or "phase_seq", or both. The FOV will then be defined purely by the READ_SELECTION or PHASE_SELECTION keywords (see 2.2.7), and the "read_var" and "phase_var" variables will be initialised with values of 0.

- **FOV_READ_OFF min, max, default, scale, var_name ;**
- **FOV_PHASE_OFF min, max, default, scale, var_name ;**
- **FOV_SLICE_OFF min, max, default, scale, var_name ;**

These keywords create three edit boxes allowing user definition of the centre of the image offset from the magnet iso-centre in the read/phase/slice directions. The edit boxes have the pre-prompts "FOV Offset - X", "FOV Offset - Y" and "FOV Offset - Z", and the post-prompt "mm". There are two options for using these keywords:

All the three keywords are present in the PPL program.

In this case the position of the centre of the image can be defined independently of the image orientation. "FOV Offset - X", "FOV Offset - Y" and "FOV Offset - Z" will therefore be independent from each other and will be set to the *physical* coordinates of the image centre, based on the required offsets in the *image* phase/read and slice directions. If the image orientation changes the image centre stays the same.

Just one of the three keywords is present in the PPL program.

In this case the offset of the slice is defined along one *image* direction only and the "FOV Offset - X", "FOV Offset - Y" and "FOV Offset - Z" are now dependent of each other and of the image orientation. For example, if the users define FOV_SLICE_OFF, but not the read and phase corresponding keywords, the user(s) will still see three edit boxes in the parameter editor. In this case, though the user will be allowed to make changes to the offset along the slice direction only, i.e. along the normal to the slice, passing through the origin. Changes of the image orientation will result in changes of the numbers displayed in the edit boxes, according to the *physical* coordinates of the slice offset chosen.

"min", "max" and "default" are integer values used to set the limits and initial value of the offset in PPL units. The PPL unit defines the offset as a fraction of the field of view. The scale value is a floating point value, which is used to convert between the offset displayed in the edit box ("user units"), and the PPL units, which will eventually be used to initialise the PPL variable. The formula used to convert between PPL units and user units is as follow:

$$\text{PPL unit} = \text{scale} * \text{edit box value} / \text{field of view.}$$

This formula is also employed to check that the PPL unit corresponding to the current edit box value lies between the "min" and "max" values. "var_name" is the name of a PPL variable used to contain the value of the offset. This keyword should be used in conjunction with the FOV_PHASE_OFF and FOV_SLICE_OFF keywords to define the offsets in the x, y and z directions. Note that if field of view offsets are required, either one or all three directions must be defined.

If multi-angle oblique orientation is defined (using the MULTI_ORIENTATION keyword), the offsets are defined per slice. If the number of slices is greater than one, buttons labelled "Slice" replaces the offset edit boxes. Selecting any of these buttons displays the Slice Parameters dialog box, which enables all slice-specific parameters to be defined. This includes the orientation for each slice.

2.2.4.3 SLICE_THICKNESS slice_val, rf_bandwidth, slice_var;

This keyword creates an edit box with the pre-prompt "Slice Thickness" and the post-prompt "mm". It is used to control the field gradient strength at the time of the selective rf pulse used for slice selection in imaging experiments.

"slice_val" is the numerical value of the stored slice select gradient waveform at the time of the slice selective pulse. For the standard library sequences, this is the maximum value, i.e. 2047. "rf_bandwidth" is the excitation bandwidth, in Hz, of the selective pulse. "slice_var" is the name of a PPL variable which contains the value for the analogue multiplying DAC of the slice select gradient. The value contained in this variable is calculated by PARAMETER EDITOR from the calibrated gradient strengths. See also the keywords SLICE_ORIENTATION and FOV.

PARAMETER EDITOR calculates the minimum and maximum possible slice thicknesses from the known gradient values. Note that the SLICE_THICKNESS control is disabled in interactive setup mode as these values cannot be changed whilst a sequence is running. In addition, the control is enabled only if the variables associated the SLICE_SEPARATION and FOV_SLICE_OFF keywords (if defined) are declared as common ints (see 2.2.4.4 and 2.2.7).

2.2.4.4 SLICE_SEPARATION min, max, default, var_Hz;

This keyword creates an edit box similar to that for slice thickness, with the pre-prompt "Slice Separation" and the post-prompt "mm". It is used to calculate an offset frequency, which will move the centre of the slice selective rf pulse the specified distance, at the gradient values calculated in response to the SLICE_THICKNESS keyword.

"default" is the initial frequency offset. "var_Hz" is the name of a PPL variable used to contain the value of the offset. The maximum possible offset would be 32767, and the minimum -32768. The PPL program must explicitly add this offset to the transmitted frequency before the pulse, and then return to the original frequency prior to acquiring data. These functions are performed by the high-level PPL functions multislice() and slice().

The keyword SLICE_THICKNESS must be present in any program that uses SLICE_SEPARATION.

2.2.4.5 ORIENTATION min, max, default, var_name ;

This keyword produces a list box containing up to six combinations of slice orientation, each corresponding to an integral index. The initial orientation will correspond to the index chosen for default. The range of values available for the index will depend upon the values of min and max.

If the PPL program author would like to restrict a user to a particular orientation, it can be done by setting the min, max and default values all to the same value. For example, the entry ORIENTATION 3, 3, 3, orient_val in the PPL PARAMLIST would restrict the user to the "viewed along Y, phase encode horizontal" orientation (see the table below), and the PPL variable orient_val would be initialised to 3.

The index number for the final orientation selected will be passed to the PPL variable `var_name`. The waveforms stored in the image sequencer boards are rotated and scaled to correspond to the selected orientation, according to the table below.

Plane	Viewed along	Phase encode direction	Seq. Edit Channel			Index
			Read	Phase	Slice	
Transverse	X	Vertical	X	Y	Z	0*
Transverse	X	Horizontal	Y	X	Z	1
Coronal	Y	Vertical	X	-Z	Y	2
Coronal	Y	Horizontal	-Z	X	Y	3
Sagittal	Z	Vertical	-Y	-Z	X	4
Sagittal	Z	Horizontal	-Z	-Y	X	5

* Default in the absence of the ORIENTATION keyword.

The program author must ensure that the gradient multiplying factors (see FOV) are also rotated the same way (though not inverted). The variable `var_name` must be used for this purpose.

Please note: the orientation control is only enabled in interactive setup mode if the variable **`var_name`** is declared as common int.

2.2.4.6 ***OBLIQUE_ORIENTATION min, max, default, phase_var, x_angle, y_angle, z_angle;***

This keyword produces a list box containing a number of predefined image orientations, referenced from an index into the list (see below). The initial orientation displayed will correspond to the index chosen for default. The range of values available for the index will depend upon the values of min and max. If the PPL program author would like to restrict a user to a particular orientation, it can be done by setting the min, max and default values all to the same value, as for the ORIENTATION keyword.

In addition to the orientation list-box, three edit boxes are created for entry of the x, y and z rotation angles. If a predefined orientation is selected within the list-box, these edit boxes are disabled, and display the appropriate angles. The boxes are enabled only if the oblique orientation is selected. The angles are passed to the PPL variables `x_angle`, `y_angle` and `z_angle`, in 1/10 degree units.

The contents of the orientation list-box are defined in c:\windows\parfilio.ini. This list may be modified if additional predefined rotations are required, new items being appended to the list.

The default list is:

List-box text	x_angle	y_angle	z_angle
Oblique	0.0	0.0	0.0
Transverse	0.0	0.0	0.0
Coronal	- 90.0	0.0	0.0
Sagittal	0.0	90.0	- 90.0

The phase encode directions are added to the list before the list-box is displayed. The phase direction is passed to the PPL variable `phase_var`, with a value of 0 for vertical and 1 for horizontal phase encode. For example, the entry: `OBLIQUE_ORIENTATION 0, 2, 1, phase, x_angle, y_angle, z_angle`; in the PPL PARAMLIST displays the transverse orientation by default (phase encode vertical), and restricts the user to the oblique, transverse and coronal orientations.

The `OBLIQUE_ORIENTATION` keyword is mutually exclusive with the `ORIENTATION` keyword. Note that the orientation control is enabled in interactive setup mode only if the `MR3040` keyword is defined in the `parfilio.ini` file:

```
[Config]
MR3040=1
```

Each of the angle edit fields is enabled in interactive setup mode if its PPL variable is defined as a common int. In order for the orientation list-box to be enabled, all three angle variables must be thus defined. In addition, none of these controls will be enabled if the variables associated with the following keywords (if defined) are not also defined as common ints:

```
FOV
FOV_READ_OFF
FOV_PHASE_OFF
FOV_SLICE_OFF
SLICE_THICKNESS
SLICE_SEPARATION
```

2.2.4.7 **MULTI_ORIENTATION** *min, max, default, phase_var, x_angle_array, y_angle_array, z_angle_array* ;

MULTI_ORIENTATION can be used instead of OBLIQUE_ORIENTATION, allowing to be in multi-angle oblique mode, i.e. to have independent control of the orientation of each slice. This keyword produces a list box containing a number of predefined image orientations, referenced from an index into the list (see below). The initial orientation displayed will correspond to the index chosen for "default". The range of values available for the index will depend upon the values of "min" and "max": if the PPL programs author wishes to restrict the user to a particular orientation, s/he may do so by setting the "min", "max" and default values all to the same value, as for the ORIENTATION keyword.

In addition to the orientation list-box, three edit boxes are created for entry of the x, y and z rotation angles. If a predefined orientation is selected within the list-box, these edit boxes are disabled, and display the appropriate angles. The boxes are enabled only if the oblique orientation is selected. The angles are passed to the PPL variables x_angle_array, y_angle_array and z_angle_array, in 1/10 degree units.

The contents of the orientation list-box are defined in the initialisation file "parfilio.ini", located in the Windows directory. This list may be modified if additional predefined rotations are required, new items being appended to the list. The default list is:

List-box text	x_angle	y_angle	z_angle
Oblique	0.0	0.0	0.0
Transverse	0.0	0.0	0.0
Coronal	- 90.0	0.0	0.0
Sagittal	0.0	90.0	- 90.0

The phase encode directions are added to the list before the list-box is displayed.

The phase direction is passed to the PPL variable phase_var, with a value of 0 for vertical and 1 for horizontal phase encode.

As this keyword defines the image orientation for each slice, if the number of defined slices is greater than one, the orientation list-box and angle edit boxes are replaced by buttons labelled "Slice". Selecting any of these buttons displays the Slice Parameters dialog box, which enables all slice-specific parameters to be defined. This includes the field of view offsets for each slice. Note that all slices must be defined with the same phase orientation, and so the orientation list-box within the dialog box does not include

phase directions. The phase for all slices is selected via a separate radio button within the dialog.

For example, the entry:

```
MULTI_ORIENTATION 0, 2, 1, phase, x_angle, y_angle, z_angle;
```

in the PPL PARAMLIST displays the transverse orientation by default (phase encode vertical) for all slices, and restricts the user to the oblique, transverse and coronal orientations.

The MULTI_ORIENTATION keyword is mutually exclusive with the OBLIQUE_ORIENTATION and ORIENTATION keywords.

When using MULTI_ORIENTATION the keyword, the value entered under SLICE_SEPARATION is used to provide the default spacing of the created/aligned slices. If no SLICE_SEPARATION is defined, the default separation is set to the current slice thickness. The SLICE_SEPARATION value is not used other than when creating new slices or aligning an existing group of slices. Note that the multi-angle obliquing is enabled in interactive setup mode only if the MR3040 keyword is defined in the initialisation file parfilio.ini:

```
[Config]
MR3040=1
```

The keyword is not available in interactive setup mode, as PPL does not support common int arrays.

2.2.5 Basic Multi-Dimensional Acquisition

2.2.5.1 NO_ECHOES min, max, default, var_name ;

This keyword creates an edit box with pre-prompt "Number of Echoes". This value is used by the acquisition program and is passed to the PPL variable "var_name". This defines to the standard acquisition routine the number of echoes collected in a multi-echo experiment. In the absence of this keyword, the acquisition program uses a default value of 1.

The initial number of echoes is "default", and the user entry is limited by "min" and "max". Entering equal values for "min", "max" and "default" prevents the user from changing the number of echoes. The PPL variable whose name is given as the fourth parameter following the NO_ECHOES keyword must be used in the PPL program to control the number of echoes. The acquisition program expects the loop over echoes to occur immediately outside the loop for number of samples (see NO_SAMPLES in 2.2.2.3), and inside the loop for averaging at the slice level (see SLICE_BLOCK in 2.2.6.2).

2.2.5.2 NO_SLICES min, max, default, var_name;

This keyword creates an edit box with the pre-prompt "Number of Slices". This value is used by the acquisition program and is passed to the PPL variable `var_name`. The initial number of slices is default, and the user entry is limited by min and max. This causes the standard acquisition routine to accumulate a number of sets of data equal to the entered value before moving on to the increment/view in a multi-dimensional experiment. The PPL variable whose name is given as the fourth parameter following the NO_SLICES keyword must be used in the PPL program to control the looping structure. In the absence of this keyword the acquisition routine defaults the number of slices to 1 (See also SLICE_BLOCK, VIEW_BLOCK in 2.2.6.2 and 2.2.6.3). Note that the NO_SLICES control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running.

Multi-receiver versions of the NO_SLICES keyword exists. See the 'Multiple Receivers' help topic for further details.

2.2.5.3 NO_VIEWS min, max, default, var_name ;

This keyword creates an edit box with pre-prompt "Number of Views" which allows the user to enter the number of views (phase encoding steps in an imaging sequence, time increments in a 2D spectroscopy sequence). The initial number of views is "default", and the user's entries are limited by "min" and "max". Entering equal values for "min", "max", and "default" prevents the user from changing the number of views. The PPL variable, "`var_name`" must be used in the PPL program to control the number of views. In the absence of this keyword, the acquisition program uses a default value of 1.

In imaging applications, if the FOV keyword is used the NO_VIEWS keyword must also be specified somewhere in the PPL PARAMLIST header, as the number of views is required in the FOV scaling calculations. Note that the NO_VIEWS control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running.

A multi-receiver version of the NO_VIEWS keyword exist. See the 'Multiple Receivers' section of this manual 2.2.8) for further details.

2.2.5.4 NO_VIEWS_2 min, max, default, var_name ;

This keyword creates an edit box with pre-prompt "Number of Views 2". This value is used by the acquisition program and is passed to the PPL variable "`var_name`". This defines the number of secondary views (phase encoding steps in an imaging sequence, time increments in a 3D spectroscopy sequence) in a three-dimensional experiment.

The initial number of views is the values set as "default", and the user's entries are limited by "min" and "max". Entering equal values for "min", "max", "default" prevents the user from changing the number of secondary views. The PPL variable, "var_name" must be used in the PPL program to control the number of secondary views. In the absence of this keyword, the acquisition program uses a default value of 1. Note that the NO_VIEWS_2 control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running.

A multi-receiver version of the NO_VIEWS_2 keyword exists. See the 'Multiple Receivers' section of this manual (2.2.8) for further details.

2.2.5.5 *EXPERIMENT_ARRAY min, max, default, var_name;*

This keyword creates an edit box with the pre-prompt "Experiment Array Size". This value is used by the acquisition program and is passed to the PPL variable "var_name". This causes the standard acquisition routine to build up an array of experiment data, the number of experiments in the array being equal to the entered value passed to "var_name". In the absence of this keyword, the acquisition program uses a default value of 1. "min", "max" and "default" are all integers that define the minimum, maximum and default value.

Data is accumulated to the end of the current block of averages (see VIEW_BLOCK in 2.2.6.3) of each simple experiment, for all increments/views of a two-dimensional experiment if appropriate, before moving to the next member of the array.

This keyword can be used in combination with the VAR_ARRAY keyword (in 2.2.1.3) to construct pulse sequences for relaxation experiments. In the absence of this keyword the standard acquisition routine defaults the experiment array size to 1. Note that the EXPERIMENT_ARRAY control is not enabled in interactive setup mode as the value set by this control is used by the acquisition program and cannot be changed while the acquisition program is running.

A multi-receiver version of the EXPERIMENT_ARRAY keyword exists. See the 'Multiple Receivers' section of this manual (2.2.8) for further details.

2.2.6 *Extended Multi-Dimensional Acquisition*

2.2.6.1 *VIEWS_PER_SEGMENT min, max, default, var_name;*

This keyword creates an edit box with pre-prompt "Views per Segment". This value is used by the acquisition program and is passed to the PPL variable "var_name".

This defines the number of primary views (phase encoding steps in an imaging sequence, time increments in a 3D spectroscopy sequence), which are incremented within a single

segment of a segmented acquisition (e.g. in segmented-EPI). These increments are applied outside the main echo loop, but within the averaging loop provided by SLICE_BLOCK (see below). In the absence of this keyword, the acquisition program uses a default value of 1.

The initial number of views per segment is "default", and the user entry is limited by "min" and "max". Entering equal values for "min", "max" and "default" prevents the user from changing the number of views per segment.

VIEWS_PER_SEGMENT may also be used in certain 3D experiments to allow interchange and intermixing of the order of primary and secondary phase encoding steps. All primary phase encoding may be performed between the echo and the slice loops by setting the VIEWS_PER_SEGMENT value equal to that of NO_VIEWS. Entering a value of 0 causes the system treat the VIEWS_PER_SEGMENT value as equal to that of NO_VIEWS.

Please note: the VIEWS_PER_SEGMENT control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running. A multi-receiver version of the VIEWS_PER_SEGMENT keyword exists. See the 'Multiple Receivers' section of this manual (section 2.2.8) for further details.

2.2.6.2 SLICE_BLOCK min, max, default, var_name;

This keyword creates an edit box with the pre-prompt "Block Size (Slice)". This value is used by the acquisition program and is passed to the PPL variable "var_name". The initial slice block size is "default", and the user's entries are limited by "min" and "max". This causes the standard acquisition routine to accumulate a number of averages equal to the entered value before moving on to the next slice of a multi-slice experiment. The PPL variable whose name is given as the second parameter following the SLICE_BLOCK keyword must be used in the PPL program to control the looping structure. In the absence of this keyword the acquisition routine defaults the view block size to 1. (See also VIEW_BLOCK below).

Please note: the SLICE_BLOCK control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running. A multi-receiver version of the SLICE_BLOCK keyword exists. See the 'Multiple Receivers' section of this manual (section 2.2.8) for further details.

2.2.6.3 VIEW_BLOCK min, max, default, var_name;

This keyword creates an edit box with the pre-prompt "Block Size (View)". This value is used by the acquisition program and is passed to the PPL variable "var_name". The initial

view block size is “default”, and the user’s entries are limited by “min” and “max”. This causes the standard acquisition routine to accumulate a number of averages equal to the entered value before saving the data to disc and moving on to the next view of a two-dimensional or arrayed experiment. In the absence of this keyword the acquisition routine defaults the view block size to the value entered for NO_AVERAGES. (See also SLICE_BLOCK, 2.2.6.2).

Please note: the VIEW_BLOCK control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running. A multi-receiver version of the VIEW_BLOCK keyword exists. See the 'Multiple Receivers' section of this manual (section 2.2.8) for further details.

2.2.6.4 BATCH_EXPERIMENTS min, max, default, var_name;

This keyword creates an edit box with the pre-prompt "Experiment Batch Size". This value is used by the acquisition program and is passed to the PPL variable “var_name”. This should be used in conjunction with the EXPERIMENT_ARRAY keyword.

“min”, “max” and “default” are all integers, which define the minimum, maximum and default values for this keyword, and “var_name” is the name of the PPL variable, which will be used to indicate the number of elements in the array of experiments.

This causes the standard acquisition routine to subdivide the acquisition of multi-phase data into batches, the number of experiments in the batch being equal to the entered value. In the absence of this keyword, the acquisition program uses a default value of 1. Setting a value of zero will cause all experiments defined using the EXPERIMENT_ARRAY keyword to be acquired in a single batch.

Data is accumulated to the end of the current block of averages (see VIEW_BLOCK above) of each simple experiment, for all slices and increments/views of a two-dimensional experiment if appropriate, before moving to the next member of the array. If used in combination with the BATCH_SLICES keyword (below), the batch of experiments is completed for the current batch of slices before moving on to the next batch of slices.

This keyword can be used in combination with the VAR_ARRAY keyword (2.2.1.3) to construct pulse sequences for relaxation experiments. In the absence of this keyword the standard acquisition routine defaults the experiment array size to 1.

Please note: the BATCH_EXPERIMENTS control is not enabled in interactive setup mode as the value set by this control is used by the acquisition program and cannot be changed while the acquisition program is running. A multi-receiver version of the BATCH_EXPERIMENTS keyword exists. See the 'Multiple Receivers' section of this manual (2.2.8) for further details.

2.2.6.5 *BATCH_SLICES min, max, default, var_name;*

This keyword creates an edit box with the pre-prompt "Slices per Batch". This value is used by the acquisition program and is passed to the PPL variable "var_name". The initial number of slices is default, and the user entry is limited by "min" and "max".

This causes the standard acquisition routine to accumulate a number of slices of data equal to the entered value before moving on to the next phase-encode-increment/view in a multi-dimensional experiment (NO_VIEWS). Once the full set of experiments has been acquired, a new batch will be started to acquire any further slices required.

In the absence of this keyword, or if a value of zero is entered, the acquisition routine defaults the number of slices per batch to equal the total number of slices defined, so acquiring all slices in a single batch (See also NO_SLICES, SLICE_BLOCK, VIEW_BLOCK, respectively 2.2.5.2, 2.2.6.2, and 2.2.6.3).

Please note: the BATCH_SLICES control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed while the acquisition routine is running. A multi-receiver version of the BATCH_SLICES keyword exists. See the 'Multiple Receivers' section of this manual (2.2.8) for further details.

2.2.6.6 *SLICE_INTERLEAVE min, max, default, var_name;*

This keyword creates an edit box with the pre-prompt "Slice Interleave" and no post-prompt. The initial number is default. The user is allowed to select an integer between "min", and "max", or the number of slices (see NO_SLICES in 2.2.5.2) if this is smaller than "max". If the BATCH_SLICES keyword is present and its value is non-zero, this keyword controls ordering within a single batch of slices, and the upper limit is set to the lower of max and the value of BATCH_SLICES (above).

SCAN will re-organise the collected data according to the interleave number. If the interleave number is 1, slices will be stored sequentially. Higher numbers can cause the slices to be shuffled before storing, re-organising the data so that an experiment that excites every nth slice will still result in sequential data. Entering a value of zero will enable centre-out ordering.

See the following table for some examples:

Slice Interleave	Number of Slices	Acquire in sequence
1	8	1 2 3 4 5 6 7 8

2	8	2 4 6 8 1 3 5 7
3	8	3 6 2 5 8 1 4 7
2	4	2 4 1 3
0	8	5 4 6 3 7 2 8 1
0	7	4 3 5 2 6 1 7

It is the responsibility of the pulse program author to ensure that data is collected in the appropriate order. The necessary actions are included in the PPL library functions `multislice()`, `slice()` and `ordertimetopos()`.

In the absence of the `SLICE_INTERLEAVE` keyword, a slice interleave of 1 (sequential) is used.

Please note: the `SLICE_INTERLEAVE` control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed whilst the acquisition routine is running. A multi-receiver version of the `SLICE_INTERLEAVE` keyword exists. See the 'Multiple Receivers' section of this manual (section 2.2.8) for further details.

2.2.6.7 `BATCH_INTERLEAVE min, max, default, var_name;`

This keyword creates a scroll bar with the pre-prompt "Batch Interleave" and no post-prompt. The user is allowed to select an integer between "min", and "max", or the number of batches (see `BATCH_SLICES` above) if this is smaller than "max". The initial number is "default".

SCAN will re-organise the collected data according to the interleave number. If the interleave number is 1, batches will be stored sequentially. Higher numbers can cause the slices to be shuffled before storing, re-organising the data so that an experiment that excites every *n*th batch of slices will still result in sequential data. Ordering within a single batch is controlled by the `SLICE_INTERLEAVE` keyword. A value of zero allows centre-out ordering of batches. See the table (above) as shown in `SLICE_INTERLEAVE`.

It is the responsibility of the pulse program author to ensure that data is collected in the appropriate order. The necessary actions are included in the PPL library functions `batchslice()`, `slice()` and `batchtimetopos()`.

In the absence of the BATCH_INTERLEAVE keyword a slice interleave of 1 (sequential) is used.

Please note: the BATCH_INTERLEAVE control is not enabled in interactive setup mode as its value is used by the acquisition routine and cannot be changed whilst the acquisition routine is running. A multi-receiver version of the BATCH_INTERLEAVE keyword exists. See the 'Multiple Receivers' section of this manual (section 2.2.8) for further details.

2.2.6.8 *SAMPLE_PERIOD_2 min, max, default, divisor, var_name;*

SAMPLE_PERIOD_2 creates an edit box in the Parameter Editor window with the pre-prompt "Sample Period 2" and the post-prompt "us". SAMPLE_PERIOD_2 entry is all in units of 100 ns. For example, when the value "5.1 us" is displayed as the entry in the sample_period_2 edit box, the value used to initialize the PPL variable var_name is 51.

"SAMPLE_PERIOD_2" allows the user to set the contents of the PPL variable "var_name" to an integer limited by "min" and "max", with the initial value "default". Only values exactly divisible by the value of "divisor" are acceptable; other user entries are adjusted to the nearest acceptable value.

The variable "var_name" must be used to define the ADC sampling interval. The "divisor" is provided to allow the sample period to be split, for example, to insert pulses between acquired data points.

The sample period is used by the reconstruction and analysis programs to define the scale of the frequency axis, and by the field-of-view keyword FOV. The keyword FOV can only be used when the sample period is fixed at a constant value, by using the same value for min, max, and default.

Please note: one or other of the keywords SAMPLE_PERIOD_2 and SWEEP_WIDTH_2 (below), but not both, may be included in two-dimensional experiments to set the time increment for the second dimension.

A multi-receiver version of the SAMPLE_PERIOD_2 keyword exists. See the 'Multiple Receivers' section of this manual (2.2.8) for further details.

2.2.6.9 *SWEEP_WIDTH_2 min, max, default, divisor, var_name;*

"SWEEP_WIDTH_2" creates an edit box with the pre-prompt "Sweep Width 2" and the post-prompt "Hz". The number passed to the PPL variable is 10e7/user value, to convert between the user units of Hz and the PPL units of 100 ns. The user value is checked for the PPL unit limits "min" and "max", and also for exact conversion to an integer.

SWEEP_WIDTH_2 allows the user to set the contents of the PPL variable “var_name” to an integer limited by “min” and “max”, with the initial value default. Only values exactly divisible by divisor are acceptable; other user entries are adjusted to the nearest acceptable value.

The variable “var_name” must be used to define the ADC sampling interval. The “divisor” is provided to allow the sample period to be split, for example to insert pulses between acquired data points.

The sample_period_2 is used by the reconstruction and analysis programs to define the scale of the frequency axis, and by the field-of-view keyword FOV. The keyword FOV can only be used when the sample period is fixed at a constant value, by using the same value for “min”, “max”, and “default”. The “min” and “max” values (in units of 100 ns) must not exceed the range 20 to 32767.

Please note: Either the keywords ‘SAMPLE_PERIOD_2’ (above) or ‘SWEEP_WIDTH_2’, (but not both) may be included in two-dimensional experiments to set the time increment for the second dimension. A multi-receiver version of the SWEEP_WIDTH_2 keyword exists. See the ‘Multiple Receivers’ section of this manual (2.2.8) for further details.

2.2.7 *Extended imaging*

The following keywords have been provided to support **volume selection**. They behave as analogues of the SLICE_THICKNESS keyword (in 2.2.4.3), but take their user input from the FOV entry (for READ_ and PHASE_SELECTION) or Slice/Slab thickness entry (for SLICE_SELECTION). They generate no visible controls of their own.

- **READ_SELECTION read_val, rf_bandwidth, read_var;**
- **PHASE_SELECTION phase_val, rf_bandwidth, phase_var;**
- **SLICE_SELECTION slice_val, rf_bandwidth, slice_var;**

The READ/PHASE/SLICE_SELECTION keywords create no visible controls, but instead take their input from the controls created by the FOV (for READ_ and PHASE_SELECTION) or SLICE_FOV (for SLICE_SELECTION) described in 2.1.4. They are used to control the field gradient strength at the time of the selective RF pulse, and are used for additional volume selection in imaging or IVS experiments.

As they take input from the FOV and SLICE_FOV keywords, their use also requires inclusion of the relevant parent keyword in the PARAMLIST.

“read/phase/slice_val” is the numerical values of the stored gradient waveform at the time of the selective pulse. For the standard library sequences, 32767 is the maximum value. “rf_bandwidth” is the excitation bandwidth, in Hz, of the selective pulse. This may

be a different value for each keyword. "read/phase/slice_var" is the names of PPL variables which will contain the values for the analogue multiplying DAC of the selection gradient. The value contained in this variable is calculated by PARAMETER EDITOR from the calibrated gradient strengths. See also the keywords FOV and SLICE_FOV in 2.1.4.

PARAMETER EDITOR calculates the minimum (and maximum) possible slice thickness from the known gradient values.

When any of these keywords are in use, allowed values of the parent controls will be limited to lie within both their normal limits and also any limits imposed by these derived controls. Additionally, if the normal functions of the parent controls are not required within the pulse sequence being developed, entering values of zero for the defined waveform values of the parent controls will disable the conventional range limiting, and the only limits imposed will be those defined by the derived controls. For example,

FOV 2047, 0, 0, read_var, dummy; for 1D projections, with no phase encoding

FOV 0, 2047, 1e-3, dummy, phase_var; for 1D phase encoding, with no "read gradient"

Please note: if these keywords are defined, enabling the FOV or SLICE_FOV controls in interactive setup will require the variables associated with these keywords to be declared as common ints.

2.2.8 Multiple Receivers

Each acquisition system requires its own acquisition processor and generates its own data file. The structure of each data file may be controlled independently using receiver specific versions of selected other keywords.

In order to provide independent control of several different receivers, modified versions of the existing receiver control keywords are provided. The modifiable keywords are as follows:

- EXPERIMENT_ARRAY
- NO_AVERAGES
- NO_SAMPLES
- NO_SLICES
- OBSERVE_FREQUENCY
- SAMPLE_PERIOD_2
- SLICE_BLOCK
- SLICE_INTERLEAVE
- SPECTRAL_WIDTH
- SWEEP_WIDTH_2

NO_ECHOES
NO_VIEWS
NO_VIEWS_2
VIEW_BLOCK
VIEWS_PER_SEGMENT
PHASE_CYCLE

The syntax for a receiver-specific version of a keyword is: **KEYWORD#n** [pre-prompt,] [post-prompt,] [min,] [max,] default, variable [,scale] [,further parameters ...], where n is the receiver number.

For example, to specify an independent number of samples for receiver number 3, the following keyword would be defined: **NO_SAMPLES#3** min, max, default, PPL_var_name;

Keywords that are defined without a receiver number define a global default parameter set, covering all receivers. Receiver-specific definitions override the global definition for the receiver specified. Hence to achieve an acquisition with channels 1 and 3 at one frequency and channel 2 at a second, with different **SPECTRAL_WIDTH** on the two channels but the same number of averages. If so, the following keywords would be required:

NO_RECEIVERS (value 3)
NO_AVERAGES
OBSERVE_FREQUENCY
SPECTRAL_WIDTH
OBSERVE_FREQUENCY#2
SPECTRAL_WIDTH#2 (NOT to be confused with **SWEEP_WIDTH_2**)
NO_SAMPLES#2

On sequence initialisation, all receiver systems are configured for the global acquisition parameters. It is the responsibility of the pulse program to configure any independently controlled systems according to the supplied parameters.

2.2.8.1 RECEIVER_MASK min, max, default, var_name;

This keyword creates an edit box with the pre-prompt "Receiver Selection (**hex**)". The user may edit the initial value "default", and the edited value is passed to the PPL variable "var_name", for optional use within the pulse-program. This keyword indicates the SCAN software to the allocation of independent acquisition systems to be used. In the absence of this keyword, the system uses the default setting currently set in SCAN.

The value is a hexadecimal representation of the bit-mask of the selection of receivers installed in the system that will be used for the acquisition. A value of:

1 will request the 1st installed receiver
2 will request the 2nd installed receiver
4 will request the 3rd installed receiver
8 will request the 4th installed receiver
10 will request the 5th installed receiver
20 will request the 6th installed receiver
...

Multiple receivers can be requested by adding together the relevant numbers from above.

Each acquisition system requires its own acquisition processor and generates its own data file. The structure of each data file may be controlled independently using receiver specific versions of selected other keywords - see the 'Multiple Receivers' help topic for further details.

2.3 Common int data type

If the variables referred to in the keyword definition are declared to be of type “common int”, they are placed in dual port RAM and can be accessed by both the host computer and the Forth processor. This is recognised by SCAN and PARAMETER EDITOR to allow interactive adjustment by running SCAN in Setup mode. Only those items which use variables declared as “common ints” remain black in the setup parameter editor screen produced by running SCAN in setup mode and are adjustable. All the other items are non-adjustable (with grey colour).

Any calculations that use “common int” variables for interactive setup must be performed within the main acquisition loop, so that the results are kept current as the “common int” variable contents change.

The items will only be interactively adjustable via PARAMETER EDITOR if all the variables are of type “common int” (e.g. OBSERVE_FREQUENCY). Items that would affect parts of the instrument (which cannot be interactively adjusted, e.g. the acquisition program) will not be adjustable interactively even though all these variables are declared as the type of “common int”. The relevant keywords are as follows:

NO_SAMPLES
SPECTRAL_WIDTH
DSP_ROUTINE
NO_AVERAGES
PHASE_CYCLE
NO_VIEWS
NO_VIEWS_2

VIEW_BLOCK
EXPERIMENT_ARRAY
NO_ECHOES
DATA_TYPE
BUFFER_SIZE
DISCARD
NO_SLICES
SLICE_ INTERLEAVE
SLICE_BLOCK
SLICE_THICKNESS
SLICE_SEPARATION
ORIENTATION
FOV
NO_RECEIVERS

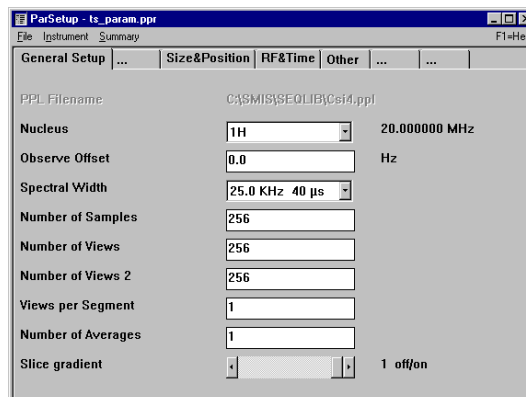
For example, the keywords of the ORIENTATION, OBLIQUE_ORIENTATION, FOV, SLICE_THICKNESS, FOV_OFFSETS, and SLICE_SEPARATION can be enabled in interactive mode, if the PPL variable for each keyword is defined as a common int. Furthermore, dependencies between the keywords exist such that controls at the top of the dependency tree are enabled only if the controls further down the tree are enabled, the dependencies being defined as follows:

OBLIQUE_ORIENTATION / MULTI_ORIENTATION
ORIENTATION
 X ANGLE (oblique_orientation only)
 Y ANGLE (oblique_orientation only)
 Z ANGLE (oblique_orientation only)
 FOV (both the read and phase variables must be common ints)
 FOV_READ_OFF
 FOV_PHASE_OFF
 SLICE_THICKNESS
 SLICE_SEPARATION
 FOV_SLICE_OFF

The (non-oblique) orientation control is enabled only if the FOV, slice thickness, slice separation and all FOV offset keywords are enabled (if defined). The x angle edit field is enabled if the x angle variable is a common int, and the FOV, slice thickness, slice separation and FOV offset controls are enabled. The slice thickness control is enabled if the slice separation and FOV slice offset controls are enabled.

2.4 Templates

There is the possibility of customizing the organization of the parameters across the PPR file TAB pages in ParSetup, such as Setup or Size & Position. Template files customisation of grouping and ordering of individual parameters on to separate tab. In addition, the display state of each parameter may be set to Enabled, Disabled or Hidden.



various
General
provide
pages.

Parameters not otherwise explicitly specified are placed on the default tab. If there are more than a limited number of parameters to be displayed on one tab, additional tabs entitled "..." are automatically created.

Templates are specified in text files (*.ptp). The default file is "**Default.ptp**" stored in the directory of c:\smis\.

Template files are formatted as follows:

[General]

DefaultTab=1

Place unspecified parameters on the default tab.
(1=true, 0=false)

DefaultTabName="Other"

DefaultDisplayState=1

Default parameter display state.
(1 = enabled, 2= hidden, 3= disabled)

[Tab1]

Title="General Setup"

Param1=PPL_NAME, 3

Parameter keyword (or variable name) with optional display state.

Param2=no_slices

Note that parameter number and tab number must be explicitly specified.

[Tab2]

Title="Size&&Position"

Param1=SLICE_THICKNESS

2.5 Appendix

Keywords' argument-list nomenclature:

"pre-prompt"	A string written to the left of the edit box or scroll bar, to describe the meaning of the parameter to the user, e.g. "Echo time".
"post-prompt"	A string written to the right of the edit box or scroll bar, normally containing user units, e.g. "ms" or "cm".
"format"	The editable number is formatted according to the "C"- type format descriptor "format". The formats "%d", "%i", "%u", "%o", "%x", "%X", "%g", "%G", "%e" and "%E" are recognised, and the number of significant figures displayed can be specified. The formatted number is displayed to the right of a scrollbar when using the SCROLLBAR keyword.
"var_name"	Variable name as declared in the PPL program. This is initialised before the PPL program runs, so the variable named need not be initialised within the PPL program. The variable is initialised to contain the final value edited by the user, converted to PPL units by the factor scale .
"array_name"	Array variable as declared in the PPL program. The first no_elements of the array are initialised before the program runs. The elements are initialised to the values edited by the user, converted to PPL units by the factor scale .
"min"	The minimum value that can be entered by the user, in PPL units. If the user attempts to enter a value less than this into an edit box they are warned of the error and required to re-enter a valid number. For a scroll bar this is the value at the left end.
"max"	The maximum value, in PPL units. If the user attempts to enter a value greater than this into an edit box they are warned of the error and required to re-enter a valid number. For a scroll bar this is the value at the right end.
"default"	The initial value in the edit box or scroll bar, in PPL units.
"[, default]"	This is an optional list of default array values in PPL units. The first default value is assigned to array_name[0], the next to array_name[1] and so on. Those array elements which do not have explicit default values defined are set initially to the min value.
"scale"	The conversion factor from user units to PPL units. The PPL variable is loaded with the value (int) (user_value*scale). The PPL unit values min, max and default are converted by the inverse function before being used as the limits and initial value presented to the user.

"no_elements" The number of elements of the array specified in the VAR_ARRAY argument list which will be edited: the array declared in the PPL program must have at least no_elements elements.

Chapter 3. LIBRARY FUNCTIONS

Common pulse program operations used in MRI experiments have been included in a function library. These include, for example, frequency and phase setting functions, multislice imaging, and various address/data combinations.

These additional functions are made available to PPL by function declarations within PPL. A function is declared in PPL as:

return_type function_name (argument_type [name], [argument_type[name]], [..]);

During the execution of a library function, the state of all output lines is controlled by the function and is not under the direct control of PPL. Following a function call the output lines are generally not returned to the state existing before the function was called. It is important to note that in order to keep the execution times of library functions as short as possible there is generally no run-time error checking. Any function called with arguments outside the valid limits will give unpredictable results, and may or may not prevent execution of the remainder of the pulse program.

Any library functions to be used in the program must be declared prior to the main section of the pulse program. The simplest way to do this is to declare all of the available functions using the command:

3.1 #include directive

The #include directive can be written before the main function in a PPL source file to direct the PPL compiler to read function and variable declarations from another file before continuing to process the line following the #include. The syntax of the #include directive is either

```
#include "filename"  
or  
#include <filename>
```

If the filename appears between quotation marks (" "), the compiler searches for the file first in the directory of the file which the #include directive appears in, then (if the current file is itself an include file) the directories of the file which included the current file and so on until finally searching the directory indicated by the PPL environment variable. The search stops as soon as a matching file is found. If the filename appears between angle brackets (< >), the compiler searches in the directory indicated by the PPL environment variable if it is set, or in the current directory if the PPL environment variable is not set.

You can use as many #include directives as you like, and mix them with #use directives and the PARAMLIST section in any order. Include files may contain library function and

variable declarations and #use statements. Note that at present all PPL variables have global scope, so that declaring a variable in an include file is exactly the same as declaring it in the main function. However, at some point in the future PPL may be modified to allow the concept of local and global variables so it is recommended that the use of include files to declare variables is kept to a minimum.

An include file (also frequently known as a header file) has been created for the new library functions described below. This file is called STDFN_15.PPH (the file extension .PPH has been chosen to indicate Pulse Program Header file), and will typically be installed in the directory indicated by the PPL environment variable. Thus, rather than reproduce the function declarations required for each library function which your PPL source file uses, you can simply place the line

```
#INCLUDE "STDFN_15.PPH"
```

at the top of the PPL file. All the example fragments given in the descriptions of the new library functions given below use this include file. A second header file, VAR_15.PPH, has been created which contains declarations for most commonly used constants and variables to date.

3.2 List of library functions

3.2.1 *Fast RF control functions*

- void frequency(int MHz, int kHz, int Hz, int rx1MHz)
- void dfrequency(int MHz, int kHz, int Hz, int dummy)
- void frequency_buffer(int buffer_number)
- void preset_frequency(int MHz, int kHz, int Hz, int rx1MHz)
- void dpreset_frequency(int MHz, int kHz, int Hz, int dummy)
- void reset_frequency()
- void dreset_frequency()
- void offset_frequency(int Hz)
- void phase_increment(int angle)
- void dphase_increment(int increment)
- void phase(int multiple)
- void dphase(int phase)

- `int aqphase(int completed_averages, int phase_cycle)`
- `void rphase(int multiple)`
- `void transmit()`
- `void receive()`
- `void acquire(int sample_period, int no_points)`
- `void initiate()`
- `void sample(int sample_period, int no_points)`
- `void complete()`
- `void discard(int no_discard_samples)`
- `void sync()`
- `void resync()`
- `void rfampon(int dummy)`
- `void drfampon(int dummy)`
- `void rfon(int amplitude)`
- `void drfon(int pulse_power)`
- `void rfpulse(int amplitude, int duration)`
- `void drfpulse(int pulse_length, int pulse_power)`
- `void rffoff()`
- `void userout(int 8bit)`
- `void systemout(int 8bit)`
- `void starttimer()`
- `int gettimer()`
- `void slice(int slice_number)`
- `void multislice(int interleave, int offset, int number_of_slices)`

3.2.2 *Instrument control RF functions*

- `void HostRequest()`
- `void ObsRFUnitEnable(int nOn)`
- `void SetObsModGain(int dB10)`

- void SetObsRecvAttn(int dB10)
- void SetObsRecvGain(int dB10)
- void DecRFUnitEnable(int nOn)
- void SetDecModGain(int dBIO)

3.2.3 RF control functions

- void MR3031setup(ALIAS, FRAME REF, int zMul, int yMul,int xMul, int RFMul)
- void Append_Boards_Ready(int nBoardAddress)
- void Enable3031(int nBoardAddress)
- int Get3031Status(int nBoardAddress)
- void HaltEnabled3031()
- void MR3031_command(int command, int board_address)
- void MR3031_go()
- void MR3031_reset()
- void MR3031_stop()
- void RunEnabled3031()
- void Set3031Address(int frame_address, int board_address)
- void Set3031LoopMode(int LoopMode, int BoardAddress)
- void Set3031Waits(int waits, int board_address)
- void Set_Addr_And_Waits(int frame_address, int no_waits, int Board1, int Board2)
- void Set_Board_Multipliers(int Mul1, int Mul2, int BaseAddress)

3.2.4 Gradient Control Functions

- void MR3040_SetListAddress (int address)
- int MR3040_InitList()
- void MR3040_Output(int frame_address, int frame_rate, int frame_ points, int loop_type)
- void MR3040_Hold(int loop_type)
- void MR3040_LoopCount(int loop_count)

- void MR3040_SetList(int address, int channel_id)
- void MR3040_Start(int channel_id)
- void MR3040_Continue(int channel_id)
- void MR3040_Break (int channel_id)
- void MR3040_Stop(int channel_id)
- void MR3040_Control(int control_bits)
- void MR3040_RunChannels(int address1, int address2, int address3
- void MR3040_Clock(int duration)
- int MR3040_SeqStatus(int mask)
- void MR3040_CreateBaseMatrix(int z_angle, int y_angle, int x_angle, int div_z, int mul_z, int div_y, int mul_y, int div_x, int mul_x)
- void MR3040_CreateMatrix(int matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle)
- void MR3040_ModifyMatrix(int_dest_matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle, int source_matrix_id)
- int MR3040_MatrixStatus(int mask)
- void MR3040_SelectMatrix(int matrix_id)

3.3 PPL library hardware control function reference

3.3.1 Fast RF control

3.3.1.1 void frequency(int MHz, int kHz, int Hz, int rx1MHz)

MHz, kHz, Hz and rx1MHz are PPL expressions defining the synthesiser frequency for both transmit and receive (see also slice()). MHz, kHz and Hz contain an integer in the range 0 to 999 for the respective frequency millennium. rx1MHz contains an integer 0 to 9 for the 1 MHz digit during receive, which will be 2 MHz different from the 1 MHz digit defined by the variable MHz. These four expressions will normally be variables, with contents generated by PARAMETER EDITOR and SCAN.

The frequency information is generally stored in the defined current frequency buffer for rapid access when required. Both the transmit and receive frequency information in the

current frequency buffer is sent to the interface board by the functions **frequency()** and **reset_frequency()**. The transmit frequency latched in the interface board is sent to the synthesiser by the functions **frequency()**, **reset_frequency()**, **rfampon()**, **transmit()**, **complete()**, and at the end of **acquire()**. The receive frequency latched in the interface board is sent to the synthesiser by the functions **receive()**, **initiate()**, and at the beginning of **acquire()**.

See also **offset_frequency()**, **slice()**, and **dfrequency()** in this section.

This function takes 139.2 μ s to execute, plus the time taken to evaluate the arguments. It is not intended to be used in time critical parts of the pulse program.

3.3.1.2 ***void dfrequency(int MHz, int kHz, int Hz, int dummy)***

The function **dfrequency()** is decoupler frequency which is same as the function **frequency()**. This function takes 2.5 μ s longer than **frequency()** to execute.

3.3.1.3 ***void frequency_buffer(int buffer_number)***

The function **frequency_buffer()** allows up to ten synthesiser frequencies to be stored for rapid frequency switching during a pulse sequence. Following a call to **frequency_buffer()** subsequent calls to the observe channel frequency controlling functions **frequency()**, **preset_frequency()**, **reset_frequency()** and **multislice()**, and the decouple channel functions **dfrequency()**, **dpreset_frequency** and **dreset_frequency()** refer to the numbered internal frequency buffer.

The argument **buffer_number** can take the values 0 to 9 inclusive. The default value is zero. The function takes 0.5 μ s, plus the time taken to evaluate the argument. See also **offset_frequency()** regarding phase offsets.

3.3.1.4 ***void preset_frequency(int MHz, int kHz, int Hz, int rx1MHz)***

This stores the supplied frequency in the current internal frequency buffer, but does not send it to the interface board or synthesiser.

This function takes 114 μ s to execute, plus the time taken to evaluate the arguments. It has been provided so that a frequency can be preset in a part of the pulse program that is not time critical, and subsequently rapidly accessed by the function **reset_frequency()**. See also **dpreset_frequency()**, and **offset_frequency()** regarding phase offsets.

3.3.1.5 ***void dpreset_frequency(int MHz, int kHz, int Hz, int dummy)***

This is preset decoupler frequency, see **preset_frequency()**. This function takes 2.5 μ s longer to execute than **preset_frequency**.

3.3.1.6 `void reset_frequency()`

Completely resets the stored frequency in the current frequency buffer, previously defined by the **frequency()** or **preset_frequency()** functions and sets the synthesiser to the transmit frequency and phase. This function *must* be called before data acquisition if the frequency has been reset by direct PPL code or by the **slice()** function, even if the transmit frequency will not be used again, as it is used in generation of the receive frequency. This function takes a total of 25.4 μ s to execute. The time from the function call until data is sent to the synthesiser is 23.7 μ s, and this is followed by 0.7 μ s to return control to PPL.

See also **frequency()**, **preset_frequency()**, **offset_frequency()**, **frequency_buffer()**, **multislice()**, **slice()**, and **dfrequency()**.

3.3.1.7 `void dreset_frequency()`

Reset decoupler frequency, see **reset_frequency()**. This function takes 2.5 μ s longer to execute than **reset_frequency**.

3.3.1.8 `void offset_frequency(int Hz)`

The function **offset_frequency()** pre-sets a new frequency in the current frequency buffer, calculated from the existing frequency plus the argument to the function (in Hz). The argument must be within the limits of - 32767 and 32768 Hz. Once preset, the frequency will not be sent to the synthesiser until the function **reset_frequency()** is called.

Frequency changes which do not affect the 1 MHz or greater frequency units (e.g. 10.000000 MHz to 10.999999 MHz) are phase continuous, whereas those which affect the 1 MHz or greater units only (e.g. 19.999999 MHz to 20.999999 MHz) are phase coherent.

A phase coherent change acts as if a separate synthesiser was switched in. Thus if a phase coherent change in frequency is followed by a change back, the phase also returns to its original value. Similarly if a phase coherent change is followed by another phase coherent change (e.g. to receive mode), then the phase of the final frequency will be unaffected by the intervening period.

A phase continuous frequency change is cumulative, according to its offset and duration. For example, a frequency increase of 1 Hz for 0.5 seconds, followed by a decrease of 1 Hz to restore the original frequency, will result in a final output 180° out of phase with the original. This may have implications in the design of an MRI experiment. In any repetitive experiment, in which the same offsets are repeated each time data is accumulated (e.g. multislice imaging), the only effect will be a rotation of the absolute phase of the received signal. Phase cycling schemes will operate correctly.

Experiments in which the phase of the synthesiser outputs changes in a nonrepetitive way between the excitation pulse and accumulation of data require the phase to be made consistent before the signal is collected. For example, consider the spectral editing spin echo sequence "90°_180°_acquire" in which two experiments are performed with different frequency offsets for the selective 180° pulses, and the resulting FIDs are subtracted. There are a number of ways to achieve this:

- ✓ Including all the frequency changes in all experiments, but only transmitting during specific periods.
- ✓ Including an equal period of frequency shift in the opposite direction.
- ✓ Using the **rphase()** function to directly set the receive reference phase (obtained by measurement or calculation).

The function **offset frequency** takes 91.0 µs to execute, plus the time taken to evaluate the argument.

3.3.1.9 void phase_increment(int angle)

Subsequent calls to **phase()** set the transmit or receive phase in units of (angle * 0.225)°. For example, if angle is an expression that evaluates to 400, phase() will set the transmit phase in steps of 90°. The angle defaults to 400 if the **phase_increment()** function is not called. This function takes 13.2 µs to execute, plus the time taken to evaluate the argument.

3.3.1.10 void dphase_increment(int increment)

Decoupler phase increment, see **phase_increment()**.

3.3.1.11 void phase (int multiple)

Sets the transmitter phase stored in the interface to (multiple * phase increments), where **phase_increment** is 90° by default, but can be set to any desired multiple of 0.225° by the **phase_increment()** function. Multiples resulting in a phase of greater than 360° are valid and the final phase is the calculated phase mod 360°. The resulting phase is immediately sent to the synthesiser, and is reset each time the **transmit()** or **complete()** function is called.

If **phase_increment()** has not been called, or has been called with an argument of 0, 100, 200, 400 or 800 (corresponding to phase increments of 0, 22.5°, 45°, 90° or 180°), the **phase()** function takes 4.6 µs, plus the time taken to evaluate the argument, to set the

synthesiser phase. For all other arguments to **phase_increment()**, **phase()** takes 28.3 μs , plus the time taken to evaluate the argument, until the phase data is sent to the synthesiser. Once the phase is set **phase()** takes 0.7 μs until control is returned to PPL.

3.3.1.12 *void dphase (int phase)*

Decoupler phase, see **phase()**.

3.3.1.13 *int aqphase(int completed_averages, int phase_cycle)*

This function returns an integer 0, 1, 2, or 3 corresponding to the phase of the receiver for the given number of completed averages and phase cycle. The argument phase cycle is generated by the parameter editor in response to the by the PARAMLIST keyword **PHASE_CYCLE**. The returned value is suitable for use as the argument to the **phase()** function, if the transmitter phase is to match the receiver, when phase steps of 90° are used (the default condition, or following a call to **phase_increment (400)**). Function **aqphase()** takes 1.2 μs plus the time taken to evaluate the arguments.

3.3.1.14 *void rphase(int multiple)*

The function **rphase()** sets the receiver phase to $(\text{multiple} * \text{phase_increment})^\circ$ in the same way as the function **phase()** set the transmitter phase. If this function is not called the receiver phase is set to 0° by default. The receiver phase is set immediately by the **rphase()** function, and is reset by the **receives** and **acquires** functions. See also **offset_frequency()**. This function takes a total of 5.3 or 29 μs to execute (see **phase()**), plus the time taken to evaluate the argument.

3.3.1.15 *void transmit()*

Sets the synthesiser to the transmit frequency and phase previously latched into the interface board by the **frequency()**, **reset_frequency()** and **phase()** functions. This function takes 1.6 μs to execute.

3.3.1.16 *void receive()*

Sets the PTS synthesiser to the receive frequency and phase previously defined by the **frequency()**, **reset frequency()** and **rphase()** functions. This function takes 1.6 μs to execute.

3.3.1.17 *void acquire(int sample_period, int no_points)*

Integrates the actions of **initiate()**, **sample()** and **complete()**, to provide a single function call for collection of data. The minimum acceptable number of (**no_samples + discard**) is 2 (e.g. 2 sample points with a discard of 0, or 1 sample point with a discard of 1). Attempting to collect single point acquisitions with 0 discard will result in acquisition being enabled for approximately 64 k sample points.

The resynchronization to the 100 kHz ADC clock takes the minimum time if the time between a call to **sync()** or **resync()** occurs exactly $(10n + 6) \mu\text{s}$ before the call to **initiate()** or **acquire()** for sample periods of $10\mu\text{s}$ or longer, or $(10n + 2.5) \mu\text{s}$ for sample periods of less than $10 \mu\text{s}$.

The ADC is enabled so as to digitise data present at the input $2.0 \mu\text{s}$ after the synthesiser switches synthesiser to the receive frequency. This will be $2.6 \mu\text{s}$ after the call to **acquire()**, plus the time taken to evaluate the arguments. See also **pulsacq()**.

This function takes a total of $23.6 \pm 5.0 \mu\text{s}$ to execute if the **sample_period** argument is less than 100, or $365.1 \pm 5.0 \mu\text{s}$ if the argument is greater than or equal to 100, plus the time taken to evaluate the arguments and the sampling time **(no_samples + no_discard)*sample_period**.

3.3.1.18 void initiate()

This function is used to initiate data collection. It sets the synthesiser to the receive frequency, enables the preamplifier (TTL line high) and demodulator, and enables the ADC synchronously with the 100 kHz clock. The ADC will only collect data at multiples of $10 \mu\text{s}$ following the **sync()** function. Thus any pulse sequence occurring between the call to **sync()** and the data collection must be of constant duration, or change only in integral multiples of $10 \mu\text{s}$ (see **sync()**).

Function **initiate()** should be followed by a delay sufficient to collect the specified number of sample points at the specified sample rate (defined by /*PARAMLIST and PARAMETER EDITOR). An appropriate delay is produced by the **sample()** function. See **complete()**.

3.3.1.19 void sample(int sample_period, int no_points)

This function produces a delay sufficient for the collection of **no_points** data points separated by the duration of **sample_period** (μs). It is not necessary to include this function, and control can be returned to the PPL program, provided that **complete()** is called precisely at the correct time following **initiate()**.

3.3.1.20 void complete()

This function is used to complete data collection. It disables the ADC, demodulator and pre-amplifier, and resets the synthesiser to the transmit frequency stored in the interface board latches.

This function can be used to write pulse programs with special acquisition requirements not covered by the use of **acquire()** or **pulsacq()**, with PPL code of the form:

initiate()

...pulses, delays etc for the precise acquisition time ...

complete().

The function **complete()** takes 5.2 μs .

3.3.1.21 **void discard(int no_discard_samples)**

Function **discard()** increases the delay produced by **sample()** and **acquire()** by sufficient time to collect an extra **no_discard_samples** data points, separated by the sample interval given to **sample()** or **acquire()**. If this function is not called the default is to collect an additional two samples.

In an NMR instrument of traditional, analogue filter design, filtering occurs before the ADC and the delay due to the filter response is dealt with by adding a small delay before enabling the ADC. This is not possible with a digital filter, where it is necessary to digitise and digitally filter, but then discard, the first **no_discard_samples** data points.

As a result the pulse program must enable the ADC for sufficient time to collect **no_discard_samples+no_samples** data points. This is carried out using the library function **discard()**, which adds time for collection of **no_discard_samples** data points to the acquisition. This must only be used in combination with an acquisition routine, e.g. the MR Solutions standard routine, which can discard the appropriate number of initial points. This function takes 0.5 μs to execute, plus the time taken to evaluate the argument.

3.3.1.22 **void sync()**

This function synchronises the pulse program to the ADC 100 kHz reference, and starts an internal 100 kHz timer. It should always be called at the beginning of a sequence. See also **resync()**, **acquires; sync()** takes a variable time to execute, but is of the order of 300 ~ 400 μs .

3.3.1.23 **void resync()**

This function synchronises the pulse program to the internal clock started by the **sync()** function. It should be called within the averages loop just before pulses are transmitted,

to ensure that the total time between the RF pulse(s) and the beginning of data acquisition remains constant.

If the time between the call to **resync()** and the data collection is changed during the pulse sequence (e.g. incrementing pulse length) it should be remembered that data acquisition only starts at integral multiples of 10 μ s following a **resync()** call. Thus pulse sequences should be written to keep the delay between **resync()** and data collection constant, or variable only by 10 μ s intervals.

The function **resync** takes a variable time to execute, up to a maximum of 10.5 μ s. The function **sync()** could be used in place of **resync()**, but is very much slower.

3.3.1.24 **void rfampon(int dummy)**

This function sets the RF amplifier blanking line high, and sets the synthesiser to the transmit frequency. This function is provided to prepare for an RF pulse, so that the pulse itself can be executed as fast as possible. The argument is not used. See also **rfon()**, **rfoff()**, **drfampon()**, **drfon()**, **drfoff()**. This function takes 0.6 μ s, plus the time taken to evaluate the argument, until the RF amplifier enable line is sent high. There is a further 0.6 μ s until control returns to the PPL program.

3.3.1.25 **void drfampon(int dummy)**

Turns on Decoupler amplifier gate, see **rfampon()**.

3.3.1.26 **void rfon(int amplitude)**

This enables the external observe channel of the RF amplifier, and turns the transmitter on. The value of amplitude controls the attenuation of the RF output:

Amplitude	Output
0	Zero amplitude, no RF output
4	0 dB attenuation

This function does not explicitly set the synthesiser to the transmit frequency, as this is normally performed by other functions in the pulse sequence, either at the end of the previous acquisition (**complete()**) or by **rfampon()**. It also does not change the value applied to the multiplying DAC for the RF analogue channel. See also **rfampon()**, **rfoff()**, **drfon()**, **drfampon()**, **drfoff()**.

The time taken from the call to this function to the beginning of the RF pulse is 0.9 μ s, plus the time taken to evaluate the argument. The time from the beginning of the RF output until control returns to the PPL program is 0.6 μ s.

3.3.1.27 ***void drfon(int pulse_power)***

This turns Decoupler on, see **rfon()**.

3.3.1.28 ***void rfpulse(int pulse_length, int pulse_power(obs_mod))***

Produces a hard RF pulse of length (duration 100ns), with an amplitude determined as for the **rfon()** function. See also **rfon()**, **rffoff()**, **pulsacq()**, **drfpulse()**.

The time taken between the call to this function and the beginning of the RF pulse is 2.4 μ s plus the time taken to evaluate the arguments. The minimum duration is 100 ns, and the maximum 3276.7 μ s. The duration of 0 will generate a pulse 100 ns long. The time taken from the end of the RF pulse until control is returned to the program is 0.8 μ s.

3.3.1.29 ***void drfpulse(int pulse_length, int pulse_power(obs_mod))***

This generates RF pulse on Decoupler channel, see **rfpulse()**.

3.3.1.30 ***void rffoff***

Turns the RF transmitter off and disables the external amplifier. This function takes a total of 1.1 μ s to execute. The time from the function call to the end of the RF pulse is 0.5 μ s, and from the end of the RF output until control returns to PPL is 0.6 μ s. In systems with decoupling, this will turn off both the observe and decouple RF channels. To turn off the observe without affecting the decouple channel, use **rfon(0)** instead.

3.3.1.31 ***void userout(int 8bit)***

This function sets the TTL outputs of the user port according to the 8 bit argument. It has the same effect as the PPL line: output (8 bit * 256 | 18); except that the output state set by the latest PPL **output()** call is restored after each library function call. Thus after calling **userout()** the pulse programmer lines will be reset to the state existing before the function call (normally 0), whilst after calling **output()** the pulse programmer line will be reset to new value immediately, and again after every library function call. See also **systemout()** This function takes 1.3 μ s to set the user port outputs, and further 0.6 μ s until control is returned to PPL.

3.3.1.32 *void systemout(int 8bit)*

This function sets the TTL outputs of the system port according to the 8 bit argument. See also **userout()**; 1.3 μ s is taken to set the user port outputs, and further 0.6 μ s till control is returned to PPL.

3.3.1.33 *void slice(int slice_number)*

This function sets the transmitter frequency correctly for the given slice in a multislice imaging experiment, according to the interleave and offset previously defined by the **multislice()** function.

The argument **slice_number** should be incremented from 1 to the number of slices. The transmitter frequency is altered symmetrically about that set by the **frequency()** function. This function should only be used with single angle oblique pulse sequences (using the "oblique orientation" keyword in the PARAMLIST).

The sequences containing "**MAO**" as defined in c:\smis\include\m3040_15.pph should use the **SELECTSLICE()** macro.

The function **reset_frequency()** must be called to reset the synthesiser before data acquisition following any calls to **slice()**. See also **multislice()**, **frequency_buffer()**. This function takes 18.5 μ s to execute, plus the time taken to evaluate the argument.

3.3.1.34 *void multislice(int interleave, int offset, int number_of_slices);*

This function is used to pre-set the frequencies for the slice selective pulses of a multislice imaging experiment.

The argument **interleave** determines the slice order, as described for the **SLICE_INTERLEAVE** keyword in the /*PARAMETER header (PPL code). The argument **offset** is the frequency offset per slice, as described for the **SLICE_SEPARATION** keyword.

This function **multislice()** calculates and stores the whole frequency derived by applying the offsets to the centre frequency in the current **frequency_buffer**. Subsequent calls to function **slice()** will set the frequencies stored at the time multislice was executed, irrespective of any intervening calls to **frequency_buffer**.

This function takes a variable time to execute, and is not intended to be used in time critical parts of a pulse program. It stores the frequency information in memory for rapid access when required. The synthesiser is set to the correct transmit frequency for a multislice imaging experiment by the function **slice()**, and reset for nonselective pulses and receive by the function **reset_frequency()**.

3.3.2 Summary of library function timing for RF controller

Function	Time after evaluation of arguments (μ s)	No. Arguments	Time if each argument is an int variable (μ s)
Acquire before frequency switch before modulator on before ADC effective total < 10 ms ≥ 10 ms	0.6 0.9 7.6 ± 5.0 Sampling time plus 23.6 ± 5.0	2 2 2	1.4 24.4 ± 5.0 + sampling 365.9 ± 5.0 + sampling
aqphase	365.1 ± 5.0	2	2
complete before ADC off total discard	1.2 0.5 5.2	 1	 0.9
frequency	0.5	4	140.8
frequency_buffer	139.2	1	0.9
initiate before frequency switch before modulator on before ADC effective Total < 10 ms ≥ 10 ms	0.5 0.6 0.9 7.6 ± 5.0 18.7 ± 5.0	2 2 2	1.4 19.5 ± 5.0 362 ± 5.0
multislice	1.2	3	
phase_increment	Variable	1	13.6
phase to RF 22.5° steps fine steps total 22.5° steps	13.2 4.6 28.3	 1 1 1	 5 28.7 5.7
rfampon to amplifier total	0.6 1.2	1 1	1 1.6

rfoff to rf off total	0.5 1.1		
rfon to rf on total	0.9 1.5	1 1	1.3 1.9
reset_frequency to rf total	23.7 25.4		
resync resync <-> acquire for minimum delay in acquire() and initiate < 10 ms ≥ 10 ms	5.2 ± 5.0 2.5 6		
rfpulse to rf on after rf total	2.4 0.8 3.2 + pulse	2 2	3.2 4.0 + pulse length
rphase to rf 22.5° steps fine steps total 22.5° steps fine steps	 4.6 28.3 5.3 29	 1 1 1 1	 5 28.7 5.7 29.4
sample ~	sample_period*((no_samples+no_discards)-3)*100 ns sample time		
		2	0.8 + sample time
slice to rf total	17.9 18.5	1 1	18.3 18.9
starttimer	0.4		
sync	Variable 300 ~ 400		
transmit	1.6		

3.3.3 *Support RF functions - slow control of instrument parameters*

The following functions allow control of the RF unit configuration. For most experiments, the default initialisations set by SCAN will be sufficient and (with the exception of HostRequest) these functions will not need to be used. As some commands provide for global configuration changes, and the hardware may take a relatively long time to settle following updates, these functions should not be used in time critical sections of the pulse program. Execution times are quoted below merely as an indication of their magnitudes; they are not guaranteed to remain constant, and are also slightly dependent on the argument values.

*Reference is made to dB10 units in subsequent sections; these are units of 0.1 dB.

3.3.3.1 *void HostRequest()*

This function checks the pulse programmer input buffer. If it contains a request for action from the host computer (indicated by a negative buffer byte count) the action is executed. If no action is required, the function takes 4.0 μ s, otherwise the time taken is dependent on the action and is at least 55 ms. Multiple commands may be queued in the input buffer, and processing will continue until the buffer remains clear for 1 PC clock cycle. The sequence should make calls to HostRequest frequently to allow PC control of the IIC bus. (The same action also occurs as part of the PPL "read" function).

3.3.3.2 *void ObsRFUnitEnable(int nOn)*

Enables/Disables the observe modulator, at IIC address 4. The argument is bit mapped as follows:

- bit 0: 0 - disable modulator
1 - enable modulator
- bit 1: 0 - external modulation source
1 - internal modulation source

i.e. the following argument values are typically used:

- 0: Disable modulator (external modulation)
- 1: Enable modulator for external modulation
- 2: Disable modulator (internal modulation)
- 3: Enable MR3330 modulator for internal modulation

The default condition set by SCAN prior to executing the sequence is for the modulator to be enabled with external modulation selected. Following termination of the pulse sequence, the modulator is disabled. This function takes approximately 1.00 ms to execute.

3.3.3.3 *void SetObsModGain(int dB10)*

Sets up the observe modulator attenuator DAC. The default condition set by SCAN prior to executing the sequence is for the gain to be set to the value of "ObserveTransmitGain" in c:\windows\win.ini. If SCAN is in setup mode, this command will be reissued by the PC on each change of gain. Accepted range of values is -459 ~ 0 (dB10 units).

Function timing: This function takes approx. 1.25 ms to execute.

3.3.3.4 *void SetObsRecvAttn(int dB10)*

Sets up the observe amplifier attenuator DAC. The default condition set by SCAN prior to executing the sequence is for the gain to be set to the value of "ObserveReceiveAttenuation" in c:\windows\win.ini. If SCAN is in setup mode, this command will be reissued by the PC on each change of gain. Accepted range of values is -149 ~ -1 (dB10 units).

Function timing: This function takes approx. 1.25 ms to execute.

3.3.3.5 *void SetObsRecvGain(int dB10)*

Sets up the observe amplifier switched gain DACs. The default condition set by SCAN prior to executing the sequence is for the gain to be set to the value of "ObserveReceiveGain" in c:\windows\win.ini. If SCAN is in setup mode, this command will be reissued by the PC on each change of gain. Accepted range of values is 0 ~ 370 (dB10 units).

Function timing: This function takes approx. 1.25 ms to execute.

3.3.3.6 *void DecRFUnitEnable(int nOn)*

Enables/Disables the decouple modulator. The argument is bit mapped as follows:

- bit 0: 0 - disable modulator
1 - enable modulator
- bit 1: 0 - external modulation source
1 - internal modulation source

i.e. the following argument values are typically used:

- 0: Disable modulator (external modulation)
- 1: Enable modulator for external modulation
- 2: Disable modulator (internal modulation)
- 3: Enable MR3330 modulator for internal modulation

The default condition set by SCAN prior to executing the sequence is for the modulator to be enabled with external modulation selected, if and only if the sequence has defined the DECOUPLE FREQUENCY keyword. Following termination of the sequence, the modulator is disabled.

Function timing: This function takes approx. 1.00ms to execute.

3.3.3.7 *void SetDecModGain(int dB10)*

Sets up the decouple modulator attenuator DAC. The default condition set by SCAN prior to executing the sequence is for the gain to be set to the value of "DecoupleTransmitGain" in c:\windows\win.ini. If SCAN is in setup mode, this command will be reissued by the PC on each change of gain. Accepted range of values is -459 ~ 0 (dB10 units).

Function timing: This function takes approx. 1.25 ms to execute.

3.4 RF control functions

There are a number of library functions for controlling the RF hardware. They provide access to the control and status registers of the RF boards. This section provides an overview of the set of functions and then describes each one in detail with its function prototype.

The MR3031_go library function is intended specifically for use with the MR3031_setup function described below. This pair of functions provides the simplest PPL interface to the RF hardware. The MR3031_setup function sets up the various registers on the RF board(s), and the MR3031_go function starts all the RF boards which have been set up via the MR3031_setup function clocking.

The remaining RF library functions can be divided into G bus functions and expansion bus functions. Recall that expansion bus functions affect individual RF boards and G bus functions affect all those boards whose G bus interface is enabled.

The expansion bus functions are listed below:

```
void Disable3031( int nBoardAddress);
```

The G bus functions are listed below:

```
void HaltEnabled3031();
```

```
void MR3031_stop();
```

```
void RunEnabled3031();
```

```
void Set_Addr_And_Waits( int frame_address, int no_waits, int Board1, int Board2 );
```

```
void Set3031Address( int frame_address, int board_address );
```

```
void Set3031 Waits( int waits, int board_address);
```

Finally there is the following function:

```
void Append_Boards_Ready( int nBoardAddress);
```

This function affects neither the expansion bus nor the G bus, but is used to append a board control register address to the array used by MR3031_go so that boards which have been set up using some of the functions listed above rather than the MR3031_setup function can be started running at the same time as boards which have been initialized via the MR3031_setup function. The following section describes each of the above functions in detail.

3.4.1 MR3031_setup

There is currently only one 'built in' function for RF control (function which are 'built in' to PPL do not require PPL function prototypes). This function is the MR3031_setup function. The MR3031_setup prototype is as follows:

```
void MR3031setup( ALIAS, FRAME REF, int zMul, int yMul,int xMul, int RFMul );
```

Please note: since the MR3031_setup function is built into the PPL compiler that this prototype is to demonstrate the type and number of the arguments for the function. You should **not** place the above prototype in the function declarations part of the PPL source file)

MR3031setup sets the start address of the RF CONTROLLER(s) indicated by the alias to the address of the frame indicated by FRAME_REF. The DAC multipliers on the RF CONTROLLER(s) are set to the values indicated by the int arguments. The number of cycles per point value is set from the value associated with the given frame in the waveform data file.

FRAME_REF can either be the number of a frame (frames are numbered starting at 1) or a quoted string, the string being a frame label.

This function uses the alias passed in the ALIAS argument to determine the addresses of the various registers on the RF CONTROLLER which need to be set. It also adds the control register addresses of those boards which it has set up to an array which is used by the library function MR3031_go.

The MR3031_setup function assumes that all the RF CONTROLLER boards in the system are disabled from the G bus, and enables those boards which are indicated by the board1 and board2 members of the alias structure. Having issued the start address and cycles per point setting commands on the G bus, the boards are returned to their original 0 bus access statuses. Note that any other boards which are enabled onto the G bus when

the MR3031_setup function is called will have their start addresses and cycles per point values changed too.

Function timing: The function MR3031_setup takes 14.6 μ s plus the time taken to evaluate the zMul, yMul, xMul and RFMul arguments.

3.4.2 Append_Boards_Ready

```
void Append_Boards_Ready( int nBoardAddress);
```

This function appends the value passed as its argument to the array used by MR3031_go and MR3031_stop. The argument value should be either an alias.board1+4 or alias.board2+4 value, i.e. the expansion bus address of an RF CONTROLLER control register.

Function timing: The Append_Boards_Ready function takes 1.7 μ s plus the time taken to evaluate the nBoardAddress argument.

3.4.3 Disable3031

```
void Disable3031( int nBoardAddress);
```

This function sets bit 3 of the value at the expansion bus address given by nBoardAddress to 1, leaving the other bits as they were. This has the effect of enabling the G bus interface the RF CONTROLLER board whose control register address is at nBoardAddress. The nBoardAddress value should be either alias.board1+4 or alias.board2+4.

Function timing: The Disable3031 function takes 1.6 μ s plus the time taken to evaluate the nBoardAddress argument.

3.4.4 Enable3031

```
void Enable3031( int BoardAddress);
```

This function clears bit 3 of the value at the expansion bus address given by nBoardAddress to 0, leaving the other bits as they were. This has the effect of disabling the G bus interface the RF CONTROLLER board whose control register address is at nBoardAddress.

The nBoardAddress value should be either alias.board1+4 or alias.board2+4.

Function timing: The Enable3031 function takes 1.7 μ s plus the time taken to evaluate the BoardAddress argument.

3.4.5 Get3031Status

```
int Get3031Status( int nBoardAddress);
```

This function returns the value read from nBoardAddress on the expansion bus. If the value of nBoardAddress is alias.board1+4 or alias.board2+4 the least significant six bits of the value returned from this function represent the status of the RF CONTROLLER board.

Function timing: The Get3031Status function takes 1.2 μ s plus the time taken to evaluate the nBoardAddress argument. Don't forget to add the time taken to access the result variable if you are using Get3031 Status in an assignment statement.

3.4.6 *HaltEnabled3031*

```
void HaltEnabled3031();
```

This function reads the G bus address which causes any RF CONTROLLER boards which have their G bus interfaces enabled to stop clocking. The value actually read from the G bus by this operation is garbage and is discarded.

Function timing: The HaltEnabled3031 function takes 0.2 μ s.

3.4.7 *MR3031_command*

```
void MR3031_command( int command, int board_address);
```

This function writes the value passed as its command argument to the address passed as its board address argument on the expansion bus. Thus this function can be used to write to any RF CONTROLLER control and multiplier register via the base addresses supplied in alias.board1 and alias.board2.

Function timing: The MR3031_command function takes 1.2 μ s plus the time taken to evaluate the command and board_address arguments.

3.4.8 *MR3031_go*

```
void MR3031_go();
```

This function enables all those boards whose addresses are contained in the array set up by the MR3031_setup and Append_Boards_Ready functions, then issues the G bus read access which starts all enabled RF CONTROLLER boards clocking. The enable states of the RF CONTROLLER boards whose control register addresses are contained in the array are then restored to their states on entry to this function. Having restored the states of all the boards listed in the array, the array is then cleared, so MR3031_setup and/or Append_Boards_Ready must be called before MR3031_go is called again. This function is similar to the RunEnabled3031 function with the difference that it enables the boards listed in the array first and restores their states afterwards.

Function timing: The timing of the MR3031_go function depends upon the number of boards which have been set up via the MR3031_setup and Append_Boards_Ready functions: the total time taken by this function is number of boards set up * 4 + 3.3 μ s. The first data point is clocked out of the RF CONTROLLER hardware number of. boards set up * 2.8 + 2.8 μ s after the function is encountered.

3.4.9 MR3031_reset

```
void MR3031_reset();
```

This function toggles the reset bit (bit 1 of the expansion bus control register) of all the RF CONTROLLER boards in the system, and leaves all the boards with their G bus interfaces disabled, in halt mode, reset bit high and streaming enabled.

Function timing: The MR3031_reset function takes 6 μ s.

3.4.10 MR3031_stop

```
void MR3031_stop();
```

This function enables the G bus interface on all the boards in the array set up by the MR3031_setup and Append_Boards_Ready functions, then issues the G bus read access command which stops the RF CONTROLLER(s) clocking.

Having done this, the G bus enable states of all the boards listed in the array are restored. The array of boards which have been set up by MR3031_setup and Append_Boards_Ready is unaffected by this function, e.g. MR3031_go which clears the array after starting the RF CONTROLLER(s) clocking.

Function timing: The MR3031_stop function timing depends upon the number of boards which have been set up in the BoardsReady array via the MR3031_setup and Append_Boards_Ready functions.

Please note: the MR3031_go function clears the BoardsReady array, so you will have to use either MR3031_Setup or Append_Boards_Ready prior to calling MR3031_stop. The total time taken by MR3031_stop is number of boards set up * 4 + 3.3 μ s. The boards actually stop clocking number of boards set up * 2.8 + 2.8 μ s after the function is encountered.

3.4.11 RunEnabled3031

```
void RunEnabled3031();
```

This function simply issues the G bus read command which starts all RF CONTROLLERS which have their G bus interfaces enabled clocking.

Function timing: The RunEnabled3031 function takes 0.2 μ s.

3.4.12 Set3031Address

```
void Set3031Address( int frame_address, int board_address);
```

This function enables the 0 bus interface of the RF CONTROLLER board indicated by the board_address argument (which should be alias.board1+4 or alias.board2+4) then writes the frame_address argument to the G bus address of the RF CONTROLLER start address register. Having done this, the G bus interface state of the board is restored to its original state.

Please note: all RF CONTROLLER boards which have their G bus interfaces enabled at the time the Set3031Address function is called will have their start addresses set to frame_address by this function.

Function timing: The Set3031Address function takes 2.9 μ s plus the time taken to evaluate the frame_address and board_address arguments.

3.4.13 Set3031LoopMode

```
void Set3031LoopMode( int LoopMode, int BoardAddress);
```

This function sets the mode of the RF CONTROLLER board indicated by BoardAddress (the value of which should be alias.board1+4 or alias.board2+4) to looping if the value of LoopMode is non-zero, or halt mode if the value of LoopMode is zero.

Function timing: The Set3031LoopMode function takes 1.9 μ s plus the time taken to evaluate the LoopMode and BoardAddress arguments.

3.4.14 Set3031Waits

```
void Set3031Waits( int waits, int board_address);
```

This function enables the G bus interface of the RF CONTROLLER board indicated by the board_address argument (which should be alias.board1+4 or alias.board2+4) then writes the waits argument to the G bus address of the RF CONTROLLER cycles per point register. Having done this, the G bus interface state of the board is restored to its original state.

Please note: all RF CONTROLLER boards which have their G bus interfaces enabled at the time the Set3031address function is called will have their cycles per point register set to the value of waits by this function.

Function timing: The Set3031Waits function takes 3 μ s plus the time taken to evaluate the waits and board_address arguments.

3.4.15 Set_Addr_And_Waits

```
void Set_Addr_And_Waits( int frame_address, int no_waits, int Board1, int Board2 );
```

This function is a concatenation of the Set3031Waits and Set3031Address functions, setting the start address and cycles per point registers of the boards indicated by Board1 and Board2, as well as those of any other RF CONTROLLER boards whose G bus interfaces are enabled when this function is called. This function is particularly intended for use with gradient board sets, where the address and cycles per point of two boards are usually the same.

Function timing: The Set_Addr_And_Waits function takes 5.6 μ s plus the time taken to evaluate the frame_address, no_waits, Board1 and Board2 arguments.

3.4.16 Set_Board_Multipliers

```
void Set_Board_Multipliers( int Mul1, int Mul2, int BaseAddress );
```

This function sets the PF3 and PF4 multiplier values of the board indicated by BaseAddress to Mul1 and Mul2 respectively. The value of BaseAddress should be alias.board1 or alias.board2.

The Set_Board_Multipliers function takes 2 μ s plus the time taken to evaluate the Mul1, Mul2 and BaseAddress arguments.

3.5 Gradient Control Functions

3.5.1 MR3040_SetListAddress

```
void MR3040_SetListAddress( int address )
```

Sets the current address pointer within sequencer memory to the supplied argument. The default initialisation condition is to set this address to 0. The argument is treated as a 16-bit unsigned integer, giving access to up to 65536 instruction locations. This function should be used with care when multiple instruction lists are being used - unpredictable results will occur if a new list is written in the middle of a currently active list.

3.5.2 MR3040_InitList

```
int MR3040_InitList()
```

Creates a single element instruction list, consisting of a STOP instruction at the next available location in sequencer memory. This location shall be the address immediately

following the final address of the current list. Returns the address to PPL, for later use in activating the list. The result should be treated as a 16-bit unsigned integer.

3.5.3 MR3040_Output

```
void MR3040_Output(int loop_type, int frame_address, int frame_rate, int  
frame_points)
```

Writes an OUTPUT instruction to the current list address, increments the list address pointer and reterminates the list with a STOP instruction. When the OUTPUT instruction is encountered, frame_points+1 waveform points are output to the matrix multiplier at a rate of frame_rate+1 units of the master clock time period, starting from waveform store location frame_address. Frame_points and Frame_rate may lie within the range 0 to 4095 inclusive. Frame_address is treated as a 16 bit unsigned integer.

Loops within the sequencer execution may be defined by supplying one of the following manifest constants as the loop_type argument.

NOLOOP	0
STARTLOOP	4 (requires a preceding call to MR3040_LoopCount)
ENDLOOP	8
RETURN	12

NOLOOP causes the instruction to have no effect on the sequencer looping structure.

STARTLOOP identifies the instruction as the first instruction in a loop. A count is associated with the loop by calling MR3040_LoopCount immediately prior to this function.

ENDLOOP and RETURN both identify the instruction as the last instruction in a loop. Once the current instruction has been executed, ENDLOOP causes a jump to the instruction identified by a corresponding STARTLOOP if the count is non-zero, then decrements the count. Hence count+ 1 passes are made through the loop. RETURN causes a jump to the top of the loop irrespective of the value of the count.

If an instruction with a STARTLOOP identifier is encountered while already within a loop, a new nested loop is started, to a depth of 7 loops.

3.5.4 MR3040_Hold

```
void MR3040_Hold( int loop_type )
```

Writes a HOLD instruction to the current list address, increments the list address pointer and determinates the list with a STOP instruction. When this instruction is encountered the waveform is paused at the current output value; the sequencer remains in the running state. The waveform will only continue once it is released using the MR3040_Continue

function. This instruction may be identified as the first or last instruction in a loop using the `loop_type` argument as described above.

3.5.5 MR3040_LoopCount

void MR3040_LoopCount(int loop_count)

Sets a loop count to be used by an immediately following `MR3040_Output` or `MR3040_Hold` function with `STARTLOOP` loop type. The argument may lie within the range 0 to 4095 inclusive. See the `MR3040_Output` description for the significance of this count.

3.5.6 MR3040_SetList

void MR3040_SetList(int address, int channel_id)

Sets the address argument as the start address of the list to be used by the identified channel. The address argument would normally have been returned to PPL by the function `MR3040_InitList()`. This function must only be used if the specified channel is inactive; it has no effect on an active channel. The argument may be one of the following manifest constants:

CHANNEL 1	0x0001
CHANNEL 2	0x0010
CHANNEL 3	0x0100

3.5.7 MR3040_Start

void MR3040_Start(int channel_id)

Starts the identified channel running, executing instructions starting from the sequencer address set using the function `MR3040_SetList()`. This has no effect if the identified channel is already running. The argument may be a combination of any of the manifest constants.

3.5.8 MR3040_Continue

void MR3040_Continue(int channel_id)

Releases the identified channel from the hold state to continue with its next instruction. If the channel is not currently held, the continue command is latched on the sequencer and will take effect immediately on reaching the next `HOLD` instruction. The argument may be a combination of any of the manifest constants described for `MR3040_SetList()`.

3.5.9 MR3040_Break

```
void MR3040_Break( int channel_id )
```

Sets the BREAK condition for the identified channel. This will suppress the jump normally taken by the next instruction encountered with the ENDLOOP or RETURN modifier, so terminating that loop. The argument may be a combination of any of the manifest constants described for MR3040_SetList().

3.5.10 MR3040_Stop

```
void MR3040_Stop( int channel_id )
```

Unconditionally stops the sequencer executing on the identified channel. The current value of the waveform from the identified channel is maintained until the sequencer is restarted. The argument may be a combination of any of the manifest constants described for MR3040_SetList().

3.5.11 MR3040_Control

```
void MR3040_Control( int control_bits )
```

Combination of the four functions above. The argument is passed directly to the Waveform Control port, and may be a combination of any of the following manifest constants:

CHANNEL1_START	0x0001
CHANNEL1_STOP	0x0002
CHANNEL1_BREAK	0x0004
CHANNEL1_CONTINUE	0x0008
CHANNEL2_START	0x0010
CHANNEL2_STOP	0x0020
CHANNEL2_BREAK	0x0040
CHANNEL2_CONTINUE	0x0080
CHANNEL3_START	0x0100
CHANNEL3_STOP	0x0200
CHANNEL3_BREAK	0x0400
CHANNEL3_CONTINUE	0x0800

3.5.12 MR3040_RunChannels

```
void MR3040_RunChannels( int address1, int address2, int address3)
```

Sets the sequencer start addresses for channel x to addressx (x = 1, 2 or 3) then starts the sequencer running on all three channels. This function has the same effect as:

```
MR3040_SetList( address1, CHANNEL 1);
```

```
MR3040_SetList( address2, CHANNEL2);  
MR3040_SetList( address3, CHANNEL3);  
MR3040_Start( CHANNEL1 | CHANNEL2 | CHANNEL3);
```

3.5.13 MR3040_Clock

```
void MR3040_Clock( int duration )
```

Sets the sequencer master clock timebase. The argument specifies the duration in 100ns units and may lie within the range 10 to 4096 inclusive.

3.5.14 MR3040_SeqStatus

```
int MR3040_SeqStatus( int mask)
```

Reads the MR3040 waveform status port and masks it by (i.e. performs a logical AND with) the supplied argument. The argument and return values may be a combination of the following manifest constants:

CHANNEL1_OVERFLOW	0x0001
CHANNEL1_HOLD	0x0002
CHANNEL1_STACK_ERROR	0x0004
CHANNEL1_RUNNING	0x0008
CHANNEL2_OVERFLOW	0x0010
CHANNEL2_HOLD	0x0020
CHANNEL2_STACK_ERROR	0x0040
CHANNEL2_RUNNING	0x0080
CHANNEL3_OVERFLOW	0x0100
CHANNEL3_HOLD	0x0200
CHANNEL3_STACK_ERROR	0x0400
CHANNEL3_RUNNING	0x0800

3.5.15 MR3040_CreateBaseMatrix

```
void MR3040_CreateBaseMatrix( int z_angle, int y_angle, int x_angle,int div_z, int  
mul_z, int div_y, int mul_y, int div_x, int mul_x )
```

Creates a base matrix from which all other matrices will be calculated. The multipliers allow for normalisation of the gradient strength outputs, the divisors allow for normalisation of the waveform file data range to the DAC data range. The angles should be those defining the patient orientation, in 1/10 of degrees. This function creates the product of matrices to the left of matrix P (inclusive) in section 'Oblique Angle transformations' (5.5.5).

3.5.16 MR3040_CreateMatrix

```
void MR3040_CreateMatrix(int matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle )
```

Creates a matrix with identifier `matrix_id` for transforming the current primary waveforms to the gradient output. The `matrix_id` should lie between 0 and 255 inclusive, and should not be the currently active matrix. If the `matrix_id` includes the manifest constant `SECONDARY_MATRIX` (defined as 256), the calculated matrix refers to the secondary rather than the primary waveforms. The angles are those defining the scan orientation with respect to that defined by the `MR3040_CreateBaseMatrix` function. The flip orientation parameter should be 0 for vertical phase-encoding and 1 for horizontal phase encoding. The multipliers should be those obtained from the `FOV`, `SLICE_FOV`, `READ/PHASE/SLICE_THICKNESS` keywords. The divisors should be the values obtained from the `GRADIENT_MODE` keyword, or default to -32768 for the normal isotropic gradient mode. This function creates the complete transformation matrix by post-multiplying the base matrix by **R · T · S** in section 'Oblique Angle transformations' (5.5.5).

3.5.17 MR3040_ModifyMatrix

```
void MR3040_ModifyMatrix(int_dest_matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle, int source_matrix_id )
```

Allows further transformation of an existing matrix (`source_matrix_id`), placing the result in matrix `dest_matrix_id`. **N.B.** the transformations will be defined with respect to the coordinate system defined by the source matrix, so rotations will not normally be additive and the source matrix should have unity scale factors. In section 'Oblique Angle transformations' (5.5.5) this function modifies the source transformation matrix by further post-multiplying the source matrix by **R · T · S**. This allows inclusion of a **R_{user}** matrix in the complete sequence provided the source matrix had identity **T** and **S** components.

3.5.18 MR3040_MatrixStatus

```
int MR3040_MatrixStatus( int mask )
```

Reads the MR3040 matrix status port and masks it by (i.e. performs a logical AND with) the supplied argument. The argument and return values may be a combination of the following manifest constants:

<code>MATRIX_POWER</code>	<code>0x0001</code>
<code>MATRIX_CALCULATING</code>	<code>0x0002</code>
<code>MATRIX_COMPLETE</code>	<code>0x0004</code>

3.5.19 MR3040_SelectMatrix

`void MR3040_SelectMatrix( int matrix_id )`

Sets the identified matrix as the active matrix for waveform output. Unpredictable results will occur if calculations are performed on the active matrix. The argument may lie in the range 0 to 255 inclusive.

3.6 Instrument parameters (WIN.INI)

The following table describes the RF parameters supported by ParSetup. Their initial values can all be found within the [smisdefs] section of c:\windows\win.ini.

VARIABLE	RANGE
"ObserveTransmitAttenuation"	-459 ~ 0 (dB10 units)
"DecoupleTransmitAttenuation"	-459 ~ 0 (dB10 units)
"ObserveReceiverGain"	0,60,120,180,240,300,360 (dB10 units)
"ObserveReceiverAttenuation"	-150 ~ 0 (dB10 units)

Chapter 4. INTRODUCTION TO PPL

The PPL language was designed to allow a programmer easy access to the MR3090(p) input and output ports, and to facilitate the specification of accurately timed delays and pulse sequences. The PPL compiler converts the PPL source file into a compiled version of the program, which can be simply downloaded and executed on the MRS board.

The following sections guide the user through simple demonstrations of all the features of PPL.

To allow these programs to be run, they must first be compiled, and have a parameter file associated with them. The compiler is started using the icon in the MRI Software toolbar. After it is open, enter the path and filename of the pulse sequence program file, and press enter. If there are any error messages, check through the program for typographical errors, and try again. Close the compiler, then, open the Parameter Editor from the toolbar. Click File - New in the menu, and select the program file. Then save the parameter file with an appropriate name. Note: not all of the tutorial programs require any parameters in the file.

To run a program, open Scan from the NMR Software toolbar, and open the parameter (.PPR) file for that program. Then click on Setup. The outputs will appear in a terminal window, along with a prompt for data entry. Some of the tutorial programs will produce the error message:

Scan aborted with status

DSP = '255', MR3020 = '6'

This is prompted by the fact that the spectrometer is not receiving any data, and is nothing to worry about. Click Abort to end the program setup.

4.1 Console input and output

The following program demonstrates simple console input and output, using the read and printf functions. Please note that the comment section headed PARAMLIST must be included.

```
/*  
PPL tutorial program 1.  
Demonstration of console input and output.  
*/  
  
/* PARAMLIST  
END  
*/
```

```
main()
{
    int number;

    /* Prompt for a number to be entered from the keyboard */
    printf("Enter a decimal number :");
    number = read(d);

    /* Print the value of the number out in hexadecimal */
    printf("\n Hexadecimal value of number = %x\n", number );

    /* Print the value of the number out in decimal */
    printf("Decimal value of number = %d\n", number );
}
```

“number” is declared to be a variable of type **int**. The function **read(d)** expects a decimal number terminated by the **Enter** key to be input from the keyboard, and returns this value.

The **%x** characters in the second **printf** string mean “print the value of the corresponding argument (in this case 'number') as a hexadecimal value”. Likewise, the **%d** characters in the third **printf** string mean “print the value of the corresponding argument as a decimal value”. The **\n** characters can occur anywhere in the **printf** string (apart from in the middle of **%d** or **%x**) and are printed as a carriage return and line feed on the console screen i.e., a “newline”.

The **main()** { at the top of the program marks the beginning of the program, and the **}** at the bottom marks the end of the program.

The output of the tut1 program if compiled, downloaded and executed would be as follows. (User input is underlined, MR3090(p) output is in **bold**).

```
Enter a decimal number: 254
Hexadecimal value of number = FE
Decimal value of number = 254
```

4.2 Numbers and constants

Numbers appear in a variety of guises in a PPL program; the following program demonstrates all the possibilities which PPL allows.

```
/*
PPL Tutorial program 2.
Demonstration of number styles and constant declarations.
*/
```

```
/* PARAMLIST
END
*/

main()
{
    const num12 12;    \\ First character in number is in range 1..9
    const mask1 0xaa;  \\ First characters in number are 0x
    const mask2 0X55;  \\ First characters in number are 0X
    const octal_num 0123; \\ First character in number is 0, followed by a digit in range 0..7
    const minus24 num12*2; \\ Constant being defined in terms of constant expression
    /* Print out the decimal values of all the above */
    printf("Value of num12 = %d, mask1 = %d, mask2 = %d\n", num12, mask1, mask2);
    printf("Value of octal_num = %d, Literal num = %d\n", octal_num, 123);
    printf("Value of minus24 = %d\n", minus24);
}
```

The **const** declaration is a useful device for giving a number a meaningful name. The syntax of a **const** declaration is:

const identifier value;

Wherever the identifier appears subsequently in the program the value is substituted at compile time. A judicious use of **const** enables a program parameter which is used in several places to be changed by updating a single **const** declaration value, rather than having to change the number wherever it is used in the program.

Values themselves are available in three bases: decimal, hexadecimal and octal. The compiler determines the base used by inspecting the first one or two characters of the value.

Firstly, if the first character is a digit rather than an alphabetic character or underscore, the compiler “knows” that it is dealing with a number rather than an identifier, and applies the following rules to determine the base:

- (1) If the first characters are 0x or 0X, the value following is a hexadecimal number.
- (2) If the first character is a 0, and the next is in the range 0..7, the value is an octal number.
- (3) If the first character is in the range 1..9, the value is a decimal number.

All these bases are used in the program example above, which produces the following output if compiled and run:

Value of num12 = 12, mask1 = 170, mask2 = 85
Value of octal_num = 83, Literal num = 123

Value of minus24 = 24

Please note: the first two **printf** statements “spill over” into two lines. This is perfectly legal in PPL: “white space” characters (spaces, tabs and newlines) are ignored by the compiler as long as they do not occur within identifiers, numbers or **printf** control strings.

The declaration of minus24 has been included to demonstrate the use of a constant expression in defining a constant. All the operands of the expression are “known” at compile time when the declaration is reached, and so a new constant can be defined in terms of previously defined constants.

The usefulness of constants and constant expressions will become much more apparent when we start looking at the assignment of meaningful names to lines in the input and output ports and pulse generation.

Indentation of text in a PPL source file is not compulsory, but is encouraged as good programming style.

The characters `/*` and `*/` delimit a comment: any text between them is ignored by the compiler: they may not be nested.

Another type of comment has also been used in the programs above; the characters `\n` tell the compiler to ignore any following text up to a new line.

4.3 Expressions

PPL has a small set of arithmetic binary operators `+`, `-`, `*`, and `/`, all of which operate on constants, variables or literal numbers. This enables simple arithmetic to be performed in a program. For example, one can write expressions such as

$$a = 2*b + c * d / (x + y);$$

where `a` has been declared as a variable, and all the other identifiers have been declared to be either variables or constants. Expressions in brackets are evaluated first.

In addition, PPL includes the relational operators `>` (greater than), `>=` (greater than or equal to), `<` (less than) and `<=` (less than or equal to).

PPL also includes the Boolean binary operators `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise EXCLUSIVE OR) and `==` (EQUALS). In PPL, a non-zero value is taken to be logic 1 (i.e. “true”) and a zero value is taken to be logic 0 (i.e. “false”).

The result of an EQUALS operation is zero if the left hand operand is not identical in value to the right hand operand, and 1 ("True") if the values of left and right hand operands are identical.

Thus one may use these operators to (amongst other things) set up masks for i/o as in the following PPL code fragment:

```
const line_0 0x1;
const line_1 0x2;
const line_2 0x4;
const line_3 0x8;
const line_4 0x10;
const lower_nibble_mask line_0 | line_1 | line_2 | line_3;
```

There are also three unary operators: `-`, `!` and `~`.

- `-` is simply the unary minus operator, and negates the value of any expression following it.

- `!` is the logical NOT operator: the logical value of the expression following it is inverted. Hence a non-zero ('true') value becomes zero, and a zero ('false') value becomes 1 ('true').

- `~` is the bitwise 1's complement operator: All set bits in result of the expression following it are cleared, and all unset bits are set. Thus **!0xff** (0xff = all bits in low order byte set) is equal to **0xff00** (all bits in high order byte set, all bits in low order byte cleared).

The order of precedence of the operators is as follows:

()	Bracketed expressions evaluated first,
! ~	then unary operators,
* /	
+	
< <= > >=	
== !=	
&	
^	
=	and finally the assignment operation is performed last.

Thus the expression

```
a = (10+9)*2+3;
```

would result in variable “a” being set to the value 41.

Please note: Remember that all values are 16 bit integers; the results of divisions will be truncated (rounded down), and if the result of a +, or * operation will not fit into 16 bits the result given will be the least significant 16 bits of the 'true' result. The programmer must be sure that the values used in expressions are representable in 16 bits.

4.4 Program flow control (loops and branches).

PPL provides conditional and unconditional branching to control the order of execution of the code. Conditional branching is provided by the **if-else** and **do-until** constructs, whose use is demonstrated in the example program below.

```
/*
PPL Tutorial program 3.
Demonstration of conditional branching.
*/
/* PARAMLIST
END
*/

main()
{
    int a;

    printf("Enter a positive decimal value :");
    a = read(d);
    printf("\n");

    /* if else construct ... */
    if( a == 3 ) printf("a = 3 ... statement following if executed\n");
    else printf("a not equal to 3 ... statement following else executed\n");
    printf("Out of conditional branch this statement is always executed\n");

    /* do until construct */
    do
    {
        printf("In do until construct a = %d\n", a);
        a = a + 1;
    }
    until( a == 0 );
}
```

The **if-else** construct works as follows. The expression in brackets following the if is evaluated, and if it is non zero the compound statement following is executed.

(A compound statement is either a single statement or any number of statements enclosed within `{ }`.)

If the expression evaluates as zero, the compound statement following is not executed. However, if an **else** appears after the compound statement following the **if**(expression), the compound statement following the else is executed if and only if the expression evaluates as zero.

After an **if**(expression) compound_statement construction, an **else** compound_statement construction is optional. The **do_until** construct consists of the word **do** followed by a compound statement, then the word **until** followed by a bracketed expression and a semicolon.

In this case, the compound statement is always executed at least once. Then the expression is evaluated, and if it is zero, the compound statement is executed again. This process is repeated until the expression evaluates as non-zero. Thus, if the program above was run we could see output as follows. (User input is underlined).

Enter a positive decimal value :2

a not equal to 3... statement following else executed

Out of conditional branch this statement is always executed.

In do - until construct a = 2

In do - until construct a = 1

Run again with a different input, we would see

Enter a positive decimal value :3

a = 3...statement following if executed

Out of conditional branch this statement is always executed

In do until construct a = 3

In do until construct a = 2

In do until construct a = 1

Unconditional branching is provided by the **for** and **goto** constructs.

The syntax of the **for** construct is as follows:

for(expression)

compound_statement

The expression is evaluated, and the compound statement is executed this value +1 times. Thus even if the expression evaluates as zero, the compound statement is always executed at least once.

The “extra” loop iteration involved here may seem unfortunate to those users familiar with other languages - it has been implemented this way in the Pulse Programming Language

to make use of the RTX2000 processor's optimized looping instructions for the sake of speed. The PPL compiler could have been written to subtract one from the loop count expression, but this would involve several extra machine instructions being executed at run time, slowing the execution of the PPL program.

The user must be aware of the syntax of the **for** construct, and to organise loop counts appropriately.

Please note: The bracketed expression following the **for** word is only evaluated once, so in the example below, the compound statement following the **for(i)** is executed five times even though the value of *i* is changed during the execution of the compound statement.

The following program demonstrates the use of the **for** and **goto** control structures.

```
/*
PPL Tutorial program 4.
Demonstration of unconditional branching.
*/

/* PARAMLIST

END
*/

main()
{
    const loop_count 4;
    int i;

    i = loop_count;

    for(i)
    {
        printf("In for loop i = %d\n", i = i - 1 );
    }
    goto label0;

    printf("This statement is skipped by the goto above\n");
    i = i - 1; \\ This statement is skipped as well
label0:
    printf("The value of i = %d\n", i );
}
```

Please note: In the **printf** statement, the value of *i* is being assigned to.

Wherever an expression may occur in PPL, an assignment may be performed, and the result of the assignment expression is the value that was assigned.

The **goto** label statement causes the execution of the program to branch to the statement immediately after the label. The label marking the statement to be 'gone to' must be followed by a colon, and is not declared at the top of the program.

Any legal identifier can be used as a label, and it may precede or follow its **goto** statement. Thus in the example above, the for loop is executed 5 times (4+1), and the value of *i* after the loop is exited is 1. Since the ***i* = *i* - 1;** statement after the **goto label0;** statement is never executed, the value of *i* which is finally printed will be -1, not -2.

4.5 Input and Output to the User Port

The input side of the port is only 8 bits, rather than the 16 bits usually dealt with by PPL. However, the **input()** function converts the eight bit value read in from the port into 16 bits as follows:

an **input(expression)**; returns a value which is the bitwise AND of the expression with the 16 bit value formed by placing the 8 bits read in from the port into the least significant byte, and zeroing the most significant byte, i.e., **input(expression)**; returns (port value) & (expression). Thus **a=input(255);** will set *a* equal to the (eight bit) value read in from the port.

Since one is frequently only interested in the state of one bit of the port, the primaries **input#0, input#1... input#7** are provided in PPL. They may be used wherever one would put a variable, constant or number, and return a value which corresponds to the status of the appropriate bit of the input port. Thus **input#0** returns a value of 1 if bit 0 of the input port is set, zero otherwise.

input#1 returns a value of 2 if bit 1 of the input port is set ... **input#n** returns a value of 2ⁿ if bit *n* of the input port is set, zero otherwise.

Values are sent to the output port by the function **output(expression)**; The value of the expression is evaluated, and then sent to the output port. The program below provides a simple demonstration of input and output functions usage:

```
/*
PPL Tutorial program 5.
Demonstrating the use of the input and output ports
using PPL's high level input and output facilities.
*/

/* PARAMLIST
END
*/

main()
{
```

```
const bit0 1;
const bit3 8;
const bit0_or_bit3 bit0 | bit3;
const evens 0xaaaa;
const odds 0x5555;
int in_val;

do
{
    output( evens ); \\ Set even bits, clear odd bits
    output( odds ); \\ vice versa
} until( in_val = input( bit0_or_bit3 ) );

if( in_val == bit0 )
{
    printf("Bit 0 of input port set\n");
}
else
    if( in_val == bit3 ) printf("Bit 3 of input port set\n");
    else printf("Bit 3 and bit 0 of input port set\n");
}
```

The program will toggle the output lines in a 'checkerboard' pattern until either or both of the input lines 0 and 3 are set, at which time the program will report which condition caused it to stop looping.

Please note: if the above program were compiled with the printing of warnings allowed (the default case), a warning message about the use of an assignment operation in the **until** expression would be printed. In this instance the warning may safely be ignored, but in general it is much more common in expressions in conditional branches for the EQUALS operation to be performed rather than an assignment, and so a warning is generated if an assignment is encountered where one would normally expect an equivalence operation. The expression in the **until()** construct could have been written as

```
inp_val = input#0 | input#3;
```

but this would have had a subtly different result: the input port is read for each **input#n**, so in the above expression the port would be read twice, and so it would be possible bit 0 to be set, read by **input#0**, cleared, bit 3 set and then read by **input#3**, resulting **inp_val** being set to $2^0 \mid 2^3 (=9)$. The program would then cease looping and report that both bits 0 and 3 had been set which was true, but not necessarily simultaneously.

It is worth bearing in mind that if more than one input bit is to be tested, using `input( n )` may well be safer than an expression involving several **input#n**'s.

4.6 Delays and Timing functions

The **delay** functions place the processor in a precisely timed loop during which it “does nothing”. These functions provide the programmer with a very accurate means of timing for outputting pulse sequences.

The simplest **delay** function is called with two parameters, a value and an order, thus.

delay(num, order);

order is one of: **us** for microseconds or **ms** for milliseconds or **sec** for seconds

num must be either a variable or a constant expression. The above style of **delay** causes the processor to effectively “wait” num seconds, milli or microseconds, depending on the value of order, before going on to execute the next statement.

The shortest possible delays depend upon the type of object which is used to provide the Time value, and the Order. These shortest possible values for Time are tabulated below:

TYPE OF OBJECT	ORDER	SHORTEST POSSIBLE VALUE
const or number	us, ms, sec	0
int	us	2
	ms	1
	sec	1
common int	us	3
	ms	1
	sec	1
int array element	us	2
	ms	1
	sec	1

The upper limits of the length of delay which it is possible to specify with a single delay function are as follows:

Upper limit for seconds = 65535 seconds using Order = sec and Time = a constant or a variable. Upper limit for milliseconds = 65535 milliseconds using Order = ms and Time = a constant or a variable.

A more sophisticated version of **delay** is also provided. This style of **delay** is called the “incrementing” **idelay**, and is passed three arguments rather than two, thus:

idelay(var, inc, order);

order is as defined before. Var, however, must be a variable as it is incremented by inc (which can be either a variable or constant expression) to provide the length of delay. This incrementation happens before the delay is started. On leaving the incrementing **idelay**, the variable retains its incremented value.

The minimum delay which can be achieved using the **idelay()** function is dictated by the order argument. The table below summarizes the minimum values for the expression *var + inc* for each value of order:

order	Minimum value for <i>var + inc</i>
sec	1
ms	1
us	5

Please note: on exit from the **idelay()** function the value of the var variable will be the incremented value.

The following program demonstrates the use of the delay functions in conjunction with the output function to generate a simple pulse sequence.

```
/*
PPL Tutorial program 6.
Demonstration of pulse sequence generation using incrementing
delays.
*/

/* PARAMLIST
END
*/

main()
{
    const line_90_on 0x1;
    const line_180_on 0x2;
    const start_sampling 0x10;
    const all_off 0;
    const increment 6;
    int pulse_width;

    pulse_width = 0; \\ Initialise variable for use in
                    \\ incrementing delay

    /* Allow the system to settle for 1 second before
       starting sequence */
    output( all_off );
    delay( 1, sec );

    /* Send a 6 microsecond pulse to line_90 */
    output( line_90_on );
```

```
idelay( pulse_width, increment, us );
\\ pulse_width + increment = 6
output( all_off );

/* Wait ... */
delay( 40, ms );

/* Send a 12 microsecond pulse to line_180 */
output( line_180_on );
idelay( pulse_width, increment, us );
\\ Now pulse_width + increment = 12
output( all_off );

/* Wait ... */
delay( 40, ms );

/* Data ready for collection */
output( start_sampling );
}
```

This program sets up the following simple sequence: line_90 (output line0) goes high for 6 microseconds, then a delay of 40 milliseconds is performed before a 12 microsecond pulse is sent to line_180 (output line 1). After a further delay of 40 milliseconds a start_sampling signal is issued by sending output line 4 high.

Here we are assuming that there is a sampling device which is configured to start sampling when this line undergoes a low to high transition.

noop statement

The noop; statement takes 100 ns to execute. It is provided to allow sub microsecond delays.

For example, the following PPL fragment generates a 500 ns pulse:

```
output(0);
output(1);
noop;           \\ do nothing for 100 ns
output(0);       \\ output of a constant takes 400 ns
```

starttimer, gettimer, waittimer

A second timing mechanism consists of these three functions.

void starttimer()

This starts an internal clock for timing sections of the sequence code. It is the status of this clock that is used by **waittimer** and returned by **gettimer**. The clock cycles from 0 to 49999 and then restarts at 0, and each tic corresponds to 0.1 μ s. Therefore a value of 10000 would correspond to a time of 1.0000 ms, 6.0000 ms, 11.0000 ms, 16.0000 ms etc.

This function requires 0.5 μ s to execute.

int gettimer()

Returns the number of 100 ns clock cycles taken since a call to **starttimer()**. The time overhead taken by the timing functions themselves (360.2 ± 5.0 us after evaluation of arguments) is compensated within the **gettimer()** function. The code generated by the compiler for the code segment "variable = gettimer();" stops the timer before making the assignment, so the time returned is exactly that taken by the intermediate code. The maximum period that can be timed by these functions is **3276.7 μ s**, beyond which the timer first generates negative values (the timer output is unsigned while the print

Please note: values between 32767 and 49999 will be printed (using `printf("%d",value)` as signed numbers, i.e. negative value equal to the positive value less 65536.

formatting is two's complement signed), then wraps round again to zero after 65535 counts.

void waittimer(int exit_time)

This function is used in conjunction with **starttimer** to provide accurately timed sections of code, without the need to determine the execution time of each intervening operation. This function will exit **exit_time** cycles of 100 ns after the most recent entry to **starttimer**. More than one call to **waittimer** may be made following a single call to **starttimer**, allowing indexing of multiple time points to a single reference.

The minimum time interval occupied by a **starttimer()** ... **waittimer()** section is 3.0 μ s, and this is achievable when there is no code between the bracketing function calls. If the requested **exit_time** is shorter than the minimum allowed by the intervening code, the time occupied by the bracketed code will be increased by complete units of 5 ms, until a time longer than that occupied by the intervening code (plus function call overheads) is found. **exit_time** values of up to 6.5500 ms may be requested, but the time will be extended by 5 ms if the intervening code occupies more than 4.9998 ms. Unpredictable delays will result if the function **HostRequest** appears within the intervening code.

Usage example:

```
MR3031_go()           /* start some gradient activity */
starttimer();         /* mark the time */
SetObsModGain(dB10);  /* variable length function: approx. 1.25ms */
waittimer(20000);     /* exits exactly 2ms after entry to starttimer */
starttimer();         /* reset the timing reference point */
rfon(4);              /* start transmitting rf */
Do_some_maths....     /* variable time <0.98ms */
waittimer(10000);     /* exits exactly 1ms after 2nd entry to starttimer, 3ms after
1st entry */
rfon(3);              /* Step down the transmitter power after 1ms radiation */
Do_some_more_maths... /* variable time... */
waittimer(20000);     /* exits exactly 2ms after 2nd entry into starttimer */
MR3031_setup(.....); /* Prepare some different gradient activity */
waittimer(25000);     /* Wait until 2.5ms through rf */
MR3031_go();          /* Start the new gradient activity */
waittimer(40000);     /* exits exactly 4ms after entry into starttimer */
rfon(4);              /* Put the rf power up again, 3ms after cutting it */
waittimer(0);         /* wait exactly 5ms through rf, timer cycled at 5ms */
rfon(0);              /* Cut the rf power */
```

4.7 Pulse Generation

The pulse function has been included in the PPL language to simplify the generation of pulses on the output lines of the MR3090(p). The idea of the pulse function is to replace the somewhat cumbersome sequence of PPL commands

```
output( lines_on );      \\ Start of pulse
delay( n, us);          \\ Duration of pulse
output( lines_off);     \\ End of pulse
```

with a single function thus:

```
pulse( lines, units, order, thousandths );
```

which allows any single output line or combination of output lines to be pulsed, whilst leaving the lines which are not to be pulsed in whatever state they were prior to the execution of the pulse function.

The "lines" argument must be either a single variable or a constant expression. The output port lines corresponding to non-zero bits of the sixteen bit value represented by the "lines"

argument will be toggled ("toggling" a line means inverting the state of the line) by the pulse function, and the output port lines corresponding to bits which are zero in the "lines" argument will be unaffected by the pulse function.

Thus giving the pulse function a value of 0x45 (hexadecimal 45 = binary 1000101) will result in the pulse appearing on lines 0, 2 and 6 of the output port. All the output lines specified by non-zero bits of the "lines" argument to the pulse function will be toggled simultaneously. The output lines specified by the bits of the "lines" argument which **are** zeros will be left unchanged by the pulse function.

The duration of the pulse is controlled by the three arguments "units", "order" and "thousandths".

The "order" argument must be one of sec, ms or us corresponding to a pulse width of the order of seconds, milliseconds or microseconds. The "units" argument, which must be either a single variable or a constant expression, specifies the width of the pulse in whatever order has been selected by the "order" argument. The "thousandths" argument specifies the number of thousandths of whatever order has been selected which will be added to the pulse width. The "thousandths" argument can be either a single variable or a constant expression if the "order" argument is **sec** or **ms**, but must be a constant expression (not a variable, as the time taken to access a variable value is significant here) if the "order" argument is **us**.

It should be noted that when an order of **us** is used, as the timing resolution available on the pulse programmer is 100ns, the constant expression given as the "thousandths" argument to the pulse function is rounded to the nearest 100ns when an order of **us** is being used.

The following examples should help to clarify the use of the pulse function.

```
/* Simple example of a 2.1 ms pulse on lines 3 and 4, line 3 being pulsed low, and line 4 being
pulsed high */
main()
{
    const line_3 0x08;
    const line_4 0x10;

    /* Output on line_3 to set line 3 high prior to pulsing, and all other lines low. */
    output( line_3 );
    /* The constant expression line_3 | line_4 is a bitwise OR
    of the constant values 0x08 and 0x10 = binary 11000, and
    hence specifies lines 3 and 4 as the lines to be toggled
    by the pulse function. The value of 100 in the
    "thousandths" argument means 100 thousandths of a
    millisecond, thus the total pulse width is 2.1 ms.
    Since line_3 is high on entry to the pulse function
    having been set high by the output function above, when
    it is toggled by the pulse function it is sent low for
    the duration of the pulse. Line 4 is already low on
    entry to the pulse function, so it is sent high for
```

```
        the duration of the pulse.  
    */  
    pulse( line_3 | line_4, 2, ms, 100);  
}
```

Below is an example of the pulse function being used with an order of us to demonstrate how the rounding of the “thousandths” argument is performed, and also demonstrating the use of a variable in the “units” argument.

```
main(){  
    const line_3 0x08;  
    const line_4 0x10;  
    int width;  
  
    width = 11;  
  
    output(line_4); \\ Setting up port prior to pulse  
  
    /* On entry to the pulse function in this example, line 4  
       is high, and all other lines low, so line 4 will be  
       pulsed low and line 3 will be pulsed high whilst all  
       other lines remain unaffected at their initial low value  
    */  
  
    pulse( line_3 | line_4, width, us, 140);  
  
    /* The “thousandths” argument above is rounded to the  
       nearest 100ns, which in this case is 100ns, thus the  
       pulse width specified above is 11.1 microseconds.  
       N.B. See Pulse Function Timing considerations */  
}
```

Please note: for the duration of the pulse function, (i.e. the pulse width plus a few hundred ns depending upon the argument types - this is described in detail in the pulse function timing section on page 87) the output port cannot be written to by the output function, as the next PPL statement after the pulse function is not executed until the pulse function has finished.

Thus the output(value); delay(time, order); output(value); sequence is not entirely redundant. The delay involved before the output port lines are changed is still less for the output function than it is for the pulse function (see the pulse function timing section on page 87 and output timing section on page 87), and so an extremely rapid sequence of pulses may be better produced by using an

```
output(...);  
delay(...);  
output(...);  
delay(...);
```

...

sequence rather than a sequence of pulse functions.

4.8 Pulse Programming Language (PPL) reference

This section of the manual describes the syntax of all features of PPL, and the time taken by the pulse programmer to execute each operation. Each operation and function and its execution time are described individually in the following sections.

The main elements of the PPL language are:

- Input and Output
- Delays
- Conditional and Unconditional branching.

Input and output statements are directed at the "G" port address G0. Delays are generated by placing the processor in a precisely timed loop; they can be specified in seconds, milliseconds or microseconds.

Looping structures, conditional branches and unconditional branches direct control flow within the program. The following sections describe all the features of the PPL language, and the time taken to execute each function and operation.

PPL Syntax - typographical conventions

All PPL reserved words and symbols are printed in **bold**. Terms which appear between `[]` are optional: they can appear zero or many times.

Identifiers are strings of up to 31 alphanumeric or underscore(_) characters of which the first character must be alphabetic. The case (i.e. UPPER CASE or lower case) of alphabetic characters is not significant in identifiers.

The symbol `:=` is shorthand for "is defined as". It is not a PPL symbol.

4.8.1 Comments.

Any text between the characters `/*` and `*/` will be ignored by the compiler. Nesting comments is not allowed. Nested `/* */` comments are not allowed.

Examples of legal comments are shown below:

```
/* This comment is legal */
```

`/* This comment is also legal - a comment may spread across any number of lines */`

The following comment is illegal, as it contains nested `/*` and `*/` characters.

`/* Start of comment1 ... /* Start of illegal nested comment ... */ */`

Single line comments are provided by the characters `\\`. Any text after a double backslash up to the end of a line (terminated with a carriage return) will be ignored by the compiler.

Examples of single line comments:

`\\ This is a single line comment`

`printf("String.\n"); \\ The printf statement function will print String.`

There is no execution time overhead associated with comments.

4.8.2 Variables and constants.

All numbers used in PPL are signed 16 bit values, and all variables and constants are also signed 16 bit values. There are three types of variable in PPL: **int**, **common int** and arrays of **int**. All variables and constants in PPL must be declared at the start of the program.

There is no execution time overhead associated with any type of variable declaration.

The syntax of constant and variable declarations is as follows:

4.8.3 Declaration of constants

const identifier value;

const identifier constant_expression;

Where value is a string of digits which can be represented by a 16 bit value. After an optional minus sign, if the first characters of value are 0x or 0X, the number following is taken to be a hexadecimal number. If the first character is a 0 and the second character is not x or X, the following number is taken to be an octal number. If the first character is in the range 1..9 the value is taken to be a decimal number.

A constant expression is an expression whose operands are all known at compile time (i.e. the operands of the expression are all declared constants and literal numbers), so the expression is evaluated at compile time and the constant set to this value.

Examples of constant declarations:

const useful_hex_number 0xaa;

`\\ 0xaa hex = 170 decimal`

```
const useful_octal_number 010;           \\ Leading zero indicates octal:  
                                         \\ 010 octal = 8 decimal  
const const_expr 123*useful_octal_number;  
/* const_expr has value 123 decimal * 010 octal = 984 decimal */
```

4.8.4 Declaration of int variables

```
int identifier [, identifier];
```

For example to declare an **int** variable called fred for later use in a PPL program, you could use the following line:

```
int fred;
```

4.8.5 Declaration of arrays of type int

```
int identifier[constant_expression];
```

The constant_expression is evaluated at compile time, and represents the number of elements in the array. Its value is used at compile time to determine how much RAM to allocate to the array. Although the declaration of **int** variables and **int** arrays are shown separately above for the purposes of clarity, they may be mixed in the same **int** declaration line in the PPL source file thus:

```
int normal_var, array[12], another_var, another_array[4];
```

Arrays are indexed from 0, so the elements of another_array[] as declared above can be accessed with indices 0, 1, 2, or 3.

Please note: extreme caution must be exercised when accessing array elements, as to maximize the speed of array element accesses there is no run-time checking of array indices: if you attempt to write into an array element whose index is greater than or equal to the constant_expression value used in the declaration of the array you will corrupt the MR3090(p)'s memory with unpredictable (and possibly catastrophic) results. Likewise, using a negative index will cause corruption of the MR3090(p)'s memory.

The onus is on the programmer to make sure that expressions used for array element indices never under or overstep the bounds of the array.

4.8.6 Declaration of common int variables

common int identifier [, identifier];

All variables of type **common int** are stored in the Dual Port RAM on the MR3090(p), where their values can be accessed both by the host PC and an executing PPL program without interfering with the timing of the PPL program (unless both the host PC and the MR3090(p) processor simultaneously attempt to access the same location in the Dual Port RAM). The following line is example of a **common int** declaration for a variable called `common_var`.

common int `common_var`;

4.8.7 32 bit data type

A declarator "long" has been added to PPL, which can be used to declare 32 bit variables, arrays, function arguments and function return types. To declare a 32 bit variable `VarName`, you use the following syntax in the declarations section of the PPL source code:

- **long** `VarName`;

To declare a 32 bit array `ArrayName` with ten elements, use the following syntax:

- **long** `ArrayName`[10];

To declare a library function `FuncName` which returns a 16 bit value and expects a 32 bit argument `Arg1` and a 16 bit argument `Arg2`, use the following syntax:

- **int** `FuncName`(**long** `Arg1`, **int** `Arg2`);

Note that the argument names are optional in a function declaration - the above declaration could also have been written thus:

- **int** `FuncName`(**long**, **int**);

To declare a library function `FuncName` which returns a 32 bit value and expects a 16 bit argument followed by a 32 bit argument, use the following syntax:

- **long** `FuncName`(**int**, **long**);

32 bit constants and literal values can be declared by appending an 'I' or 'L' suffix to the value. Thus to declare a named constant `ALongConst` having a value of 128000, use the following syntax:

- **const** `ALongConst` 128000I; or **const** `ALongConst` 128000L;

The compiler will then substitute the 32 bit value 128000 whenever the identifier `ALongConst` subsequently appears in the PPL program. The following PPL code fragment shows the use of the 'L' suffix to insert literal 32 bit values into your PPL program:

`long` `ALongVar`;

```
int AnIntVar;  
...  
ALongVar = AnIntVar * 1000000L;
```

4.8.7.1 *Mixing 16 and 32 bit data types*

The PPL compiler follows the same integer promotion rules as the C language for converting between int and long data types. Note that in the rules describing the data type conversions below, int and common int are synonymous. Conversion from int to long automatically takes place if an int value is passed as an argument to a function which expects a long and if an int value is assigned to a long variable. Likewise if one operand of a binary operator is long and the other is int, the int operand is converted to long. Note that all automatic conversions of ints to longs are signed. Conversion from long to int takes place automatically if a long value is assigned to an int variable: the int value is obtained from the least significant 16 bits of the long value. Likewise if a long value is passed as an argument to a function which expects an int value, the long value is converted to an int. If warnings are enabled, the PPL compiler will warn of the possible loss of accuracy which this conversion involves.

4.8.7.2 *Restrictions on 32 bit data types*

The 32 bit long data type cannot (yet) be used in the PARAMLIST section of a PPL file. There is no common long data type.

4.8.8 *Declaration of library functions*

Library functions are routines which are defined in the library file which is linked with the code produced by PPL compiler. Library functions will typically be provided by MR Solutions to provide a PPL interface to add-on hardware.

In order to use a library function, you must declare it, so that the PPL compiler “knows” whether or not the function returns a value, and how many arguments the function requires.

A significant difference between library functions and those which are “built-in” to PPL (such as the delay functions described later in this section) is that no compile-time checking is performed on the values of arguments passed to library functions, whereas a limited amount of range checking is performed by the compiler on the values of arguments passed to some of the built-in functions. Thus you have to be extra careful when passing arguments to library functions to make sure that the values of the arguments you pass to these functions are within the range of values which the function expects.

A function which returns a value is declared as follows:

```
int function_identifier(decl_arg_list);
```

Where decl_arg_list:= [**int** [dummy_identifier] [, decl_arg_list]]

The function_identifier is the name of the function as defined in the library. The case of identifiers used in function declarations is not significant.

The string decl_arg_list enclosed between the brackets **()** declares to the compiler the number of arguments which the function expects. This information is subsequently used by the compiler when it encounters calls to library functions to check that the correct number of arguments have been supplied.

Each argument declared with the keyword **int** may be followed by an optional dummy identifier. This dummy identifier is ignored by the compiler, and is intended to allow the use of meaningful strings in the function declaration.

The use of the **int** keyword in the function argument declaration list indicates that functions take 16-bit values as arguments. It does not prevent the use of **common ints**, **int** array elements or functions returning **int** values as arguments.

An example of a library function which returns a 16 bit value is the **kbhit** library function. In order to use this function, it must be declared thus:

```
int kbhit();
```

This function takes no arguments, and returns a 16-bit value. One might use it in a do ... until loop, for example, to allow a set of PPL statements to be executed until a keypress was performed on the console keyboard thus:

```
main()
{
    int kbhit(), value;

    do{
        /* any number of PPL statements ... */
    }
    until( kbhit() );
    /*
    When this point is reached, a keypress must have occurred,
    and a read() function call is required to clear the input
    buffer.
    */
    value = read(d);
    /* Further PPL statements ... */
}
```

Library functions which do not return a value must be declared as being of type **void**. Their declaration syntax is as follows

```
void function_identifier(decl_arg_list);
```

where the definition of `decl_arg_list` is the same as that given above for functions which do return a value.

Library function arguments are all passed by value: that is, the value of the variable used as a function argument is not changed in the routine which calls the library function. If the value of a variable is to be changed by the action of a library function, the only way this can be achieved is by assigning the variable to the result of the function. For example, suppose we have a library function called "func" which returns the sum of its arguments declared thus

```
int func( int operand1, int operand2 );
```

and we pass it the variables `a` and `b` as arguments, and assign the result of the function to the variable `c` (where `a`, `b`, and `c` are declared as variables of type `int` or common `int`) thus:

```
c = func( a, b);
```

The values of `a` and `b` will be unchanged by the function, `func`. The value of `c` will be changed.

4.8.9 *Library functions*

Note that in the following descriptions of each of the new library functions the function declaration is given to show the function argument and return types to the reader: if you use the `#include <STDCONVS.PPH>` directive at the top of your PPL source code you should not declare the function in the declarations section of your main PPL function as well - the PPL compiler will issue error messages about redeclarations if you do.

4.8.9.1 *UnsignedToLong*

`long UnsignedToLong( int n);`

Execution time: 0.2 microseconds + argument evaluation time. The `UnsignedToLong` function treats the 16 bit value of `n` as unsigned when converting to long. For example,

```
#include <STDCONVS.PPH>
```

```
main()
{
    int n;
    long LongVal;

    n = -1;
    LongVal = UnsignedToLong(n);
    printf("n = %d : Unsigned conversion to long = %ld", n, LongVal);
}
```

The above code fragment gives output `n = -1: Unsigned conversion to long = 65535`

4.8.9.2 *IntToLong*

long IntToLong(int n);

Execution time: 0.5 microseconds + argument evaluation time. The IntToLong sign extends its 16 bit argument to return a 32 bit value, for example,

```
#include <STDCONVS.PPH>
main()
{
    int n;
    long LongVal;

    n = -1;
    LongVal = IntToLong(n);
    printf("n = %d : Unsigned conversion to long = %ld", n, LongVal);
}
```

The above code fragment gives output n = -1: Conversion to long = -1.

4.8.9.3 *UnitThouMillToLong*

long UnitThouMillToLong(int n1, int n2, int n3);

Execution time: 22.2 microseconds + argument evaluation time. This function returns the 32 bit value corresponding to $n1 + n2 * 1000 + n3 * 1000000$, for example,

```
#include <STDCONVS.PPH>
main(){
    long LongVal;

    LongVal = UnitThouMillToLong(1,2,3);
    printf("UnitThouMillToLong(1,2,3) = %ld", LongVal);
}
```

The above code fragment gives output UnitThouMillToLong(1,2,3) = 3002001.

4.8.9.4 *MillThouUnitToLong*

long MillThouUnitToLong(int n1, int n2, int n3);

Execution time: 22.2 microseconds + argument evaluation time. This function returns the 32 bit value corresponding to $n1 * 1000000 + n2 * 1000 + n3$. The order of arguments to this function has been chosen to match those of the frequency library function - i.e. void frequency(int MHz, int KHz, int Hz), for example,

```
#include <STDCONVS.PPH>
main(){
    long LongVal;
```

```
LongVal = MillThouUnitToLong(123,456,789);  
printf("MillThouUnitToLong(123,456,789) = %ld", LongVal);  
}
```

The above code fragment gives output `MillThouUnitToLong(123,456 ,789 ) = 123456789`.

4.8.9.5 *HiWord*

int HiWord(long LongVal);

Execution time: 0.3 microseconds+argument evaluation time. This function returns the most significant 16 bits of its 32 bit argument, for example,

```
#include <STDCONVS.PPH>  
main(){  
    long LongVal;  
  
    LongVal = 123456L;  
    printf("HiWord(%ld) = %d", LongVal, HiWord(LongVal) );  
}
```

The above code fragment gives output `HiWord(123456) = 1`

4.8.9.6 *LoWord*

int LoWord(long LongVal);

Execution time: 0.2 μ s + argument evaluation time. This function returns the least significant 16 bits of its 32 bit argument, for example,

```
#include <STDCONVS.PPH>  
main(){  
    long LongVal;  
  
    LongVal = 123456L;  
    printf("LoWord(%ld) = %d", LongVal, LoWord(LongVal) );  
}
```

The above code fragment gives output `LoWord(123456) = -7616`.

4.8.9.7 *LongToUnits*

int LongToUnits(long LongVal);

Execution time: 121.3 μ s + argument evaluation time. This function returns the remainder of its argument divided by 1000 as a 16 bit value, for example,

```
#include <STDCONVS.PPH>  
main(){  
    long LongVal;  
    LongVal = 123456L;  
    printf("LongToUnits(%ld) = %d", LongVal, LongToUnits(LongVal) );  
}
```

```
}
```

The above code fragment gives output `LongToUnits( 123456) = 456`.

4.8.9.8 *LongToThou*

int LongToThou(long LongVal);

Execution time: 244.9 μ s + argument evaluation time. This function returns `(LongVal/1000)%1000` as a 16 bit value, for example,

```
#include <STDCONVS.PPH>
main()
{
    long LongVal;

    LongVal = 123456L;
    printf("LongToThou(%ld) = %d", LongVal, LongToUnits(LongVal) );
}
```

The above code fragment gives output `LongToThou( 123456) = 123`.

4.8.9.9 *LongToMill*

int LongToMill(long LongVal);

Execution time: 123.8 μ s + argument evaluation time. This function returns `LongVal/1000000` as a 16 bit value, for example,

```
#include <STDCONVS.PPH>
main()
{
    long LongVal;

    LongVal = 789123456L;
    printf("LongToMill(%ld) = %d", LongVal, LongToUnits(LongVal) );
}
```

The above code fragment gives output `LongToMill( 789123456) = 789`.

4.8.9.10 *QuickDiv*

int QuickDiv(int dividend, int divisor);

Execution time: 5.7 μ s + argument evaluation time. This function returns `dividend/divisor` using symmetric division. It is an alternative to the `/` operator for ints when rapid division is required, for example,

```
#include <STDCONVS.PPH>
main()
{
    int n, m;
    n = 23789;
```

```
m = 12345;
m = QuickDiv( n, m);
printf("QuickDiv result = %d", m );
}
```

The above code fragment gives output QuickDiv result = 1.

4.8.9.11 QuickScale

int QuickScale(int a, int b, int divisor);

Execution time: 5.4 μ s + argument evaluation time. This function returns $(a*b)/divisor$ where 32 bit precision is used for the product $a*b$. It is a faster alternative to the built in scale function, for example,

```
#include <STDCONVS.PPH>
main()
{
    int n, m, divisor;
    n = 23789;
    m = 20;
    divisor = 12345;

    m = QuickScale( n, m, divisor);
    printf("QuickScale result = %d", m );
}
```

The above code fragment gives output QuickScale result = 38.

4.8.9.12 FloorDiv

int FloorDiv(int dividend, int divisor);

Execution time: 11.1 μ s + argument evaluation time. This function returns dividend/divisor using floored division. It has been included for back-compatibility with older versions of PPL where the / operator was implemented as a floored division operator. For example,

```
#include <STDCONVS.PPH>
main()
{
    int n, m;

    n = -3;
    m = 2;
    m = FloorDiv(n,m);
    printf("FloorDiv result = %d", m );
}
```

The above code fragment gives output FloorDiv result = -2.

4.8.10 Operators

= Assignment operator.

syntax:

variable = expression;

The value of the variable is set to the value of the expression. There is no execution time overhead for the assignment operation itself when it is used as shown above - what execution time overhead there is in the above statement can be thought of as being the sum of the expression evaluation time and the variable access time. Variable access times and expression evaluation times are summarised in section 4.9.2.

The assignment operator may also be used within an expression to store, for example, an intermediate calculation result. When used in this way, there is an execution time overhead of 100 ns each time the = operator appears within the expression.

These execution time overheads may perhaps be made clearer by considering the execution times involved in the following examples:

a = b; \\ a and b are both declared variables.

The execution time of the PPL line above is simply the sum of the a and b variables' access times. In the following line, we will use the = operator within an expression:

a = b = c; \\ a, b and c are all declared variables.

In the line above, the execution time is the sum of all the variables' access times and an extra 100 ns for the = operator between a and c, as this is an example of the use of the assignment operator within an expression.

A simple rule of thumb for deciding the execution time of an assignment operator is that it always takes 100 ns unless the only PPL symbol to the left of it before the previous; is a declared variable.

Arithmetic binary operators.

PPL provides four binary arithmetic operators; plus, minus, multiply, and divide.

+ Plus operator

syntax:

expression1 + expression2

The above evaluates as the arithmetic sum of expression1 and expression2.

The execution time of the + operator is 100 ns.

- Minus operator

syntax:

expression₁ - expression₂

The above evaluates as the value of expression₁ minus the value of expression₂.
The execution time of the - operator is 100 ns.

*** Multiply operator**

syntax:

expression₁ * expression₂

The above evaluates as the product of expression₁ and expression₂.
The execution time of the * operator is 400 ns.

/ Divide operator

syntax:

expression₁ / expression₂

The above evaluates as the quotient of divisor expression₁ and dividend expression₂.
The execution time of the / operator is 11.1 µs.

Boolean binary operators

PPL provides three binary boolean operators: AND, OR, and EXCLUSIVE OR, each which has an execution time of 100 ns.

& AND operator

syntax:

expression₁ & expression₂

The above evaluates as the bitwise AND of expression₁ and expression₂. The above evaluates as the bitwise AND of expression₁ and expression₂. For example 5 & 2 gives 0 as in binary 5 = 0000 0000 0000 0101 and 2 = 0000 0000 0000 0010 giving 0000 0000 0000 0000 which equals 0 and 10 & 3 gives 2 as in binary 10 = 0000 0000 0000 1010 and 3 = 0000 0000 0000 0011 giving 0000 0000 0000 0010 which equals 2 and -2 & 8 gives 8 as in binary -2 = 1111 1111 1111 1110 and 8 = 0000 0000 0000 0100 giving 0000 0000 0000 0100 which equals 8. This has an execution time of 100 ns.

See also: operator precedence, expressions, ^, ==, &&, |, ^, &, |.

&& Logical AND operator

The `&&` operator result is 0 unless both its operands are non-zero, in which case it returns -1.

Please note: the size of this result is always 16 bits, regardless of the type of the operands.

| OR operator

syntax:

`expression1 | expression2`

The above evaluates as the bitwise OR of `expression1` and `expression2`.

For example `5 | 2` gives 7 as in binary `5 = 0000 0000 0000 0101` and `2 = 0000 0000 0000 0010` giving `0000 0000 0000 0111` which equals 7 and `10 | 3` gives 11 as in binary `10 = 0000 0000 0000 1010` and `3 = 0000 0000 0000 0011` giving `0000 0000 0000 1011` which equals 11 and `-2 | 8` gives -2 as in binary `-2 = 1111 1111 1111 1110` and `8 = 0000 0000 0000 0100` giving `1111 1111 1111 1110` which equals -2. This has an execution time of 100 ns.

See also: operator precedence (4.8.11), expressions, `^=`, `==`, `&&`, `||`, `^`, `&`, `|`.

|| Logical OR operator

The `||` operator result is -1 unless both its operands are zero, in which case it returns 0. Note that the size of this result is always 16 bits, regardless of the type of the operands.

^ EXCLUSIVE OR operator

syntax:

`expression1 ^ expression2`

The above evaluates as the bitwise EXCLUSIVE OR of `expression1` and `expression2`.

For example `5 ^ 2` gives 7 as in binary `5 = 0000 0000 0000 0101` and `2 = 0000 0000 0000 0010` giving `0000 0000 0000 0111` which equals 7 and `10 ^ 3` gives 9 as in binary `10 = 0000 0000 0000 1010` and `3 = 0000 0000 0000 0011` giving `0000 0000 0000 1001` which equals 9 and `-2 ^ 8` gives -6 as in binary `-2 = 1111 1111 1111 1110` and `8 = 0000 0000 0000 0100` giving `1111 1111 1111 1010` which equals -6. This has an execution time of 100 ns.

See also: operator precedence (4.8.11), expressions, `^=`, `==`, `&&`, `||`, `^`, `&`, `|`.

Comparison binary operators

PPL provides 6 comparison operators: GREATER THAN, LESS THAN, GREATER THAN OR EQUAL, LESS THAN OR EQUAL, EQUALS and NOT EQUALS.

== EQUALS Operator

syntax

expression₁ == expression₂

The above evaluates as -1 (non-zero means "TRUE" in PPL) if expression₁ has the same value as expression₂, and zero ("FALSE") otherwise. The execution time of the == operator is 400 ns. See also: operator precedence, expressions, ==,>,<,<=,>=.

!= NOT EQUALS Operator

syntax

expression₁ != expression₂

The above evaluates as non (non-zero means "TRUE" in PPL) if expression₁ has the same value as expression₂, and zero ("FALSE") otherwise. (The exact non-zero value depends on the values of the expressions, as in version 1.00 of PPL the != operator is implemented using an exclusive or operation.) The execution time of the != operator is 100 ns.

< LESS THAN Operator

syntax

expression₁ < expression₂

The above evaluates as -1 (non-zero means "TRUE" in PPL) if expression₁ evaluates to a number which is less than the value of expression₂, and zero ("FALSE") otherwise. Note that the < operator treats its operands as signed values. The execution time of the < operator is 100 ns. See also: operator precedence, expressions, ==,>,<,<=,>=.

<= LESS THAN OR EQUALS Operator

syntax

expression₁ <= expression₂

The above evaluates as -1 (non-zero means "TRUE" in PPL) if expression₁ has the same value or less than as expression₂, and zero ("FALSE") otherwise. Note that the <= operator treats its operands as signed values. The execution time of the <= operator is 200 ns. See also: operator precedence, expressions, ==,>,<,<=,>=.

> GREATER THAN Operator

syntax

expression₁ > expression₂

The above evaluates as -1 (non-zero means "TRUE" in PPL) if expression₁ has a greater value than expression₂, and zero ("FALSE") otherwise. Note that the > operator treats its operands as signed values. The execution time of the > operator is 100 ns. See also: operator precedence, expressions, ==,>,<,<=,>=.

>= GREATER THAN OR EQUAL Operator

syntax

expression₁ >= expression₂

The above evaluates as -1 (non-zero means "TRUE" in PPL) if expression₁ is equal to or has a greater value than expression₂, and zero ("FALSE") otherwise. Note that the >= operator treats its operands as signed values.

The execution time of the >= operator is 200 ns. See also: operator precedence, expressions, ==,>,<,<=,>=.

Unary operators

PPL provides three unary operators: complement, "not" and unary minus.

- unary minus operator

syntax:

-expression

The - operator changes the sign of the value of expression. The above evaluates to the 1's complement of the value of expression. Note that the 1's complement is different to the unary minus operator as

$-2 = -(0000\ 0000\ 0000\ 0010) = 1111\ 1111\ 1111\ 1110$
 $\sim 2 = \sim(0000\ 0000\ 0000\ 0010) = 1111\ 1111\ 1111\ 1101$

The execution time of the - operator is 100 ns.

! not operator

syntax:

! expression

The above changes the boolean value of the expression: if the expression is zero, ("FALSE") it is changed to 1 ("TRUE"), and if expression is non zero ("TRUE") it is changed to zero ("FALSE"). The execution time of the ! operator is 400 ns.

~ complement operator

syntax:

~expression

The above evaluates to the 1's complement of the value of expression. The above evaluates to the 1's complement of the value of expression. Note that the 1's complement is different to the unary minus operator as

$-2 = -(0000\ 0000\ 0000\ 0010) = 1111\ 1111\ 1111\ 1110$

$\sim 2 = \sim(0000\ 0000\ 0000\ 0010) = 1111\ 1111\ 1111\ 1101$

The execution time of the ~ operator is 100 ns. See also: operator precedence, expressions, ~, !

% Remainder operator

The remainder operator is implemented such that expression1 % expression2 returns the remainder of expression1 divided by expression2. Note that the sign of the remainder is the same as the sign of the quotient.

4.8.11 Operator precedence

The order of precedence of the operators is as follows:

()	Bracketed expressions evaluated first,
! ~	then unary operators,
* /	
+	
< <= > >=	
== !=	
&	
^	
=	and finally the assignment operation is performed last.

4.8.12 Scale function

The **scale** function has been provided in PPL to perform the operation $(a * b) / c$, where the intermediate product $a * b$ is a 32 bit value, rather than the 16 bit value normally produced by the PPL $*$ multiplication operator.

This function is provided to allow “scaling” calculations in PPL to be performed without the loss of accuracy inflicted by the use of the 16 bit multiplication operator.

4.8.12.1 **scale()** function.

syntax:

scale(expression₁, expression₂, expression₃);

Return value:

$((\text{expression}_1) * (\text{expression}_2)) / (\text{expression}_3)$

The time taken to execute the scale function is the time taken to evaluate each of the expressions plus 11.1 μs . See also: $*$, $/$, operator precedence.

4.8.13 Console I/O functions

There are two functions read and printf in PPL which allow reading the console keyboard and print formatted text on the console screen. Both these functions require the cooperation of a program running on the host PC to operate - PPE is such a program.

4.8.13.1 **read** function

The read function reads the console keyboard, expecting the user to enter either a decimal or hexadecimal number terminated by a carriage return. If an invalid string is entered, an error message “invalid number” is displayed and the function continues to read the keyboard until a valid value is entered. There must be no space characters in the string of digits typed at the console keyboard.

read() function.

syntax:

read(base);

read(x) returns a hexadecimal number read from the console keyboard.

read(d) returns a decimal number read from the console keyboard.

The execution time of this function is inevitably variable, depending as it does on both the typing skills of the user and the speed of communications between the host PC and the MR3090(p). As a result of this variable execution time, it is inadvisable to use the read function in those sections of your code where the timing is critical.

4.8.13.2 *printf* function

The **printf()** function provides a method of producing formatted text and number output to the console.

printf() function.

syntax:

printf("control string" argument list);

The control string consists of text which is to be output to the console, and format specifications controlling the format of the printing of the corresponding argument in the argument list. The **printf** function does not return a value.

The argument list consists of [, expression] (Recall that objects in [] can occur zero or more times). What constitutes an expression is defined on page 80.

A format character, %, must appear in the control string in order to print out the value of an expression in the argument list. It must be followed by **d** or **x** to control the style used to print the value of the expression.

%d causes the value to be printed as a signed decimal value. **%x** causes the value to be printed as a hexadecimal number. Numbers are printed in a free field format with a trailing space.

Thus, the following PPL fragment (with a previously declared as an int)

```
a = 15;
printf("The decimal value of a=%d\n",a);
printf("The hex value of a=%x\n",a);
```

will result in the following console output.

The decimal value of a =15

The hex value of a =F

The characters **\n** may appear anywhere in the control string (apart from between a % and **d** or **x**) and result in a carriage return and linefeed being printed on the console. The characters **%%** appearing in the control string will result in a single % being printed on the console.

The execution time of the **printf** function is variable, depending upon such factors as the number of characters in the format string, the complexity of the expressions which need to be evaluated in the argument list and most importantly the speed with which the host PC communicates with the spectrometer. As a result of this variable execution time, it is inadvisable to use the **printf** function in those sections of your code where timing is critical.

4.8.14 Expressions

An “expression” as used in the above definitions is as follows:

expression:= [unary op.] primary [binary op. [unary op.] primary]

Terms in [] are optional: any number or zero can appear.

A primary is any of the following: a variable, a constant, a number, one of PPL's **input** functions, the PPL **scale** function, a library function declared as returning **int** or an expression. A variable can be an array element, an int or a common int.

The following are all legal expressions:

a = ~(b * input(0xf0)); \\ a must be an int or common int
 \\ as it is being assigned to.
 \\ b can be either an int, common int
 \\ or a const

a = a + (!input#1); \\ a must be an int or a common int
(a == b^c); \\ a, b and c can be any
 \\ combination of ints, common ints
 \\ or consts.

a = --!b; \\ Legal if a is an int or a common int and b
 \\ is either an int, a common int
 \\ or const. Any number of unary operators
 \\ may precede a primary, and the result is
 \\ evaluated right to left.

vector[a*b+10] = c;

/* Legal if vector has been declared as an int array, and a, b and c have been declared as variables or consts. */

The following are all illegal expressions:

const_val = read(d); \\ Illegal when const_val has been
 \\ declared as a const

a += b; \\ Illegal: Two consecutive binary operators
 \\ is not allowed!

labn:

...

labn = a*b; \\ Illegal - can't assign to labels

4.8.15 Control structures.

PPL has four control structures: **if**, **goto**, **for**, and **do-until**.

4.8.15.1 Conditional branching: *if*

if

syntax:

```
if(expression)
    compound_statement1
[else
    compound_statement2]
    compound_statement3
```

If the expression evaluates as non-zero, then `compound_statement1` is executed followed by `compound_statement3`.

If the expression evaluates as zero, `compound_statement1` is not executed, and execution of either `compound_statement2` followed by `compound_statement3` or `compound_statement3` commences, depending upon the presence of the **else** statement.

Execution time: There are two different execution timings, depending upon whether there is an **else** statement following the **if**.

Taking the case where there is no **else** statement, execution of `compound_statement1` or `compound_statement3` commences 100 ns + expression evaluation time after the **if** statement is encountered.

When there is an **else** statement present, things are a little more complicated:

- If "expression" is true (non-zero), `compound_statement1` commences 100 ns + expression evaluation time after the **if** statement is encountered, followed by `compound_statement3` 100 ns after the completion of `compound_statement1`. This 100 ns comes from having to make an unconditional jump over `compound_statement2`.
- If "expression" is false (zero), `compound_statement2` commences 100 ns + expression evaluation time after the **if** statement is encountered, followed by `compound_statement3` immediately after the completion of `compound_statement2`.

4.8.15.2 Unconditional branching: goto

goto

syntax:

label:

statements

goto label;

or

goto label;

statements

label:

A label is an identifier (not declared as an **int** or **const**) followed by a colon. When a **goto label;** statement is reached, control passes to the statement following the **label:**. Up to ten labels may be used.

Please note: a **label:** can only occur once per label identifier, but it can be “gone to” from any number of **goto** statements. If the same identifier were used as a label in two different places, the **goto label;** statement would not “know” which place to go to. Execution time: The **goto label;** statement takes 1.1 μ s to execute. Thus the labelled statement will begin executing 1.1 μ s after the **goto** statement is reached. The **label:** statement itself has no execution time - it simply causes an address to be calculated at compile time for later use by a **goto** statement.

4.8.16 Looping

for loops:

for()

syntax:

for(expression)

loop_compound_statement

The expression is evaluated, then the **loop_compound_statement** is executed expression +1 times. Note that the value of expression is only evaluated once, and the **loop_compound_statement** is always executed at least once.

Execution time:

The **loop_compound_statement** starts to execute for the first time 100 ns plus the expression evaluation time after the **for(expression)** statement is encountered. There is a 100 ns delay after the completion of the **loop_compound_statement** before either the

statement following or loop_compound_statement itself is executed, depending upon whether the required number of loop iterations have been performed.

The 100 ns delay after the loop_compound statement is due to a conditional branch being performed.

do-until loops

do-until

syntax:

do

compound statement

until(expression);

The compound statement is executed, then the expression is evaluated. If the expression is zero (false) the compound statement is executed again and the expression re-evaluated. This process continues until the expression evaluates as non-zero (true).

Execution time:

The time taken at the end of execution of the compound statement following the do is the time taken to evaluate the expression following the until + 100 ns. The do statement itself produces no code, and hence has no execution time - it simply serves as a marker to tell the until branch where to go if the expression evaluates as zero.

4.8.17 Statements

A compound statement is either a single statement, or any number of statements inside braces { }.

A statement can be an assignment statement:

variable = expression;

a **printf** statement:

printf(" control string" [,argument list]);

an output statement;

output(expression);

or one of the control structures (**if**, **for**, **do-until**, **goto**) just described, a **pulse(...)** function, an **input(...)** function, a **void** library function, an **idelay(...)** function or a **delay(...)** function. **delay(...)** functions are described below.

4.8.18 Program syntax

A PPL program consists of optional **#use** statements (one per line) followed by the name “**main**”, followed by open and close parentheses, an opening brace, optional function and variable declaration statements, the program statements then a closing brace.

4.9 Execution times summary

4.9.1 Delay timings.

The **delay** functions will not take expressions involving variables as arguments as the run time involved in evaluating such an expression would interfere with the timing of the **delay**. Constant expressions, however, can be evaluated at compile time and hence will not interfere with the timing of the **delay**.

4.9.2 Expression timings.

To calculate the time involved in evaluating an expression, apply the following rules:

Variable access:	long :	700 ns
	int :	400 ns
	common int :	1200 ns
Array access:	int array element :	index access time + 500 ns
	long array element :	index access time + 900 ns

Constant access times:

int constant expression:	200 ns
long constant expression:	400 ns

Automatic promotion of int to long: 300 ns

Automatic truncation of long to int : 100 ns

Assignment operator execution times

The assignment operator = adds 100 ns each time it appears within an expression, but does not require any execution time as part of an assignment statement (the execution time of the = is absorbed into the access time of the variable which is being assigned to when used in an assignment statement).

Arithmetic operator execution times

The operators + and - each add 100 ns.

The * operator adds 400 ns.

The / operator adds 11.1 µs.

Boolean operator execution times

The **&**, **^**, **|** and **~** operators each add 100 ns.
 The **!** operator adds 400 ns.

Comparison operator execution times

The **>**, **<** and **!=** operators each add 100 ns.
 The **>=** and **<=** operators each add 200 ns.
 The **==** operator adds 400 ns.

For example, the expression, `a = 100*b;` \\ `a` and `b` are **int** variables, will take 1.4 μ s to execute:

400 ns for '`a`', 200 ns for '`100`', 400 ns for '`*`' and 400 ns for '`b`'.

`a = 100*const_n;` \\ `const_n` is a constant

will take 600 ns to execute: 400 ns for '`a`', plus 200 ns for constant expression '`100*const_n`'.

Operator precedence and execution times:

Operator	Description	Associativity	long operand execution time (100ns units)	int operand execution time (100ns units)
!	Logical NOT	Right to left	7	4
~	Bitwise NOT		2	1
-	Unary minus		4	1
*	Multiply	Left to right	90	3
/	(Symmetric) Divide		1232	111
%	Remainder		1208	53
+	Add	Left to right	8	1
-	Subtract		8	1
>	Greater than	Left to right	12	1
<	Less than		12	1
>=	Greater than or equal to		13	2

<=	Less than or equal to		13	2
==	Equal	Left to right	9	4
!=	Not equal		9	4
&	Bitwise AND	Left to right	11	1
^	Bitwise exclusive OR	Left to right	11	1
	Bitwise OR	Left to right	11	1
&&	Logical AND	Left to right	24	13
	Logical OR	Left to right	24	13
=	Assignment in expression. It takes 0 time as it is absorbed into the left hand side variable access time.	Right to left	3	1

4.9.3 Examples of expression timing calculations:

The following PPL fragments show how to use the above timing information to calculate the execution time of various PPL statements. First, let's look at a simple expression involving only long values:

```
main()
{
    long IVal1, IVal2, IVal3;
    /* Various PPL statements ... */
    IVal3 = IVal2 + IVal1; ...
}
```

The execution time of the addition expression is the sum of the following values: IVal3 access time, IVal2 access time, IVal1 access time and binary operator + (with long operands) execution time. The assignment operator = has no explicit contribution to make to the execution time as it is absorbed into the access time of IVal3. Thus, the execution time (in 100 ns units) is given by

7 (IVal2 access time) + 7 (IVal1 access time) + 8 (execution time of + operator with long operands)

+ 7 (IVal3 access time) = 2.9 μ s.

4.9.4 Mixed type expression timing

Now, let's complicate matters by examining an expression involving a mixture of long and int values:

```
main()
```

```
{  
    long IVal1, IVal2;  
    int nIntVal;  
    /* Various PPL statements ... */  
    IVal2 = nIntVal + IVal1;  
    ...  
}
```

Now the execution time of the addition expression has a further implicit element: the 16 bit nIntVal is converted to a 32 bit value before it is added to IVal1. This conversion takes 300 ns. Thus the execution time is given by

4 (nIntVal access time) + 3 (int to long promotion time) + 7 (IVal2 access time) + 8 (execution time of + operator with long operands) + 7 (IVal3 access time) = 2.9 μ s.

Please note: although the expression evaluation times for the two examples above are the same, you cannot simply assume that whenever you have an expression involving a mixture of ints and longs that you just treat the access times of all the ints as longs.

The following example shows why:

```
main()  
{  
    long IVal1, IVal2;  
    int nIntVal1, nIntVal2;  
    /* Various PPL statements ... */  
    IVal2 = IVal1 + nIntVal1 * nIntVal2;  
    ...  
}
```

Here, the precedence of operators has an effect both on the value of the result and the execution time of the above expression. As * has a higher precedence than +, it will be executed first, and in the above example both its operands are ints. Therefore the int operand version of * will be used, and its int result converted to a long prior to adding to IVal1. The execution time of the above expression therefore breaks down thus (in 100 ns units):

4 (nIntVal1 access time) + 4 (nIntVal2 access time) + 3 (execution time of * operator with int operands) + 3 (int to long promotion time) + 7 (IVal1 access time) + 8 (execution time of + operator with long operands) + 7 (IVal2 access time) = 3.6 μ s.

Please note: the execution time of the * operator with long operands (9 μ s) is very different to that with int operands (0.3 μ s). Since in the above expression the operator precedence determined which version of the operator to use it is clear that using brackets to force a different order of evaluation can change the expression evaluation time. (This was not true in earlier versions of PPL).

for example:

```
main()  
{
```

```
long IVal1, IVal2;
int nIntVal1, nIntVal2;
/* Various PPL statements ... */
IVal2 = (IVal1 + nIntVal1) * nIntVal2;
...
```

The above is very similar to the previous example, but the addition of brackets forces the addition to be performed before the multiplication. The `IVal1 + nIntVal1` subexpression will have a long result, which in turn will force the promotion of `nIntVal2` to long and the long operand version of the `*` operator to be employed. The expression evaluation time therefore breaks down as follows (in 100 ns units):

7 (IVal1 access time) + 4 (nIntVal1 access time) + 3 (int to long promotion time) + 8 (execution time of `+` operator with long operands) + 4 (nIntVal2 access time) + 3 (int to long promotion time) + 90(execution time of `*` operator with long operands) + 7 (IVal2 access time) = 12.6 μ s.

The automatic truncation of longs to ints takes place under the following circumstances:

- When the long result of an expression is assigned to an int variable.
- When a long expression is passed as an argument to a function which expects an int value (see the Function calls section below).
- When a long expression is used where a Boolean value is required, i.e. in until and if expressions.
- When a long expression is used to in a For expression.

The truncation takes 100 ns.

Let's modify one of our previous examples which assigned the result of a mixed expression to a long so that it assigns to an int instead:

```
main()
{
    long IVal1;
    int nIntVal1, nIntVal2, nIntResult;
    /* Various PPL statements ... */
    nIntResult = (IVal1 + nIntVal1) * nIntVal2;
    ...
}
```

The timing calculation of the result is almost identical to that used when the result was a long - the only difference is now that the access time of the long result variable is replaced with the long to int truncation time plus the access time of the int result variable:

7 (IVal1 access time) + 4 (nIntVal1 access time) + 3 (int to long promotion time) + 8 (execution time of `+` operator with long operands) + 4 (nIntVal2 access time) + 3 (int to long promotion time) + 90(execution time of `*` operator with long operands) + 1 (long to int truncation time) + 4 (nIntResult access time) = 12.4 μ s.

4.9.5 Scale function timing

The **scale** function execution time is the sum of its argument expression times and 11.1 microseconds.

4.9.6 Output timings.

Recall the syntax of an output statement is **output(expression);** The time taken to perform the operation is given by time taken to evaluate

expression (see *Expression timings*) + 200 ns to output the value. Thus,

output(line_90_on); \\ line_90_on is a constant

will take 300 ns to execute: 100 ns to access constant + 200 ns to send data to output lines.

output(a); \\ a is a variable

will take 600 ns: 400 ns to access variable + 200 ns to send data to output lines.

4.9.7 Input timings.

input#n takes 300 ns.

Thus the partial expression

a + input#0; \\ a is an **int** variable

takes 800 ns to execute (400 ns to access the variable a, 300 ns to perform the **input#0** plus 100 ns to perform '+'.)

input(expression) takes 300 ns plus the time taken to evaluate the expression. Thus,

a = input(lines_1_or_3); \\ lines_1_or_3 is a const, a is an **int**

takes: 200 ns to access constant lines_1_or_3 + 300 ns to perform the input operation + 400 ns to access the variable a = 800 ns total.

4.9.8 Pulse function timings

There is a delay between the control flow of the program reaching the pulse function and the start of the pulse appearing at the output port. If the "lines" argument is a constant expression, the start of the pulse will appear at the output port 600 ns after the pulse function is encountered. If the "lines" argument is an **int** variable, the start of the pulse will appear 700 ns after the pulse function is encountered. If the "lines" argument is a **common int** variable, the start of the pulse will appear 1.6 µs after the pulse function is

encountered. If the “lines” argument is an **int** array element, the start of the pulse will appear 400 ns + array element access time after the pulse function is encountered.

There is also a 100 ns delay before the end of the pulse appears at the output port after the time specified by the “units”, “order” and “thousandths” has elapsed. Thus all pulse widths are 100 ns longer than the time specified by the arguments to the pulse function. One can employ this fact to achieve the shortest possible pulse by using 0 for both the “units” and “thousandths” arguments thus:

pulse(lines, 0, ms, 0);

will produce a 100 ns pulse on the specified lines.

Please note: when using variables as the “units” or “thousandths” arguments the values of the variables must be greater than or equal to the minimum value of 2.

Thus the minimum pulse width possible with a variable “thousandths” argument and constant “units” argument is 2.1 microseconds (the 0.1 microseconds coming from the delay before the pulse output ceases), and can be achieved as in the example below:

```
main(){
    const line_3 0x08;
    const all_off 0;
    int thou;
    thou = 2;
    output( all_off ); \\ All output lines low
    pulse( line_3, 0, ms, thou );
    /* Line 3 goes high for 2.1 microseconds */
}
```

4.9.9 For loop timings.

If one wished to use a loop to generate a repetitive pulse sequence whose timing was critical, the time taken to execute the jumps in control flow would have to be taken into consideration. The times involved are as follows:

```
for( expression )
{
    statements
    |
    |
}
```

The expression following the **for** is only evaluated once, on entry to the **for** loop. The time taken to evaluate the expression will depend on the complexity of the expression. (See

Expression timings). The jump back to the start of the statement block during the execution of the loop takes 100 ns.

4.9.10 do - until loop timing

The time taken at the end of execution of the compound statement following the **do** is the time taken to evaluate the expression following the **until** + 100 ns.

Thus in the code fragment

```
do{
    delay( n, us );
    output( val );
} until(input#0);
```

it will take 400 ns (300 ns to evaluate **input#0**, + 100 ns to jump back or continue) before either the next statement following the **until(expression);** is executed or the **delay(n,us);** is re-executed.

4.9.11 If - else timings

```
if( expression )
{
    if_statement1;
    |
    |
    if_statementn;
}
[else
{
    else_statement1;
    |
    |
    else_statementn;
}]
next_statement;
```

The time taken before the first if_statement is executed is the time taken to evaluate the expression + 100 ns if the expression evaluates as non-zero.

If there is an **else** statement, and the expression evaluates as zero, the time taken before the first `else_statement` is executed is the same; i.e. the expression evaluation time + 100 ns.

If the expression evaluates as non-zero ('true'), and an **else** statement is present the compound statement following the **if** will be executed after the delay involved in evaluating the expression+100ns, and there will be a further delay of 100ns after the last statement of the **if** compound statement is executed before the first statement following the **else** compound statement is executed.

This further 100 ns delay is the time it takes to jump 'over' the **else** compound statement. There is no such delay between the execution of the last statement of an **else** compound statement and the execution of the statement following.

When there is no **else** statement following the **if** statement, there is also no delay between the execution of the last of the statements in the **if** compound statement and the first statement following. This somewhat complicated timing can be summarized thus:

```
if( expression)
{
    first_if_statement; \\ reached at expr_time+100ns if
    |                  \\ expr is 'true'
    |
    last_if_statement; \\ delay of 100ns following this
                      \\ statement before next_statement
}
else
{
    first_else_statement;    \\ reached at expr_time
    |                      \\ +100ns if expr is 'false'
    |
    last_else_statement;
}
next_statement;
```

OR

```
if( expression)
{
    first_if_statement; \\ reached at expr_time +100ns if
    |                  \\ expr 'true'
    |
```

```
    last_if_statement;  
}  
next_statement;           \\ reached at expr_time+100ns if expr 'false'  
                          \\ or immediately after last_if_statement if  
                          \\ expr 'true'.
```

4.9.12 Goto timing

The **goto** label; statement takes 1.1 μ s to execute. Thus the labelled statement will begin executing 1.1 μ s after the **goto** statement is reached.

4.9.13 Printf and Read timing

Both these operations are relatively slow as they involve handshaking communications between the spectrometer and host PC.

It is not possible to specify an exact execution time for either of these functions, and they should therefore not be used where timing is critical.

4.9.14 Function calls

All library function calls must be declared before they are used, so the PPL compiler "knows" the types of the return value and arguments. This information is used when a function call is made to automatically insert type converting code if necessary. Thus if you pass an int expression as an argument to a function which expects a long the compiler will generate code to convert the int result to a long, adding 300 ns to the int expression evaluation time. Conversely, if you pass a long expression to a function which expects an int argument, 100 ns will be added to the long expression evaluation time as the compiler generates code to convert the long value to an int.

The same type conversion rules apply to the values returned by function calls as described in the mixed type expression timing examples given above: if a function call returns an int and a long is needed, the compiler will generate the int to long promotion code which in turn adds an extra 300 ns to the expression evaluation time. Likewise, if a function returns a long which is assigned to an int variable, the compiler will generate code to truncate the long value to an int prior to storing it in the int variable, adding 100 ns to the expression evaluation time. For example, let's look at the (new) UnsignedToLong library function which expects a single int argument and returns a long result. If we pass it a long and assign the result to an int, both the argument and result types will need to be converted. The example is obviously a rather useless piece of code - it has been chosen to illustrate automatic truncation during assignment and argument passing.

```
main(){  
    long lVal, UnsignedToLong(int);  
    int nIntVal;  
    /* Various PPL statements ... */  
    nIntVal = UnsignedToLong(lVal);    ...  
}
```

Breaking down the timing of the above PPL fragment, we have (in 100 ns units):

7 (lVal access time) + 1 (long to int truncation time: UnsignedToLong has been passed a long, and it needs an int) + 2 (execution time of UnsignedToLong function) + 1 (long to int truncation time: UnsignedToLong returns a long, but we're assigning to an int) + 4 (nIntVal access time) = 1.5 μ s.

Chapter 5. PULSE SEQUENCE WRITING GUIDE

5.1 Introduction

The main emphasis of this document is to explain pulse programming through illustrated examples. The reader may prefer to start with these simple example programs, or indeed go straight to the example imaging program rather than reading through this document in logical order. All the examples use several simple include files, except the last one which illustrates the use of macros for ease of programming. A description of all macros used in the current release can be found in the text file `include.txt` resident in the directory `c:\smis\include`.

5.2 Software Overview

This section is an overview of some of the common points of all pulse sequences currently used. All pulse sequences, RF or gradient control, make use of a C-Pre-Processor (CPP). The CPP reads a PPL program, expands macros and "include" files and writes the result to an intermediate file that can then be compiled by the standard p2f compiler. This is taken care of with the batch file in `c:\smis\compile.bat`. So in order to compile a pulse sequence, for example `1PULS.PPL`, type the following in a DOS environment:

PPL compiler Version 6.02

Current Directory : `c:\smis\seqlib`

Source filename ? `c:\test\1puls.ppl` **enter!**

No extension is given to the program name. The result of this action is to create an intermediate file **1puls.a.i**, which contains the macro and include expansions, and the forth file, e.g., **1puls.a.fth**. Using the p2f icon application will execute this batch file automatically and only the file name without the extension is required.

The old RF controller MR3031 and gradient controller MR3040 have been super-seeded by a single board called MR3080. To minimize the changes needed to software all the functions with reference to RF/gradient controller use the same MR3031/3040 keywords.

5.3 Structure of MRI data

Dimensions or levels of nesting, which SCAN uses to store data are listed below:

DIMENSION

SAMPLES**VIEWS****SECONDARY VIEWS****SLICES****ECHOES****EXPERMENTS**

PROCESSING AVAILABLE

FFT**FFT****FFT**

The size and dimensions of a given experiment are defined by their declaration in the PARAMLIST HEADER section of a pulse program. The NO_SAMPLES (number of samples) must be defined and any combination of other parameters may be used in conjunction with this. The names given to the different levels reflect their usage in MR imaging. The VIEWS is the term for the phase encode dimension. The order of storage of the **6 dimensions** is related to the way in which the data is processed after acquisition. The first three levels should be used when processing will involve a Fourier transform in RECON or SPECANA. The other three levels act as "bins" for data as some other parameter is varied. This could be spatial (e.g. SLICES would be used for multislice imaging experiments) or in sequences such as CPMG and inversion recovery the ECHOES and EXPERIMENTS dimensions may be used to acquire images at different echo times or inversion times, respectively. Note that for other applications the last three dimensions can be used for whatever the user chooses, i.e., one may use the SLICES dimension for considering other parameters, such as temperature changes.

The multi-dimensional data array, which is defined by the pulse program, is written into by SCAN as data is acquired. SCAN fills up this data set in an order defined below, which we consider to be sensible for an imaging application. Pulse sequences must therefore be written so that the order of the experiment is consistent with this defined order of data storage.

ORDERAVERAGING AVAILABLE

NO_SAMPLES

NO_ECHOES

VIEWS_PER_SEGMENT

SLICE_BLOCK

NO_SLICES

VIEW_BLOCK

NO_VIEWS

NO_SECONDARY VIEWS

BATCH_EXPERIMENTS

} Time-critical part
of the sequence

BATCH_SLICES
NO_EXPERIMENTS
NO_AVERAGES

Averaging can take place at three different levels (SLICE_BLOCK, VIEW_BLOCK, NO_AVERAGES) as shown on the chart above. Any or none of these may be selected and again they are defined in the PARAMLIST HEADER (see Chapter 2)

- ✓ SLICE_BLOCK - refers to averaging at the slice level. This hardly uses in imaging, but averaging at this level provides a facility for other applications. If SLICE_BLOCK is not defined, then by default SCAN sets its value to one.
- ✓ VIEW_BLOCK - results in averaging taking place at the views level (see example below). Note that VIEW_BLOCK must be a multiple of SLICE_BLOCK.
- ✓ NO_AVERAGES - takes place outside all the dimensions of data acquisition. It is essentially the total number of averages taken for each experiment and as such must be a multiple of VIEW_BLOCK * SLICE_BLOCK.

Please note: If the VIEW_BLOCK keyword is not used then the NO_AVERAGES averaging takes place at the VIEW-BLOCK level by default. This feature is built in because in imaging applications one nearly always wants to average at each phase encode step.

As an example, consider what would happen if the NO_AVERAGES keyword was set to 20, VIEW_BLOCK set to 10 and SLICE_BLOCK set to 5. In this case, the data would be averaged 5 times at the NO_SLICES level (i.e., 5 acquisitions on each slice in a multislicing imaging sequence). This would then be repeated at the NO_VIEWS level (i.e. 2 acquisitions at each phase encode step as k-space is traversed), giving 10 averages in total per view. Once the sequence has completed all its views the entire sequence would be repeated to give a total number of averages for each experiment of 20.

5.4 Introduction to pulse program layout

The MR Solutions pulse programming language is properly introduced in Chapter 5. In this section, the program syntax is briefly described and how pulse programs are built is explored.

5.4.1 Program syntax

In general, a PPL program consists of #use statements (one per line), the PARAMLIST header (/*PRARMLIST ~ END*/), #include, #define statements (one per line), and "main()" with open and close curly brackets, { ~ }. In the main() function, optional function and variable declaration statements, the program statements can be included. The below code is an example.

```
#use name '*.seq' alias

/*
PARAMLIST
.
.
.
END
*/

#define

#include '*.pph'
main()
{
/* Declarations
int a, b, c, array[5];
common int d, e, f;
long g, h;      Long variables
const trig_line-on OXIO;
const trig_tine-off Oxo;

/* Start of program statements */
NEWSHAPE_MAC(1, pf1,"3lobe_sinc_3kHz" , 1332, 3000)
NEWSHAPE_MAC(2, pf1,"3lobe_sinc_1500Hz" , 2664, 1500)

averages_loop:
current_view = 0;
gp_loc = 0;
no_dummy_cycles = 1;

MR3040_Start(CHANNEL_S);
MR3031_RFSTART(rfnum_inv,tset_inv,p_inv_mul,befinv+tramp,rf_on*inv_on)
delay(affinv, us);
MR3040_CONTINUE(CHANNEL_S);
delay(tramp,us);

acquire( sample_period, no_samples );

        if (image_av < no_averages)
            goto averages_loop;
    end:
}
```

The block diagram of Fig 1 shows the functional sections of an MRI pulse program. The reader will be shown in the next section how these fit into the syntax described above.

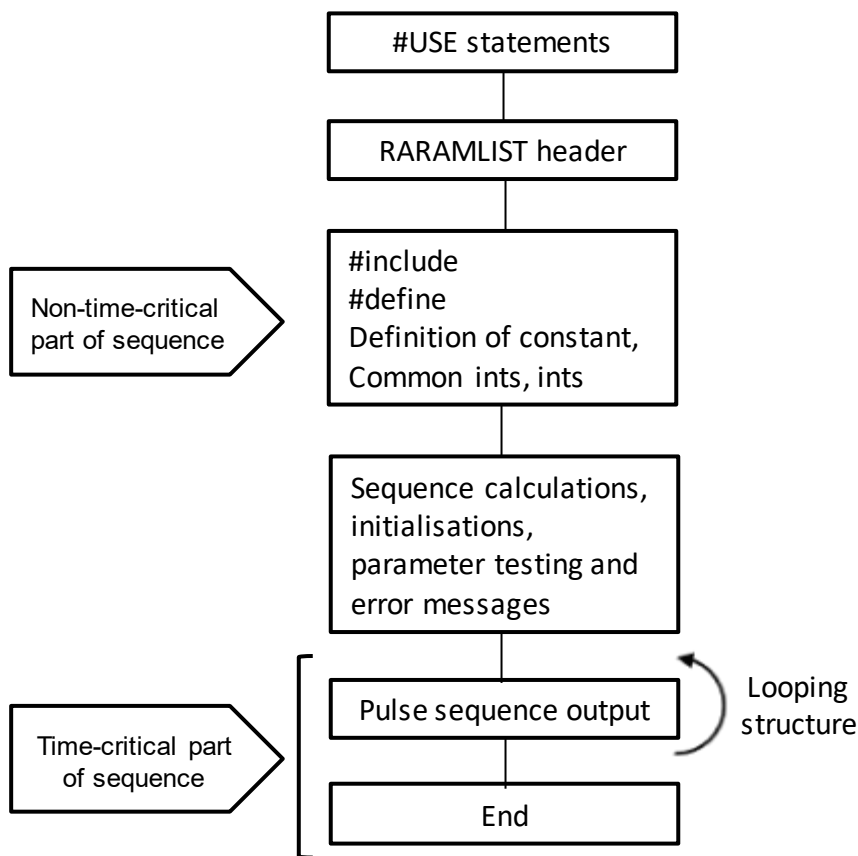


Fig 1: Typical pulse sequence program layout

5.4.2 **#USE and # INCLUDE statements**

The #USE statements at the top of any pulse program links the sequence with a waveform editor file (WavEd: see PowerScan User Guide,). The keyword at the front specifies whether the waveform data file is intended for use with RF channel modulation control (e.g., RF1 or RF2) or gradient control (e.g., GRAD). If no keyword is given then the data file is assumed to be a gradient data file. The WavEd file name is written next within quotation marks and the alias at the end is used to call up that file within the main part of the pulse sequences, i.e., to send out a gradient shape to the waveform boards.

The #INCLUDE statements (section 3.1) are used to declare all library functions described in Chapter 0.

5.4.3 **PARAMLIST HEADER**

The items which appear in the **PARAMETER EDITOR** are declared by the PARAMLIST HEADER in the beginning of the PPL program. As far as the compiler is concerned the PARAMLIST is just a comment, which can be ignored (non-time-critical). The available keywords have already been described in Chapter 2.

5.4.4 **CONSTS, INTS and COMMON INTS**

All constants, ints and common ints in PPL are signed 16 bit values and must be declared at the start of any program. A constant expression has an operand, which is known at compile time and can be either hexadecimal, octal, or decimal (see Chapter 5). Integers can either be a single expression or an array, although care has to be taken when using arrays, as there is no run-time checking of array indices to maximize speed. Common ints are variables which can be adjusted interactively during SETUP in SCAN. These appear black in the PARAMETER EDITOR screen produced by running SETUP. Variables such as for pulse length and repetition time can be adjusted interactively if declared as common ints but keywords such as PHASE_CYCLE and NO_SAMPLES cannot have variables which can be changed interactively, even if declared as common ints. This is because they are involved in the actual data acquisition part of the pulse program (see 2.3).

5.4.5 **Sequence setup, error messages, etc.**

Sequence setup takes place outside the main looping structures of the pulse program and therefore is not time critical. It consists of setting up frequencies, phases, rephase gradient strengths, calculating delay times etc. This part of the program also checks that some of the non-interactive variables are allowed, i.e., that NO_AVERAGES is a multiple of VIEW_BLOCK. In imaging sequences, the ORIENTATION keyword is used to allocate gradients for frequency encoding, phase encoding and slice selection. Also the sequence is synchronized to 100 kHz using the SYNC() command. **It is worth noting that common int calculations should not be done in this section of the program if interactive adjustment is required, as they will not be updated during SETUP until all the averages have been completed.**

Error messages are optional and can be placed at any point in the pulse program, though it is advised not to put them in the sections that are very time critical. They warn the user by displaying a window in SCAN if incorrect values have been used, either in the PARAMETER EDITOR or interactively by changing one of the common int variables. The following code shows the use of the error message. Assume that the number of averages variable was set to 3 and the user had also chosen to do DC phase cycling, an error message would be printed as the user needs to change one of the parameters.

```
if (phase_cycle == 2){  
    if(no_averages & 1 != 0){  
        print("\nNo. Averages is not a multiple of two.");  
        goto end;  
    }  
}
```

5.4.6 Pulse sequence output

This particular part of the pulse program is unique for each sequence, but there are certain similarities, which shall be discussed in this section. Figure 4.1 shows the looping structure of a typical pulse sequence with many of the dimensions included. Most sequences would not be as complicated as this with maybe only one or two dimensions, but the diagram indicates clearly how the looping is arranged and where averaging takes place. MR Solutions recommends the use of goto statements because there is a limit on the amount of code that can be included within a do-loop.

To the left of the diagram are indications as to which parts of the sequence are time-dependent. The first part has been referred to as semi-time critical due to it being inside the repetition time delay. As such, any commands or calculations must be accounted for to obtain an accurate delay time (see chapter 3 for library function timings). In this section of the program frequency buffers are setup, interactive delay times calculated and possible lookup tables used, for example. The `hostrequest()` command, which allows interactive instrument control via the IIC, is also used here along with the `resync()` function which synchronizes the sequence to the ADC. The last section of the pulse program is where RF pulses and any gradient shapes are sent out from the waveform boards to the RF unit and appropriate gradient drivers respectively. Small delay times are used to balance out the sequence and therefore any parts of code, which are not necessary, should be placed somewhere else in the program. The importance of accurate timing depends entirely on the nature of the experiment. Conventional NMR imaging is essentially unaffected by function execution times. Other applications in solids or EPI, for example, will require more care. Finally, data is captured using the `acquire` command and the looping counters are incremented for the various dimensions and averaging keywords.

5.5 Gradient Controller

5.5.1 Naming Convention

The three channels are labelled as:

- ✓ CHANNEL_R: readout channel, if no rotation this is the physical x channel output.
- ✓ CHANNEL_P: phase encoding channel, if no rotation this is the y channel output.
- ✓ CHANNEL_S: slice selection channel if no rotation this is the z channel output.

The reason for this naming convention rather than CHANNEL_X, CHANNEL_Y or CHANNEL_Z is if rotations are used then the output of each channel will not be along a physical x, y or z axis but the required gradients will be applied in rotated frame of

reference. CHANNEL_R, CHANNEL_P and CHANNEL_S are therefore labels used within the pulse programs to define the logical gradients. The physical gradient outputs G_x , G_y and G_z will be determined by the rotation equation specified in section 'Oblique Angle transformations' (5.5.5).

5.5.2 ParSetup Keyword Definitions

The orientation of the image co-ordinate system is defined with respect to the physical axes of the field gradient system via three angles. The orientation is defined via the OBLIQUE_ORIENTATION keyword, mutually exclusive with the ORIENTATION keyword.

5.5.3 OBLIQUE_ORIENTATION

The syntax for the OBLIQUE_ORIENTATION keyword is:

OBLIQUE_ORIENTATION min, max, default, phase_var, x_angle, y_angle, z_angle;

This keyword produces a list box containing a number of predefined image orientations, referenced from an index into the list (see below). The initial orientation displayed will correspond to the index chosen for default. The range of values available for the index will depend upon the values of min and max. In addition to the orientation listbox, three edit boxes are created for entry of the x, y and z rotation angles. If a predefined orientation is selected within the listbox, these edit boxes are disabled and display the appropriate angles. The boxes are enabled only if the oblique orientation is selected. The angles are passed to the PPL variables x_angle, y_angle and z_angle, in 1/10 degree units.

The user entry limits for the x, y and z angles (in 1/10 degrees) are as follows:

- 1799 <= x angle <= 1800
- 900 <= y angle <= 900
- 1799 <= z angle <= 1800

The contents of the orientation listbox are defined in c:\windows\parfilio.ini). This list may be modified if additional predefined rotations are required, new items being appended to the list. The default list is:

Listbox text	X angle	Y angle	Z angle
Oblique	0.0	0.0	0.0
Transverse	0.0	0.0	0.0
Coronal	- 90.0	0.0	0.0

Sagittal	0.0	90.0	- 90.0
----------	-----	------	--------

The phase encode directions are added to the list before the listbox is displayed. The phase direction is passed to the PPL variable `phase_var`, with a value of 0 for vertical and 1 for horizontal phase encode. For example, the entry `OBLIQUE_ORIENTATION 0, 2, 1, phase, x_angle, y_angle, z_angle`; in the PPL PARAMLIST displays the transverse orientation by default (phase encode vertical), and restricts the user to the oblique, transverse and coronal orientations.

5.5.4 GRADIENT_STRENGTH

The syntax for the GRADIENT_STRENGTH keyword is:

GRADIENT_STRENGTH `grad_var`; (See also 2.2.4.1)

This keyword enables the gradient strengths used at the time of the experiment to be saved to the output file. The variable '`grad_var`' should be defined in PPL as a four-element integer array, which will be initialised with the minimum value, followed by the defined values, of the physical gradient strengths defined in `c:\windows\parfilio.ini`. The gradient strengths are assumed to be isotropic, and this minimum value is used in calculating the limits of FOV, slice thickness and FOV offsets. On reading the PPR file, the values in the initialisation file override the PPR value for gradient strength.

5.5.5 Oblique Angle Transformations

The equation that governs the output of the physical gradients (G_x , G_y and G_z in DAC units) is as follows:

$$\begin{pmatrix} G_x \\ G_y \\ G_z \end{pmatrix} = \begin{pmatrix} A & 0 & 0 \\ 0 & B & 0 \\ 0 & 0 & C \end{pmatrix} \mathbf{R} \cdot \begin{pmatrix} \frac{S_r}{D_r} & 0 & 0 \\ 0 & \frac{S_p}{D_p} & 0 \\ 0 & 0 & \frac{S_s}{D_s} \end{pmatrix} \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix} = \mathbf{G} \mathbf{P} \mathbf{R} \mathbf{T} \mathbf{S} \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix}$$

Where:

$$\mathbf{G} = \begin{pmatrix} A & 0 & 0 \\ 0 & B & 0 \\ 0 & 0 & C \end{pmatrix}$$

A = - $G_{min}M_{xm}/G_{xmax}M_w$

B = - $G_{min}M_{ym}/G_{ymax}M_w$

C = - $G_{min}M_{zm}/G_{zmax}M_w$

G P is created by the **MR3040_CreateBasematrix** function (see 3.5.15). It creates a base matrix to which all subsequent rotation matrices are referred.

R T S is created by the **MR3040_Creatematrix** function (see 3.5.16 for more information).

Gmin is the minimum gradient strength of the three physical gradients which is returned by PARSETUP to the variable grad_var[0] via the gradient strength keyword.

Gzmax, Gymax, Gxmax are the gradient strengths in Hz/mm units read from c:\windows\parfilio.ini and returned to the elements grad_var[1], grad_var[2] and grad_var[3] respectively by PARSETUP via the gradient strength keyword.

Mxm, Mym, Mzm are the (minimum) DAC values for maximum output on all three channels and Mw is the lowest limiting value for the input waveforms. These values are defined in parfilio.ini and are all -32768 for 3040 gradient control.

P is a patient orientation matrix and **R** is an image rotation matrix. Both are applied to the read, phase, and slice axes of the rotated image plane to produce an x, y, z vector. They both have the following form:

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \mathbf{R} \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix} = \mathbf{R}_x \mathbf{R}_y \mathbf{R}_z \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix} \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix} \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix}$$

$$= \begin{pmatrix} 1cc & -1cs & 1s1 \\ ssc + c1s & c1c - sss & -sc1 \\ s1s - csc & s1c + css & cc1 \end{pmatrix} \begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix}$$

Please note: In the matrices above, “**c**” is used to represent cos(angle) and “**s**” is used to represent sin(angle). In the combined matrix, the elements are represented as ordered triads, e.g., c1s = cos(x) · 1 · sin(z) and csc = cos(x) · sin(y) · cos(z), x is the rotation about the physical x axis, y the rotation about the physical y axis and z the rotation about the physical z axis.

T is one of the following phase encode matrices:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \text{ for vertical phase encoding, or } \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \text{ for horizontal phase encoding.}$$

S_r, **S_p** and **S_s** are multipliers for the logical gradients and **D_r**, **D_p** and **D_s** are the corresponding divisors.

$\begin{pmatrix} \text{READ} \\ \text{PHASE} \\ \text{SLICE} \end{pmatrix}$ is the values retrieved from the wavED files assigned to the read, slice and phase directions of the rotated image.

5.6 Appendix A: #include File

This chapter lists standard functions and variables which are defined using #include keyword. All the extra variables and functions which are not in the files listed have to be declared and defined separately in the program. All the PPH files are resident in the directory c:\smis\include, as is the text of this appendix in the file include.txt. The PPH files used are:

STDFN_15.PPH	Definition of standard PPL functions and 16 ~ 32 bit conversion and math utilities.
STD3040.PPH	Definitions for the 3040 demo programs - for training purposes only.
CENTRE_OUT.PPH	Lookup table for centre-out phase encoding (Maximum views = 256).
SNAPSHOT.PPH	Lookup table for centre-out phase encoding for snapshot_flash sequence (Maximum views = 128).
M3040_15.PPH	All definitions and macros for gradient controller programming.
M3031_15.PPH	All definitions and macros for RF controller calls.
OFFST_15.PPH	Contains macros to calculate the offsets in frequency and phase for all three directions.
VAR_15.PPH	Definitions of all variables pertaining to gradient, RF timing and frequency control.
TSTEX_15.PPH	This verifies the combination of averages, view block and xlice. An error condition will terminate the program.
RAND_15.PPH	A set of randomly generated numbers for gradient pulse amplitudes.
TIME_15N.PPH	Timing variables for the FSE sequence.

5.6.1 M3040_15.PPH

This macro file contains definitions for the 3040 gradient control functions and the following macros:

5.6.1.1 DELAY(*tdel, clock, alias*)

This macro writes out a delay *tdel* to the sequencer memory with amplitude defined by the first point of the frame *alias*. Clock is the setting of the master clock. The number of clock units executed is defined as $tdel/clock$ so *tdel* must be a multiple number of the master clock setting.

5.6.1.2 PULSE(*top, clock, alias_up, alias_down*)

This macro writes a gradient pulse to the sequencer memory with the initial ramp up defined by the frame *alias_up* and the ramp down defined by the frame *alias_down*. The plateau is maintained for time *top*. Again *top* must be a multiple number of the clock setting.

5.6.1.3 POSPULSE(*top, clock*)

This macro defines a +ve trapezoid gradient waveform with a plateau time of *top* (which must be a multiple number of the master clock setting). The ramp up is defined by the alias "0_max" and the ramp down by the alias "max_0". The actual ramp time will be defined by $tramp = clock * 50$ (there are 50 points in each frame).

5.6.1.4 NEGPULSE(*top, clock*)

This macro defines a -ve trapezoid gradient waveform with a plateau time of *top*. "0_min" and "min_0" are the frames used in this macro.

5.6.1.5 POSPULSE_SEC(*top, clock*)

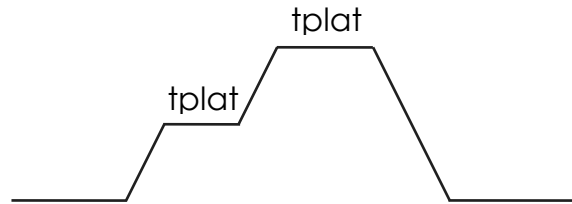
This macro defines a positive trapezoid gradient waveform in the secondary channel with a plateau time of *top*. "0_mx_sec" and "mx_0_sec" are the frames used in this macro.

5.6.1.6 NEGPULSE_SEC(*top, clock*)

This macro defines a negative trapezoid gradient waveform in the secondary channel with a plateau time of top. "0_mn_sec" and "mn_0_sec" are the frames used in this macro.

5.6.1.7 *POSPULSE_2(tplat1, tplat2, clock)*

This macro defines a gradient trapezoid as shown below:



The first plateau is held for tplat1 and the second is held for tplat2. All three ramp-times are the same (tramp). The shape is made up by addition of primary and secondary channels. Both plateau times must be multiple of the master clock.

5.6.1.8 *NEGPULSE_2(tplat1,tplat2, clock)*

This is the same macro as above except both amplitudes are negative.

5.6.1.9 *POSPULSE_HOLD()*

This macro generates a +ve gradient trapezoid. However after the first ramp has been output to the sequencer memory the level is held with the MR3040_HOLD command. This level will be held until the MR3040_continue command is encountered. No arguments are necessary for this macro.

5.6.1.10 *NEGPULSE_HOLD()*

This macro generates a -ve gradient trapezoid and works in the same way as POSPULSE_HOLD().

5.6.1.11 *POSPULSE_SEC_HOLD()*

This macro generates a +ve gradient trapezoid on the secondary channel. However after the first ramp has been output to the sequencer memory the level is held with the MR3040_HOLD command. This level will be held until the MR3040_continue command is encountered. No arguments are necessary for this macro. "0_mx_sec" and "mx_0_sec" are the frames used in this macro.

5.6.1.12 NEGPULSE_SEC_HOLD()

This macro generates a -ve gradient trapezoid on the secondary channel. However after the first ramp has been output to the sequencer memory the level is held with the MR3040_HOLD command. This level will be held until the MR3040_continue command is encountered. No arguments are necessary for this macro. "0_mn_sec" and "mn_0_sec" are the frames used in this macro.

5.6.1.13 PULSE_HOLD(alias_up, alias_down)

This macro uses alias_up as the frame to define the initial ramp the last point of the frame is then held and when the mr3040_continue command is encountered alias_down is written to the sequencer.

5.6.1.14 POSPULSE_2_HOLD()

This macro outputs a gradient shape as in POSPULSE_2 except that the plateaus are defined by the MR3040_hold function. One MR3040_contine call is required for the first plateau and a second one is required for the second plateau.

5.6.1.15 NEGPULSE_2_HOLD()

This is the same as the above macro except the output amplitudes are negative.

5.6.1.16 EPIREAD(top, clock)

This macro outputs the basic cycle of an EPI read gradient i.e. a positive trapezoid and then a negative trapezoid of plateau duration top. The clock setting will determine the ramp time used.

5.6.1.17 EPIPHASE(top, clock)

This macro creates a blip gradient for EPI sequences. A blip is defined as a ramp up followed immediately by a ramp down. The argument top is the delay before the gradient blip. This macro has to be used in conjunction with a MR3040_Loopcount(n-1) function so that the blip will be repeated n times. See EPI_SE.PPL for an example use of this macro.

5.6.1.18 BASEMATRIX(zang, yang, xang)

This macro creates the base matrix. All subsequent rotations are made relative to this base matrix. Only the required rotations for patient position need to be passed over to this macro.

5.6.1.19 *CREATE_MATRIX(mat, slice, phase, read)*

This macro creates a matrix called mat with gradient amplitudes slice, phase and read along the three channels. The rotation values used are those values passed back from ParSetup.

5.6.1.20 *MR3040_DEGLITCH*

This macro outputs zero on all three channels. This should be called at the start of a program to reset the 3040 board and ensure the outputs are zero.

5.6.2 *M3031_15.PPH*

5.6.2.1 *NEWSHAPE_MAC(rfnum, pf1, frame, pulse_length, pul_bwidth)*

This macro assigns the waveform "frame" defined in the sequence file by 'pf1' to the index number 'rfnum'. The pulse duration and bandwidth must also be specified in the arguments 'pulse_length' and 'pul_bwidth' respectively.

5.6.2.2 *NEWSHAPE_SETUP(rfnum, rfscale, pf1)*

This macro calls the append boards ready function etc. for output of the waveform specified by the index number 'rfnum'. This macro is called from the MR3031_RFSTART macro and as such should NOT be used in a PPL program.

5.6.2.3 *MR3031_RFSTART(rfnum, rfpw, rf_scale, pred, mod_level)*

This macro outputs the RF shape with a pulse width rfpw and multiplier rf_scale. The value pred is executed before the start of the pulse. This has a minimum value of 50 μ s. The calculations to set up the pulse are performed during this pre-delay so overhead times are taken into account and the RF pulse will be clocked out after the delay pred. This macro calls NEWSHAPE_SETUP macro for output of the waveform.

5.6.2.4 *MR3031_HARDPULSE(pred, rfpw, rf_scale, mod_level)*

This macro outputs a hard pulse of width rfpw after a delay pred with a multiplier given by rf_scale. The overhead times for warmup and all other calls are taken into account so that the RF pulse will be clocked out after a time pred. Control is returned to PPL 22 μ s after the end of the RF pulse.

5.6.2.5 RFSPoil(rf_inc, phase_90, phase_117, phase_rec)

This is a macro for RFSPoil. This is achieved by incrementing the phase of the receiver and transmitter by 117 on every scan. Phase_90 is the number of phase resolution steps to achieve a 90° pulse and phase_117 is the number of phase resolution steps to achieve a 117 pulse. Phase_rec is the current value of the receiver phase to which will be added to the random spoiler value achieved via this macro.

5.6.2.6 RFSPoil_DC(rf_inc, phase_90, phase_117, phase_rec, dc_shft)

This is a macro for RFSPoil. This is achieved by incrementing the phase of the receiver and transmitter by 117 on every scan. Phase_90 is the number of phase resolution steps to achieve a 90° pulse and phase_117 is the number of phase resolution steps to achieve a 117 pulse. Phase_rec is the current value of the receiver phase to which will be added to the random spoiler value achieved via this macro. The value for dc shift is also accepted as the last argument.

5.6.2.7 RFSPoil_SE(rf_inc, deg_90, deg_117, phase_90, phase_180, phase_rec)

This is a macro for RF spoiling but does not set the transmitter or receiver phase. Deg_90 is the number of phase increments for a 90° pulse and deg_117 for 117° pulse. The RF spoiling increment is then added to each of the parameters Phase_90, phase_180, and phase_rec and these modified values passed back to the main program.

5.6.2.8 MR3031 DEGLITCH

This macro outputs zero to the RF controller board. The call to this macro should be made before the sequence starts. This will then remove an unwanted delay of about 20 μ s which otherwise occurs before the very first RF pulse.

5.6.3 OFFST_15.PPH

5.6.3.1 SELECTSLICE(sliceno)

This macro is used for Multi angle Oblique and keeps a track of the current slice number so that offsets defined in the list box from PARSETUP can be correctly tracked. If MAO is not defined in your sequence this macro has no meaning.

5.6.3.2 *READOFF(read_off, samp_int, sc, read_freq_long, fov_read_freq, kHz, Hz)*

This macro calculates the offset in the read direction in Hz. read_off is the value passed back from ParSetup Samp_int is the sample interval and sc is the scaling value used in the paramlist header. The result is passed back the value fov_read_freq, kHz, Hz and read_freq_long.

5.6.3.3 *PHASEOFF(phase_off, ph_inc, sc, fov_phase_deg)*

This macro calculates the number of phase increments required to give the offset in mm in the phase encode direction. Phase_off is the value passed back from ParSetup and ph_inc is the value in degrees of a unit increment of phase. However this must be passed over multiplied by a factor of 1000. E.g. if the phase increments 0.225° then the value passed over to this macro should be 225. Sc is the scaling factor defined in the paramlist header. The result is passed back to the parameter:fov_phase_deg.

5.6.3.4 *SLICEOFF(slice_off, bwd, sc, slice_freq_long, fov_slice_freq, kHz, Hz)*

This macro calculates the offset in the slice direction based on the bandwidth of the RF pulse (bwd). Slice_off is the value passed back from ParSetup and sc is the scaling value used in ParSetup. The result is passed back to fov_slice_freq, kHz, Hz, slice_freq_long.

5.6.4 *VAR_15.PPH*

This is an include file containing a set of standard definitions for looping control, RF, gradient and timing control. Also the warmup time for the RF amplifier is defined, as is the phase resolution of the synthesiser of the system. (This value is defined as phase_res and should be multiplied by 1000. For example, the phase resolution of the PTS310 is 0.225° degrees then the value assigned to the variable phase_res should be 225. The value for the DSP filter time (defined as tfilter) is calculated in this include file such that tfilter = 380 μ s if the spectral width is < 100 kHz and is 30 μ s for all other spectral widths. The naming convention is such that phase encode gradients are denoted by G_p , slice gradients by G_s and read gradients by G_r . One important point is that the variable temp_mac and

temp_mac_long are reserved exclusively for temporary calculations within macros and SHOULD NOT BE USED IN A PPL PROGRAM.

5.6.4.1 *POS_INDEX(pos_index)*

This macro will define a set of variables for offset control for MAO.

5.6.4.2 *LONGDELAY(longval)*

This macro executes a delay of longval, where longval is a long and may exceed 32767 μ s. The value is divided into a millisecond component and a microsecond component; the overhead time of the macro is taken into account so a delay of 'longval' microseconds is guaranteed. The minimum value of longval is 150 μ s.

5.6.4.3 *US_TO_MS(us,ms)*

This macro converts the argument 'us' to the nearest millisecond by rounding it up or down. The result is passed back to the parameter ms.

5.7 Appendix B: Coding Pilot Capability into PPL Programs

The pilot software will work with either the OBLIQUE_ORIENTATION keyword (for gradient controller) or the ORIENTATION keyword (RF controller). The keywords for offset definition are FOV_READ_OFF, FOV_PHASE_OFF and FOV_SLICE_OFF. The offsets are displayed in ParSetup as offsets from the isocentre of the magnet and appear as FOV_Offset_X, FOV_Offset_Y and FOV_Offset_Z in mm. The relationship between the physical offsets in mm and the values passed back to the PPL variables are as follows:

```
read_off_pplvalue = (FOV_Offset_X /FOV)*Scale  
phase_off_pplvalue= (FOV_Offset_Y /FOV)*Scale  
slice_off_pplvalue = (FOV_Offset_Z /FOV)*Scale
```

and the format of the offset keyword in the Parameter list header of the pulse program is:

FOV_Offset_ Keyword, minimum, maximum, default, Scale.

PPL values are integer so the minimum increment of an offset would be (FOV/Scale) and the maximum allowed offset would be (FOV/Scale)*32767. For example, a FOV of 180 mm and a scale factor of 4000 would give a minimum offset increment of 0.045 mm and a maximum offset of 1475 mm. So the choice of the scaling factor is a matter of the required accuracy in the offset values and the maximum value required. For example, in microscopy applications very small offsets would be required whereas offset accuracy of 0.005 mm for whole body applications would not really be necessary. An offset in the read direction is achieved by offsetting the receiver by the appropriate value in Hz:

$(\text{FOV_Offset_X}/\text{FOV}) * \text{Acquisition bandwidth.}$

So the calculation for the read offset in Hz in the PPL program is:

$(\text{read_off_pplvalue}/\text{Scale})\text{acquisition bandwidth.}$

Offsets in the phase encode direction are achieved by incrementing the receiver phase with respect to the transmit phase after each phase encoding step. This achieves a pseudo frequency shift in the phase encode direction. If the offset is to be 1 FOV then the increment should be 360° from view to view. The calculation for this increment in the PPL program is, therefore:

$(\text{phase_off_pplvalue}/\text{scale})*360^\circ.$

Offsets in the slice direction are achieved by offsetting the RF transmit frequency by the required fraction of the RF pulse bandwidth. The calculation of this offset in Hz within the PPL program is then:

$(\text{slice_off_pplvalue}/\text{scale})*\text{RF Pulse bandwidth.}$

All these calculations are made at the start of the gradient controller imaging sequences in the offset macros: READOFF, PHASEOFF and SLICEOFF.

5.8 Appendix C: Timings of Gradient Controller Functions

5.8.1 **MR3040_SetListAddress.**

This function takes 2.1 μ s + argument evaluation time to execute.

5.8.2 **MR3040_InitList()**

This function takes 4.9 μ s + argument evaluation time to execute.

5.8.3 **MR3040_Output(int loop_type, int frame_address, int frame_points, int frame_rate)**

This function takes 12.9 μ s + argument evaluation time to execute.

5.8.4 **MR3040_Hold(int loop_type)**

This function takes 7.6 μ s + argument evaluation time to execute.

5.8.5 **MR3040_LoopCount(int loop_count)**

This function takes 1.9 μ s + argument evaluation time to execute.

5.8.6 **MR3040_SetList(int address, int channel_id)**

This function takes 1.2 μ s + 1.9 μ s per selected channel + argument evaluation time to execute.

5.8.7 **MR3040_Start(int channel_id)**

This function takes 2.3 μ s + argument evaluation time to execute.

5.8.8 **MR3040_Continue(int channel_id)**

This function takes 2.3 μ s + argument evaluation time to execute.

5.8.9 *MR3040_Break(int channel_id)*

This function takes 2.3 μ s + argument evaluation time to execute.

5.8.10 *MR3040_Stop(int channel_id)*

This function takes 2.3 μ s + argument evaluation time to execute.

5.8.11 *MR3040_Control(int control_bits)*

This function takes 1.7 μ s + argument evaluation time to execute.

5.8.12 *MR3040_RunChannels(int address3, int address2, int address1)*

This function takes 6.6 μ s + argument evaluation time to execute.

5.8.13 *MR3040_Clock(int duration)*

This function takes 1.9 μ s + argument evaluation time to execute.

5.8.14 *MR3040_SeqStatus(int mask)*

This function takes 1.9 μ s + argument evaluation time to execute.

5.8.15 *MR3040_CreateBaseMatrix(int z_angle, int y_angle, int x_angle, int div_z, int mul_z, int div_y, int mul_y, int div_x, int mul_x)*

If successful, this function takes 56.5 μ s + argument evaluation time to execute. If unsuccessful, this function takes 3.8 μ s + argument evaluation time to execute. The matrix calculation will take between 18.0 μ s and 71.0 μ s after the function returns to complete, depending upon the argument values.

5.8.16 *MR3040_CreateMatrix(int matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle)*

If successful, this function takes 60.1 μ s + argument evaluation time to execute. If unsuccessful, this function takes 4.1 μ s + argument evaluation time to execute. The matrix calculation will take between 19.0 μ s and 75.0 μ s after the function returns to complete, depending upon the argument values.

5.8.17 MR3040_ModifyMatrix(int_dest_matrix_id, int div_3, int mul_3, int div_2, int mul_2, int div_1, int mul_1, int flip_orientation, int z_angle, int y_angle, int x_angle, int source_matrix_id)

If successful, this function takes 60.1 μ s + argument evaluation time to execute. If unsuccessful, this function takes 4.1 μ s + argument evaluation time to execute. The matrix calculation will take between 19.0 μ s and 75.0 μ s after the function returns to complete, depending upon the argument values.

5.8.18 MR3040_MatrixStatus(int mask)

This function takes 1.9 μ s + argument evaluation time to execute.

5.8.19 MR3040_SelectMatrix(int matrix_id)

This function takes 1.9 μ s + argument evaluation time to execute.

5.9 Appendix D: File format specification

5.9.1 MRD: Raw spectrometer data files

Spectral data is stored in a file with an extension .MRD. The file format consists of:

- 1) a 256 byte header
- 2) a 256 byte text description
- 3) the data proper
- 4) an ASCII string containing the sample information filename
- 5) an ASCII copy of the parameter file (.PPR file) used

byte header

Bytes (hex)	'C' data type	Usage
0-3	Long (4 byte integer)	Dimension 1, number of data samples
4-7	Long	Dimension 2, number of views (2D data)
8-B	Long	Dimension 3, number of secondary views (3D data)

C-F	Long	Dimension 4, number of slices
...	Unspecified	
12-13	Int (2 byte integer)	Data type code
...	Unspecified	
98-9B	Long	Dimension 5, number of echoes
9C-9F	Long	Dimension 6, number of experiments

The data type code indicates the format of individual data elements, as follows:

0x00	unsigned char	1 byte
0x01	signed char	1 byte
0x02	short	2 bytes
0x03	int	2 bytes
0x04	long	4 bytes
0x05	float	4 bytes
0x06	double	8 bytes
0x10	bit mask indicating complex data as real, imaginary pairs of base data type	

All .MRD files are at present signed char, int or floating point, either real or complex. The default is complex floating point, data type 0x15.

256 byte text

Bytes (hex)	'C' data type	Usage
100-1FF	256 bytes ASCII	Text, 0 terminated string

This is normally unused

Data

Bytes (hex)	'C' data type	Usage
200 -->	Complex or single value	Data
	Signed char, int or float	

Multi-dimensional data are stored sequentially, the index of dimension 1 (samples) varying most rapidly, followed by the other dimensions in order i.e. samples (1DFT), secondary views (3DFT), views (2DFT), slices, echoes, and experiments.

Sample file name

This is the fully qualified path name of the sample information file, as a zero terminated ASCII string, currently padded to 120 bytes.

Parameters

The contents of the .PPR parameter file. This is ASCII text, in lines delimited by <CR> <LF> (0x0d, 0x0a). The format is described below.

In the .MRD and .SUR file, the .PPR file proper is followed by entries indicating the instrument parameter values at the time of the acquisition.

```
:_ObserveTransmitGain <dB10><CR><LF>
:_ObserveReceiverGain <dB10><CR><LF>
:_ObserveReceiverAttenuation <dB10><CR><LF>
:_DecoupleTransmitGain <dB10><CR><LF>
:_ObserveModulatorOffset <DACValue><CR><LF>
:_ObserveModulatorFineOffset <DACval><CR><LF>
:_DecoupleModulatorOffset <DACval><CR><LF>
:_DecoupleModulatorFineOffset <DACval><CR><LF>
```

<db10> is the decimal value of the gain in units of 0.1 dB. <DACval> is the decimal value, lying between 0 and 63 inclusive, written to the appropriate RF unit device.

The .MRD file is terminated by the entry

```
:END<CR><LF>
```

5.9.2 SUR: Reconstructed image data files

Reconstructed data is stored in a file with an extension .SUR. They are in a format very similar to the first part of that used for raw data (.MRD). They consist of:

- 1) a 256 byte header
- 2) a 256 byte text description
- 3) the data proper
- 4) the pulse program parameters (PPR file)
- 5) other parameters

256 byte header

Bytes (hex)	'C' data type	Usage
0-3	Long (4 byte integer)	Number of pixels in X direction

4-7	Long	Number of pixels in Y direction
8-B	Long	Number of pixels in Z direction (currently 1)
C-F	Long	Dimension 4 (currently 1)
...	Unspecified	
12-13	Int (2 byte integer)	Data type code
...	Unspecified	
30-33	Float (4 bytes)	Scaling factor from highest pixel to 4095
6D	Char (1 byte)	Number of bits per pixel (currently 12)
...	Unspecified	

The data type code indicates the format of individual data elements, as follows:

0x00	unsigned char	1 byte
0x01	signed char	1 byte
0x02	short	2 bytes
0x03	int	2 bytes
0x04	long	4 bytes
0x05	float	4 bytes
0x06	double	8 bytes

All .SUR files are at present integer, indicated by data type 0x03.

256 text description

Currently left blank.

Data

The pixel positions increments in the X dimension until the next increment in the Y dimension.

Parameters

The contents of the .PPR parameter file that was used to obtain the image. This is ASCII text, in lines delimited by <CR> <LF> (0x0d, 0x0a). The format is described below.

In the .MRD and .SUR file, the .PPR file proper is followed by entries indicating the instrument parameter values at the time of the acquisition.

Individual .SUR format images also contain orientation information for each individual image after the .PPR file information

:IM_ORIENTATION <x_angle>, <y_angle>, <z_angle><CR><LF>

:IM_OFFSETS <x_mm>, <y_mm>, <z_mm><CR><LF>

:IM_FOV <x_mm>, <y_mm>, <z_mm><CR><LF>

5.9.3 PPR: sequence parameter files

Pulse sequence parameters are stored in a parameter file generated by the Parameter editor program ParSetup. This is an ASCII file with the extension .PPR. The parameters recorded are only those required by the specific pulse program, as defined in the PARAMLIST header section of the pulse program.

The .PPR file consists of a variable number of lines of ASCII text. Each entry starts with a colon (:) followed immediately by a keyword, and optional data elements delimited by a comma and/or white space. Further information may be included after the data required by the keyword, which is considered as a comment (this facility is not currently used). The presence of a colon character preceded by white space indicates the start of a new file entry.

Most PPR file entries are in a common format, although a few keywords exist with different formats; these are described after the more general case.

Entries of the general format are as follows:

:<keyword> <ppl_var>, <ppl_value><CR><LF>

<keyword> is a string normally identical to the corresponding keyword in the PARAMLIST section of the .PPR file, although a different keyword is used in certain cases.

<ppl_var> is the name of the .PPL/Forth variable to which the value will be sent. This will normally appear with character cases as in the .PPL PARAMLIST header, although the Forth system is case-insensitive.

<ppl_value> is the decimal value in .PPL units which is sent to the pulse programmer and is the initialisation value of the named .PPL variable. The relationship between this value and the value entered in the ParSetup program, and the allowed range is documented on a keyword by keyword basis in the NMR system manual, with updates appearing in subsequent release notes.

The possible .PPR keywords with this format are as follows:

NO_AVERAGES
NO_SAMPLES
NO_VIEWS
NO_VIEWS_2
NO_SLICES
NO_ECHOES

EXPERIMENT_ARRAY
SLICE_BLOCK
VIEW_BLOCK
PHASE_CYCLE
DISCARD
SLICE_INTERLEAVE
ORIENTATION
SAMPLE_PERIOD (from SAMPLE_PERIOD or SPECTRAL_WIDTH)
SAMPLE_PERIOD_2
NO_RECEIVERS
VAR (from EDITTEXT or SCROLLBAR)

In multiple receiver systems, a number of the keywords may be modified by appending “#n” to indicate a receiver specific parameter.

Variables defined by :VAR are general purpose variables, whose contents may be used for many purposes within the pulse program. There may be many such lines in the .PPR file, generated in response to the PARAMLIST keywords EDITTEXT and SCROLLBAR. The variables may be of PPL language type int or common int (interactively adjustable). Such variables are used for any parameter in the pulse program that does not also affect another part of the instrument (e.g. filter settings, dimensionality of acquisition). Delay times and pulse times are such variables.

The following keywords are obtained from the FOV (field of view) PPL keyword.

```
:READ_VAR <read_var>, <read_value><CR><LF>  
:PHASE_VAR <phase-var>, <phase_value><CR><LF>  
:FOV <fov_value><CR><LF>  
:SWX <sw_value><CR><LF>  
:SMX <sm_value><CR><LF>  
:SWY <sw_value><CR><LF>  
:SMY <sm_value><CR><LF>  
:SWZ <sw_value><CR><LF>
```

<fov_value> is the entered field of view in mm (floating point). <read_value> and <phase_value> are the values written to the PPL/Forth variables <read_var> and <phase_var> to be used as the multiplier values to achieve the desired FOV.

<sw_value> is the signed floating point number (absolute value lying between 1.0 and 0.5) used to scale the gradient waveform data to achieve the desired FOV and slice thickness

<sm_value> is the floating point number by which the full scale multiplier value was scaled to generate the <read_var> and <phase_var> values.

The following entries have their own format

:PPL <filename><CR><LF>

:DSP_ROUTINE <filename><CR><LF>

<filename> is the fully qualified pathname of the file associated with the keyword

:OBSERVE_FREQUENCY "nucleus" <freq>, <MHz>, <kHz>, <Hz>, <rx1MHz><CR><LF>

:DECOUPLE_FREQUENCY "nucleus" <freq>, <MHz>, <kHz>, <Hz>, <rx1MHz><CR><LF>

<nucleus> is the text string by which the nucleus is identified in the [nuclei] section of parfilio.ini. <freq> is the scanner centre frequency in Hz. <xHz> are the PPL/Forth variable names to which the appropriate sections of the centre frequency are written.

The acquisition program Scan splits the frequency into MHz, kHz and Hz components which are passed to the first three .PPL variables in the list, and also passes a value to the fourth PPL variable (consult 3.3.1.1 for details). The frequency is derived by looking up the text string <nucleus> in a table in the [nuclei] section of parfilio.ini, and adding this to an offset in the Parameter Editor. If the number in the file is changed directly and then the Parameter Editor restarted it will regenerate the offset value, and care must be taken to prevent this exceeding the limits defined in the PARAMLIST section of the PPL file.

:VAR_ARRAY <ppl_var>, <no_elements>, <element0>, <element1> ... <CR><LF>

<ppl_var> is the name of the PPL/Forth array variable to which the data values will be sent. <no_elements> is the number of values to send to the array. <elementx> is the value to be sent to array element ppl_var[x]. This entry may extend over as many lines as necessary by separating values either by commas and white space or by <CR><LF> as required.

:DATA_TYPE 0x<value><CR><LF>

<value> is the data type code for the data file (also contained in the .MRD file header), in ASCII HEX.

:DISCARD PPL VarName, <int><CR><LF>

<int> is the number of additional sample points collected and discarded in the digital filter during acquisition.

:BUFFER_SIZE <value><CR><LF>

<value> is the requested size in bytes (decimal format) of the blocks of data transferred from the DSP to the data file. The DSP will attempt to offload data by batching together data sets that are contiguous in both time and the order of the data file.

:SLICE_THICKNESS <ppl_var>, <ppl_value>, <mm><CR><LF>

:SLICE_SEPARATION <ppl_var>, <ppl_value>, <mm><CR><LF>

<mm> is the slice thickness in mm, appended to the otherwise standard format

:FOV_READ_OFF <ppl_var>, <no_slices> {,<multslicen>} * number of slices<CR><LF>

:FOV_PHASE_OFF <ppl_var>, <no_slices> {,<multslicen>} * number of slices<CR><LF>

:FOV_SLICE_OFF <ppl_var>, <no_slices> {,<multslicen>} * number of slices<CR><LF>

:FOV_OFFSETS <no_slices>{, <x_mm>, <y_mm>, <z_mm>} * number of slices<CR><LF>

for a multi-slice experiment <multslicen> is the slice offset for slice n in the specified direction sent to the PPL/Forth variable, <x_mm>, <y_mm>, <z_mm> the offsets in mm.

either

:MULTI_ORIENTATION <no_slices> {,<orientation_index>} * number of slices<CR><LF>

or

:OBLIQUE_ORIENTATION <orientation_index>

:X_ANGLE <ppl_var>, ,no_slices> {, <angle10>} * number of slices<CR><LF>

:Y_ANGLE <ppl_var>, ,no_slices> {, <angle10>} * number of slices<CR><LF>

:Z_ANGLE <ppl_var>, ,no_slices> {, <angle10>} * number of slices<CR><LF>

:PHASE_ORIENTATION <ppl_var> 0/1<CR><LF>

For single angle oblique the keywords containing <no_slices> have just a single bracketed entry. For multi-angle oblique the entries are specified for each slice as above. <angle10> is the angle in 0.1 degree steps.

Chapter 6. COMPILER WARNINGS AND ERROR MESSAGES

6.1 Warnings

The compiler issues warning messages if it encounters situations which could be a "non-fatal" programmer error. Compilation and the production of a downloadable **.fth** file will still occur after a warning has been issued, but the programmer is urged to check the PPL source file before continuing with downloading and running. The compiler issues the following warning messages:

- **Warning 1: text after closing bracket on line n.**

If any PPL code is detected after the closing bracket of the program, this text is ignored by the compiler.

- **Warning 2: name "identifier" too long.**

If any identifier in the program exceeds 31 characters, only the first 31 characters of the identifier are used and rest is ignored.

- **Warning 3: label lab_name defined on line n never used.**

If label lab_name appears in the program without ever appearing in a **goto** statement.

- **Warning 5: Possibly incorrect assignment on line n.**

The expressions in **if** and **until** structures are checked in case the programmer intended "==" rather than "=".

- **Warning 6: Pulse function timing expects int array index on line n.**

The pulse function timing has been arranged to compensate for the time taken to access its arguments: however, if an array element is used as an argument to this function, the timings will only be accurate if the array element index is an int.

6.2 Error messages

Whenever an error is encountered by the compiler the production of an output **.fth** file is aborted, an error message is produced, and the compiler proceeds with error checking as far as possible before exiting to DOS. The error messages are intended to be self-explanatory, but there are several messages (#7, 8, 10, 14, 30, and 33) which may require further explanation.

- Error 1: re-declaration of XXXX on line n
Variable XXXX is declared twice (or more): each variable and constant must

have a unique identifier. It generally happens when a user does not notice that the variable has already declared in VAR_15.PPH.

- **Error 2: unexpected end of file encountered on line n**
This error message can be the result of a number of circumstances, the most frequently occurring being that of mismatched braces - the closing brace at the end of the program not being found due to a mismatched brace somewhere in a compound statement. Alternatively, the error may arise if a close comment is omitted, or a closing parenthesis in an expression may be missing.
- **Error 3: expecting main in line n**
The name "main" is the first name the compiler expects to encounter.
- **Error 3: expecting open bracket in line n**
- **Error 3: expecting close bracket in line n** - After main, open and close parentheses are expected.
- **Error 3: expecting open brace on line n** - After main() a { is expected.
- **Error 3: expecting variable name in line n**
After the declaration keyword int, a list of variable names, separated by commas and terminated with a semi-colon is expected.
- **Error 3: expecting ";" in line n** - A semi-colon is expected at the end of a declaration.
- **Error 3: expecting constant name in line n**
An identifier is expected after the declaration keyword constant.
- **Error 3: expecting number in line n**
A number is expected after constant identifier, followed by a semi-colon.
- **Error 4: Unexpected character x on line n**
The character x is not an alphanumeric character or one of the recognized punctuation characters.
- **Error 5: Can't find close comment matching open on line n**
This is a missing */ after the opening /* on line n.
- **Error 6: Can't nest comment on line n1 within comment on line n2**
/**/ Comments cannot be nested. However, \\ comments can appear within a comment, where they are ignored.
- **Error 7: Not enough space to allocate n bytes for symbol table**
If this error message is produced, more memory must be made available before the compiler run - this might be achieved by removing any unnecessary memory - resident programs.

- **Error 8: Symbol table overflow**

This error occurs when the compiler runs out of space for storing variable, constant and label identifiers. Again, it is unlikely that a user will ever encounter this error, but should it arise, it may be overcome by replacing long identifiers with shorter identifiers.

- Error 9: = expected after xxxx on line n

xxxx has been declared as a variable, and is expected to be followed by an assignment operator on line n.

- **Error 10: Not enough space to allocate n bytes for parse tree**

Likewise, more memory must be made available.

- Error 11: syntax error after token xxx on line n

A syntax error has been encountered in an expression on line n - something other than a primary has been encountered.

- Error 12: right bracket missing on line n.

A closing bracket is missing after the expression on line n.

- Error 13: expected binary operator, not xxx on line n.

A binary operator was expected in the expression on line n.

- **Error 14: parse tree overflow on line n**

This error occurs when the compiler runs out of memory whilst translating a very long expression. It is highly unlikely that a user will ever encounter this error, but should it happen it can be remedied by breaking the expression up into shorter "sub" expressions thus:

$a = b * c / d + (f * g + h) / l - p * g / (r - Z);$ \\ \\ Long expression can be rewritten

$tmp1 = b * c / d + (f * g + h) / l;$ \\ \\ as two shorter expressions

$tmp2 = p * g / (r - Z);$

$a = tmp1 - tmp2;$

- Error 15: compiler error: tried to translate token xxx in line n - please contact MR Solutions.

- Error 16: found xxx where semi-colon expected in line n

Either a semi-colon has been omitted, or there is a syntax error in the expression on line n.

- Error 17: can't assign to [constant/undeclared object] xxx on line n

Assignment is only legal if the left hand side of the assignment operator is a declared variable.

- Error 18: line n: xxxx undeclared

An undeclared object has been encountered in an expression. All constants and

variables must be declared at the top of the program, immediately following main() {.

- Error 19: compiler error: Line n: name xxx has symbol table def x - please contact MR Solutions.
- Error 20: expected open bracket after xxxx on line n – "("is missing after xxx.
- Error 21: close bracket missing after xxx(on line n ")" is missing after the expression xxx{.
- Error 22: Can't assign to non-variable on line n
Only declared variables are allowed on the left hand side of the assignment operator.
- Error 23: "until" expected on line n1 to match "do" on line n2
The syntax of the do-until control structure is do compound-statement until(expression);
- Error 24: opening " missing in printf statement on line n
The syntax of a printf statement is printf(" string" [,args]); - the opening and closing quotation marks are compulsory.
- Error 25: newline in string on line n
The control string in a printf statement must not contain any newline characters. If a newline is required on the console output, the characters \n must be inserted in the control string.
- Error 26: comma expected in argument list on line n
Arguments to the printf and delay functions must be separated by commas.
- Error 27: arguments mismatch in printf statement on line n
For each %d or %x in the printf control string, there must be a corresponding argument in the control list, and vice versa.
- Error 28: xxxx procedure not defined on line n.
In this version of the P2F compiler, there are no user definable functions or procedures.
- Error 29: read expects d or x as arguments on line n
read(d); returns the value of a decimal number read in from the console keyboard, and read(x); returns the value of a hexadecimal number read in from the keyboard.
- **Error 30: compiler error. Failed to set up input buffer**
are both the result of inadequate memory resources. p2f uses buffered file I/O to optimise disk accesses, and needs at least 2 K of memory to be used as buffer space.

- Error 31: compiler error: can't open library file lib.fth - lib.fth file needs to be in c:\smis\ppl\lib
- Error 32: can't open file xxxx
The output file cannot be created. This is mainly because the file directory typed is wrong.
- **Error 33: compiler error. Can't allocate buffer in copy_hdr()**
are both the result of inadequate memory resources. p2f uses buffered file I/O to optimise disk accesses, and needs at least 2 K of memory to be used as buffer space.
- Error 34: compiler error: Problems writing to xxx in copy_hdr - please contact MR Solutions.
- Error 35: expected a second \ on line n
The syntax of the single line comment is \\
comment ... text after \\
is ignored up to the end of the line.
- Error 36: label xxx already declared [constant/variable] on line n
A label must have a unique identifier, and is declared by its identifier being followed by a colon at the place in the program where the label is required.
- Error 37: too many labels! (Last label encountered on line n) - Only 10 labels are allowed.
- Error 38: label expected after goto on line n
The syntax of a goto statement is goto label where somewhere in the program the statement label: appears.
- Error 39: Number expected after identifier in const declaration on line n
Syntax of const declaration is const identifier number;
- Error 40: label xxx encountered on line n already in use
Each label can only mark one position in the program.
- Error 41: %? conversion not recognized in line n
- The only % conversions recognized by printf are %% to print the % character, %d to print a decimal value and %x to print a hexadecimal value.
- Error 42: compiler error: object xxx of unknown class ? on line n - please contact MR Solutions.
- Error 43: goto on line n uses undefined label xxx.
The label which the goto is attempting to reach does not exist within the program.
- Error 44: constant, number or variable expected as arguments to delay procedure on line n

- Error 47: sec, ms or us expected in argument list to delay on line n
The syntax for the delay functions is `delay( number/constant/variable, us/ms/sec);`
- Error 48: digit expected after input# on line n - The digit can be one of 0 ~ 7.
- Error 50: x is invalid after input# on line n - a digit in the range 0 ~ 7 is expected.
- Error 51: arguments missing from xxx function on line n
The function xxx expects some arguments: `input()` and `output()` expect an expression as an argument which is used to mask input values and set output values.
- Error 52: syntax error on line n. Token xxx unexpected.
This most frequently occurs as the result of trying to recover from previous errors and continuing to error check. However, it may occur if a primary such as `input#1` occurs at the beginning of a statement, i.e. in an unexpected place.
- Error 54: argument xxx to procedure delay is a label on line n.
Labels can only appear after `goto` or on labelling statements.
- Error 55: syntax error in argument list to procedure delay on line n - See error 47.
- Error 56: closing bracket not found after delay on line n
There must be a `);` after the arguments to the procedure
- Error 57: invalid characters xxx in number on line n
Decimal numbers start with a digit in the range 1 ~ 9, and can only contain digits.
Hex numbers start with `0X` or `0x` and can only contain digits or the letters `a ~ f` or `A ~ F`.
- Error 58: syntax error on line n. Binary operation `xx` on 2 constants - please contact MR Solutions.
- Error 59: token xxx found on line n where unary operator expected - please contact MR Solutions.
- Error 60: value of delay must be positive on line n.
When `delay(const_expression, order );` is called, `const_expression` is checked.
- Error 64: Compiler error: can't allocate buffer in `concat()`
This message will appear when the system is running low on memory: this may be due to the presence of a large number of "terminate and stay resident" programs. Try removing them and recompiling.
- Error 65: Argument XXXX to function XXXX on line n is a label
Labels cannot appear as function arguments: all function arguments must be expressions which evaluate as int or common int.

- Error 66: sec, ms or us expected in argument list to XXXX on line n
This message appears if there is a syntax error in the argument list of the pulse or delay functions.
- Error 67: line n: const expression or variable expected as arg n to XXXX
This message is produced by the compiler for those functions which will not take complex expressions involving variables as arguments.
- Error 68: Arg 4 of pulse must be a const. expression on line n
- Error 69: Pulse width argument must be positive on line n
- Error 70: Expecting XXXX after YYYY on line n
"General purpose" error message: the strings XXXX and YYYY should provide sufficient information to fix the error.
- Error 71: Failed to find terminating " after #use on line n
The syntax of the #use statement is #use "pathname" alias - see section 5.4.2.
- Error 72: Compiler error. Can't allocate buff in XXXX
This error message is symptomatic of running low on memory - see Error 64 on page 162 for suggestions on fixing this.
- Error 73: Compiler error. Failed reading into buff in XXXX
If this message appears, there has been an unexpected file reading error: please contact MRRS
- Error 74: Image sequence file XXXX: version n is incompatible
Currently, only version 1 image sequence files are supported.
- Error 75: Seek failed in Image Sequence File XXXX
The Image Sequence File may be corrupted.
- Error 76: Failed reading time frame names from XXXX
The Image Sequence File may be corrupted.
- Error 77: Expected alias after #use XXXX on line n
See Error 71.
- Error 78: Can't open Image Sequence File XXXX
- Error 79: Corrupt/invalid data read from Image Sequence File XXXX
- Error 80: Image Sequence data will overflow Image Sequencer!
- Error 81: Redclaration of alias XXXX on line n
- Error 82: Compiler error. Can't find XXXX in symbol table - please contact MR Solutions.

- Error 83: Compiler error. XXXX has symbol table entry c XXXX - please contact MR Solutions
- Error 84: XXXX defines n channels. Expected n.
All image sequence files used together must all define the same number of channels.
- Error 85: Line n: Can't find XXXX in XXXX - Invalid frame reference.
- Error 86: Syntax error in time frame label argument to im_seq on line n
- Error 87: Duration of n in time frame exceeds limit of n in line n
- Error 88: Missing (after scale on line n
- Error 89: Missing , after first scale argument on line n
- Error 90: Missing , after second scale argument on line n
- Error 91: Missing) after third scale argument on line n
- Error 92: Unexpected token XXXX after YYYY on line n
"General purpose" error message - the XXXX and YYYY strings should make the cause of the error clear.
- Error 93: Identifier type XXXX unexpected as primary on line n
- Error 94: Syntax error in time frame label string on line n
- Error 95: Index to array XXXX out of range on line n
Compile-time checking of array indices is performed if the array index is a literal number or declared constant. Do not relax your vigilance, however - it is still possible to overstep the bounds of an array by using a variable for the array index, and the compiler will not detect this!
- Error 96: XXXX argument to YYYY on line n must be a ZZZZ
"General purpose" error message produced when an unexpected function argument is encountered. The XXXX, YYYY and ZZZZ strings should make the cause of the error clear.

Chapter 7. PULSE PROGRAMS EXAMPLES

All these demonstration pulse program examples are located in c:\smis\seqlib. It is intended for the user to run these programs and monitor the output on an oscilloscope in addition to reading this document. The DEMO programs should be run with the GRADIENT AND RF hardware turned **OFF**. They are only intended as a gradient controller teaching aid and are not useful as a real MRI experiment. The GE_DEMO.PPL program is, for example, a simple gradient echo sequence that will run on any system, providing the selected gradient ramp times are achievable with the system gradient hardware. The following examples are used to explain the various gradient controller function calls. DEMO_A.PPL is an important, but simple, example program. This should be closely studied because it contains many elements that are common to all gradient controller pulse programs.

DEMO_A.PPL:	Demonstrates the use of the MR3040_Output command, which is the most important function for gradient controller programming.
DEMO_B.PPL:	Demonstrates the use of the MR3040_Continue and MR3040_Hold functions.
DEMO_C.PPL:	Demonstrates the use of primary and secondary matrices and the MR3040_Clock function.
DEMO_D.PPL:	Demonstrates the use of the loop structure within the MR3040_Output function.
GE_DEMO.PPL:	GE imaging sequence with variable ramp times and 5-lobe sinc 1500 Hz RF pulse.
GEDEM_MC.PPL:	GE imaging sequence identical to GE_DEMO.PPL, except that macro calls are used for ease of programming.
FSE_PF.PPL:	Fast Spin Echo with partial k-space acquisition and reconstruction option

7.1.1.1 DEMO_A.PPL

/*****

Sequence Name: DEMO_A.PPL
 Author: MR Solutions Ltd.
 Date: 08 February 2011
 Note: This is the PPL DEMO file for testing all RF units.

This 3040 demo program is a very simple example of the use of the MR3040 board. Two gradient pulses are output along the read channel X axis. These illustrate the use of the

MR3040_Output function which is the main function for using the 3040 board. Two gradient pulses are applied along the X axis of duration fixed duration = 5 ms and variable duration tplat2.

The ramp times used are 1ms in duration. The RF blanking line may be used as an external trigger for an oscilloscope.

PLEASE NOTE THIS DEMO SEQUENCE IS ONLY INTENDED AS A TEACHING AID TO MR3040 PROGRAMMING AND AS SUCH SHOULD ONLY BE USED WITH THE RF AND GRADIENT HARDWARE TURNED OFF.

```

***** /

#use RF1 "c:\smis\seqlib\RFstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad

/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
SPECTRAL_WIDTH 1000, 400000, 25000, sample_period ;
GRADIENT_STRENGTH grad_var;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT "Repetition time","ms","%d",10,2000,600,1,tr;
EDITTEXT " Second pulse duration","us","%d",15,32000,500,1,tplat2;
NO_AVERAGES 1, 10000, 2, no_averages ;
DSP_ROUTINE "c:\smis\dsp" ;
DISCARD 0, 10, 0, no_discard ;
DATA_TYPE 0x13;
END
*/

#include "stdfn_15.pph"
#include "std3040.pph"
main(){
const tramp          1000 ;  \\ ramp time (in us) at each gradient.
/* constants */
const aq_mat          3;
const calc_mat         255;          \\ Select an unused matrix during matrix calculations
const caldelay         100;          \\ Give DSP 100us to finish new matrix
const CHANNEL_R        0x0002;      \\ For use in MR3040_Start etc
const CHANNEL_P        0x0020;
const CHANNEL_S        0x0200;
const CHANNEL_PS       0x0220;
const ALL_CHANNELS     0x0222;
const NOLOOP           0;
const STARTLOOP        4;
const COUNTLOOP        8;
const JUMPBACK          12;
const max_waits         4095;

/* looping control */
int no_averages, completed_av, sample_period, no_samples, no_discard, phase_cycle, no_acq,
obs_power;
/* timing variables */
int tplat1, tplat2;

```



```
common int tr;
/* frequency control */
common int MHz, kHz, Hz, rx1MHz, rec_freq ;

/* Variables for gradient timings */
int ret, grad_var[4], slice_list, read_list, read_lf1;

/* orientation variables */
common int phase_var, s_angle_var, p_angle_var, r_angle_var;

/*****
```

Point 1:

```
ret = MR3040_CreateBaseMatrix( 0, 0, 0, grad_var[3], grad_var[0], grad_var[2],
    grad_var[0], grad_var[1], grad_var[0] );
if(ret){
    printf( "Could not create base matrix, status = %d\n", ret);
    goto end;
}
delay(caldelay,us);
```

Point 2:

```
MR3040_SetListAddress(0);
read_list = MR3040_InitList();
```

Point 3:

```
MR3040_Output( NOLOOP, grad.address."grad_pulse", grad.size."grad_pulse",
grad.waits."grad_pulse");
MR3040_Output( NOLOOP, grad.address."ramp_zero_to_max_1ms",
grad.size."ramp_zero_to_max_1ms",
    grad.waits."ramp_zero_to_max_1ms");
MR3040_Output( NOLOOP, grad.address."ramp_max_to_zero_1ms", 1, tplat2);
MR3040_Output( NOLOOP, grad.address."ramp_max_to_zero_1ms",
grad.size."ramp_max_to_zero_1ms",
    grad.waits."ramp_max_to_zero_1ms");
sync(); \\ Synchronise to 100kHz.
\\ ***** MAIN ACQUISITION LOOP *****
\\ Looping structure follows that demanded by SCAN.
completed_av = 0;          \\ counter for averages loop
averages_loop:
```

Point 4:

```
/* Calculations supporting real-time adjustments. Check for overflow */
MR3040_SelectMatrix( calc_mat ); /* Select unused matrix during calcs */

\\ DEFINE THE PRIMARY MATRIX
ret = MR3040_CreateMatrix( aq_mat,1024, 0,1024, 0,1024, -512, phase_var, s_angle_var,
p_angle_var, r_angle_var );
if ( ret ){
```

```

        printf( "Failed to create aq matrix, return code = %d\n", ret );
        goto end;
    }
    delay( caldelay, us );
    hostrequest();
    resync();
    \\ ***** SEQUENCE STARTS HERE *****

```

Point 5:

```

MR3040_SetList(read_list, CHANNEL_R );
MR3040_SelectMatrix( aq_mat );
/* Output a RF pulse so that RF blanking can be used as a trigger */
MR3031_setup(pf1,"rf_analog_on",0,0,0,2000);
rfampon(0);
delay(20, us);
rfon(1);
MR3031_go();
delay(100, us);
rfoff();
MR3031_setup(pf1,"rf_off",0,0,0,0);
MR3031_go();
/* Start 3040 channel _R */

```

Point 6:

```

MR3040_Start(CHANNEL_R);
ret= 1000 + tplat2 + (tramp*4);
delay(ret,us);
acquire( sample_period, no_samples );
delay(tr,ms); /* Wait for tr */
completed_av=completed_av+1;
if(completed_av < no_averages)
    goto averages_loop;
end:
}

```

• Point 1 (DEMO_A.PPL)

The MR3040_CreateBaseMatrix creates a base matrix from which all other rotation matrices are calculated as well as rescaling the gradients to be isotropic. The first three arguments are the z, y and x rotations for the base matrix. Under normal circumstances no rotation is required for this base frame of reference. A good example of a base matrix rotation would be if a patient is placed in a magnet feet first so the z axis should be rotated by 180° to give the same image orientation as a patient placed in the magnet head first. In other words these rotations define the patient orientation. (the rotation matrices governing the physical gradient output). The array grad_var contains 4 elements the minimum gradient strength, the x, y and z gradient strengths which are read from the PARFILIO.INI file by PARSETUP. All 3 gradients are then scaled down to the minimum value so that the gradient efficiencies are equal in all three directions (i.e.

isotropic). It is important that the gradient strengths are isotropic otherwise the calculations for gradient amplitudes to give a particular rotation would be inaccurate. As example consider a gradient set with the following strengths:

$$G_x = 255 \text{ Hz/mm}, G_y = 270 \text{ Hz/mm and } G_z = 300 \text{ Hz/mm}$$

then the values placed in grad_var would be:

$$\text{grad_var}[0] = 255, \text{grad_var}[1] = 255, \text{grad_var}[2] = 270, \text{grad_var}[3] = 300$$

and in the base matrix call all 3 gradients would be scaled down to the minimum value of 255. There is a delay of 100 μs (defined as "caldelay" in the program) to allow for execution time of this function. This function returns zero value if successful and a non-zero value if unsuccessful, for example if the DSP is still processing a previous command.

- **Point 2 (DEMO_A.PPL)**

The MR3040_SetAddress sets the current address pointer to the supplied argument, which in this case is zero. All lists, i.e., sets of instructions written to the sequencer memory will start from this address. The next instruction MR3040_InitList() creates a single element instruction list consisting of a STOP instruction at the next available location. At this point in the program the sequencer will be loaded with the following instructions:

ADDRESS 0: STOP;

and the address 0 will be identified by the label "read_list". There then follows a series of MR3040_Output instructions. Each time a call to an MR3040_Output function is made the current instruction is overwritten, so if there is one call to this function then the instruction list becomes:

ADDRESS 0: OUTPUT
STOP

If there are two calls to the output function made then the instruction list would be:

ADDRESS 0: OUTPUT
OUTPUT
STOP

and three calls would be :

ADDRESS 0: OUTPUT
OUTPUT
OUTPUT
STOP

The MR3040_Output function has 4 arguments. The 1st argument refers to the type of loop structure used by the function. In this example NOLOOP is used. It is possible to use this

command in a loop structure; an example of this is given in the demonstration program called DEMO_D.PPL. The next 3 arguments refer to the frame address, the number of points in the frame and the number clock cycles to wait between output of successive points. This function makes use of the waveform file alias structure (see Chapter 7). The first frame used is called "grad_pulse". This is a gradient trapezoid of ramp times 1 ms and plateau time of 1 ms. The frames used in this example contain 1000 points describing a linear ramp from zero to maximum in 1 ms and a linear ramp from maximum to zero. The MR3040 master clock is set at its default setting of 1 μ s per point, hence a 1000 points clocked out at this rate takes 1 ms.

- **Point 3 (DEMO_A.PPL)**

This next output instruction outputs only one point, the 1st point, of the frame "ramp_max_to_zero_1ms" but then waits "tplat1" master clock cycles, i.e., a delay of tplat1 with the gradient output determined by the value of the first point in this frame. The third output function outputs the frame "ramp_max_to_zero_1ms". The final output of these 3 instructions is a gradient trapezoid of 1 ms ramp time and plateau duration of tplat1 ms.

- **Point 4 (DEMO_A.PPL)**

This function creates a matrix with the label aq_mat. The first three pairs of arguments are divisor and multiplier ratios for each gradient. The first ratio refers to the z axis, the second ratio to the y axis and the last ratio to the x axis of the rotated frame of reference, i.e., the logical gradients and not the physical x, y, and z gradient output. The next set of arguments refers to the rotation values relative to the base matrix defined previously. The convention chosen is to s_angle_var for slice selection rotation and p_angle_var for phase encode rotation and r_angle_var for the readout rotation. In this example - 512/1024 is used for the scaling of the read gradient which will be 1/2 the maximum output, so a ratio of 1.0 would give maximum output along this axis. The shape defined in the waveform store is positive but an extra negative sign is introduced to the ratio to give a positive output from the console. This is necessary because the create function must work within the integer range - 32768 to + 32767 and as such introduces an extra negative sign which a negative ratio removes. In other words a positive waveform file with a negative ratio gives a positive output.

- **Point 5 (DEMO_A.PPL)**

The function call MR3040_SetList(read_list, CHANNEL_R) assigns the set of instructions defined by the label read_list to the CHANNEL_R. This list could also be assigned to the other channels or all three channels. The next function call MR3040_SelectMatrix(aq_mat) makes the rotation matrix defined by "aq_mat" as the

active matrix to perform the required gradient rotations. Recalculations on this matrix are not possible once it has been selected as the active matrix.

- **Point 6 (DEMO_A.PPL)**

The MR3040_Start(CHANNEL_R) instructs the sequencer to start executing the set of instructions assigned to this channel. The instruction takes 2.6 μ s to execute before control returns back to the PPL program. A delay of 4*tramp and both plateau times is then executed so that the acquisition takes place immediately after output of the two gradient pulses.

7.1.1.2 DEMO_B.PPL

/*****

Sequence Name: DEMO_B.PPL

Author: MR Solutions Ltd.

Date: 08 February 2011

Note: This is the PPL DEMO file for testing all RF units.

This gradient controller demo program is a very simple example of the use of the gradient controller board. Two gradient pulses are output along the read channel X axis. These illustrate the use of the MR3040_Output function which is the main function for using the gradient controller board. Two gradient pulses are applied along the X axis of duration tplat1 and tplat2. The ramp times used are 1ms in duration. The RF blanking line may be used as an external trigger for an oscilloscope. This program also illustrates the use of the MR3040_Hold and MR3040_Continue function. The duration of the first pulse can be altered dynamically in setup mode.

PLEASE NOTE THIS DEMO SEQUENCE IS ONLY INTENDED AS A TEACHING AID TO MR3040 PROGRAMMING AND AS SUCH SHOULD ONLY BE USED WITH THE RF AND GRADIENT HARDWARE TURNED OFF.

*****/

```
#use RF1 "c:\smis\seqlib\rfstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad
/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
SPECTRAL_WIDTH 5000, 50000, 25000, sample_period ;
GRADIENT_STRENGTH grad_var;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT "Repetition time","ms","%d",10,2000,600,1,tr;
EDITTEXT "First pulse duration","us","%d",15,4000,500,1,tplat1;
EDITTEXT "Second pulse duration","us","%d",15,32000,500,1,tplat2;
NO_AVERAGES 1, 10000, 2, no_averages ;
```



```
DSP_ROUTINE "c:\smis\dsp";  
DISCARD 0, 10, 0, no_discard ;  
DATA_TYPE 0x13;  
END
```

```
*/
```

```
#include "stdfn_15.pph"
```

```
#include "std3040.pph"
```

```
main(){  
  const tramp          1000 ;          \\ ramp time (in us) at each gradient.  
  
  /* constants */  
  const aq_mat          3;  
  const calc_mat        255;           \\ Select an unused matrix during matrix  
  calculations  
  const caldelay        100;           \\ Give DSP 100us to finish new matrix  
  const CHANNEL_R       0x0002;        \\ For use in MR3040_Start etc.  
  const CHANNEL_P       0x0020;  
  const CHANNEL_S       0x0200;  
  const CHANNEL_PS      0x0220;  
  const ALL_CHANNELS    0x0222;  
  const NOLOOP          0;  
  const STARTLOOP       4;  
  const COUNTLOOP      8;  
  const JUMPBACK        12;  
  const max_waits       4095;  
  
  \\ looping control  
  int no_averages, completed_av, sample_period, no_samples, no_discard, phase_cycle, no_acq,  
  obs_power;  
  
  \\ timing variables  
  int tplat2;  
  common int tr, tplat1;  
  
  \\ frequency control  
  common int MHz, kHz, Hz, rx1MHz, rec_freq ;  
  
  \\ Variables for gradient timings  
  int ret, grad_var[4], slice_list, read_list, read_lf1;  
  
  \\ orientation variables  
  common int phase_var, s_angle_var, p_angle_var, r_angle_var;  
  ret = MR3040_CreateBaseMatrix( 0, 0, 0, grad_var[3], grad_var[0], grad_var[2],  
                                grad_var[0], grad_var[1], grad_var[0] );  
  if( ret ) {  
    printf( "Could not create base matrix, status = %d\n", ret);  
    goto end;  
  }  
  delay(caldelay,us);  
  MR3040_SetListAddress(0);  
  read_list = MR3040_InitList();  
  MR3040_Output( NOLOOP, grad.address."grad_pulse", grad.size." grad_pulse ", grad.waits."  
grad_pulse ");  
  MR3040_Output( NOLOOP, grad.address."ramp_zero_to_max_1ms",  
grad.size."ramp_zero_to_max_1ms",
```

```
grad.waits."ramp_zero_to_max_1ms");
```

Point 1:

```
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."ramp_max_to_zero_1ms",
grad.size."ramp_max_to_zero_1ms",
grad.waits."ramp_max_to_zero_1ms");
sync(); \\ Synchronise to 100kHz.

\\ ***** MAIN ACQUISITION LOOP *****
\\ Looping structure follows that demanded by SCAN.
completed_av = 0; \\counter for averages loop
averages_loop:

/* Calculations supporting real-time adjustments. Check for overflow */
MR3040_SelectMatrix( calc_mat ); /* Select unused matrix during calcs */
ret = MR3040_CreateMatrix( aq_mat,1024, 0,1024, 0,1024, -512, phase_var, s_angle_var,
p_angle_var, r_angle_var );
if ( ret ) {
printf( "Failed to create aq matrix, return code = %d\n", ret );
goto end;
}
delay( caldelay);
hostrequest();
resync();

\\ ***** SEQUENCE STARTS HERE *****
MR3040_SetList(read_list, CHANNEL_R );
MR3040_SelectMatrix( aq_mat );

MR3031_setup(pf1,"rf_analog_on",0,0,0,2000); /* Output a RF pulse so RF blanking can be used as
a trigger */
rfampon(0);
delay(20, us);
rfon(1);
MR3031_go();
delay(100, us);
rfoff();
MR3031_setup(pf1,"rf_off",0,0,0,0);
MR3031_go();
MR3040_Start(CHANNEL_R); /* Start 3040 channel_R */
delay(1000,us);
delay(tramp, us);
delay(tramp,us);
delay(tplat2,us);
```

Point 2:

```
MR3040_Continue(CHANNEL_R);
delay(tramp,us);
acquire( sample_period, no_samples );

delay(tr,ms); /* Wait for tr */

completed_av=completed_av+1;
```

```

        if(completed_av < no_averages)
            goto averages_loop;
    end:
}

```

The MR3040_Hold function at **Point 1** instructs the sequencer to maintain the gradient output level at the current level until it is released with the MR3040_Continue function, issued at **Point 2**. In this program the duration of the first plateau (tplat1) is controlled via the use of these two functions. The advantage of this is that tplat1 is defined as a common int and can be interactively changed when scan is in setup mode.

7.1.1.3 DEMO_C.PPL

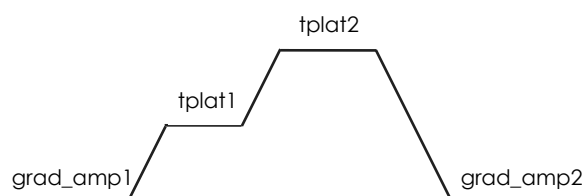
/*****

```

Sequence Name:      DEMO_C.PPL
Author:             MR Solutions Ltd.
Date:              08 February 2011
Note:              This is the PPL DEMO file for testing all RF units.

```

This demo program outputs a 2 level gradient pulse along the read channel (as shown):



The times of both plateaus are changeable in setup mode, as are both gradient amplitudes (defined in the paramlist header). This shape is achieved by the addition of primary and secondary matrices. The ramp time is also changeable in real time while in setup mode via changes to the master clock.

PLEASE NOTE THIS DEMO SEQUENCE IS ONLY INTENDED AS A TEACHING AID TO MR3040 PROGRAMMING AND AS SUCH SHOULD ONLY BE USED WITH THE RF AND GRADIENT HARDWARE TURNED OFF.

*****/

```

#use RF1 "c:\smis\seqlib\rfstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad
/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
SPECTRAL_WIDTH 5000, 50000, 25000, sample_period ;
GRADIENT_STRENGTH grad_var;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT " Repetition time","ms","%d",10,2000,600,1,tr;

```



```

EDITTEXT " 1st Plateau time","us","%d",15,2000,500,1,tplat1;
EDITTEXT " 2nd Plateau time","us","%d",15,2000,500,1,tplat2;
EDITTEXT " 1st grad amplitude ","(0 - 1024)","%d",-1024,1024,-512,1,grad_amp1;
EDITTEXT " 2nd grad amplitude ","(0 - 1024)","%d",-1024,1024,-1024,1,grad_amp2;
EDITTEXT " Ramp time","us","%d",100,1000,1000,1,ramp;
NO_AVERAGES 1, 10000, 100, no_averages ;
DSP_ROUTINE "c:\smis\dsp" ;
DISCARD 0, 10, 0, no_discard ;
DATA_TYPE 0x13;
END
*/
#include "stdfn_15.pph"
#include "std3040.pph"

main(){
/* constants */

```

Point 1:

```

const aq_mat          3;
const aq_sec          259;          \\ matrix for 2nd channel. primary_mat+256 =259.
const calc_mat        255;          \\ Select an unused matrix during matrix calculations
const caldelay        100;          \\ Give DSP 100us to finish new matrix
const CHANNEL_R       0x0002;      \\ For use in MR3040_Start etc
const CHANNEL_P       0x0020;
const CHANNEL_S       0x0200;
const CHANNEL_PS      0x0220;
const ALL_CHANNELS    0x0222;
const NOLOOP          0;
const STARTLOOP       4;
const COUNTLOOP       8;
const JUMPBACK        12;
const max_waits       4095;

\\ looping control
int no_averages, completed_av, sample_period, no_samples, no_discard, phase_cycle, no_acq,
obs_power;
\\ timing variables
int clock;
common int tr, tplat1,tplat2;
\\ frequency control
common int MHz, kHz, Hz, rx1MHz, rec_freq ;
\\ Variables for gradient timings
int ret, grad_var[4], slice_list, read_list, read_lt1;
common int ramp, grad_amp1,grad_amp2;
\\ orientation variables
common int phase_var, s_angle_var, p_angle_var, r_angle_var;
/*****
ret = MR3040_CreateBaseMatrix( 0, 0, 0, grad_var[3],grad_var[0],grad_var[2],
grad_var[0],grad_var[1],grad_var[0] );
if( ret )
{
printf( "Could not create base matrix, status = %d\n", ret);
goto end;
}
delay(caldelay,us);
MR3040_SetListAddress(0);

```

```
read_list = MR3040_InitList();
```

Point 2:

```
MR3040_Output( NOLOOP, grad.address."0_max", grad.size."0_max", grad.waits."0_max");
MR3040_Output( NOLOOP, grad.address."0_max",
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."max_p_s", grad.size."max_p_s", grad.waits."max_p_s");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."mx_0_sec", grad.size."mx_0_sec",
grad.waits."mx_0_sec");

sync();          \\ Synchronise to 100kHz.
\\ ***** MAIN ACQUISITION LOOP *****
\\ Looping structure follows that demanded by SCAN.
    completed_av = 0; \\ counter for averages loop
    averages_loop:
/* Calculations supporting real-time adjustments. Check for overflow */
MR3040_SelectMatrix( calc_mat ); /* Select unused matrix during calcs */
```

Point 3:

```
\\ DEFINE THE PRIMARY MATRIX
    ret = MR3040_CreateMatrix( aq_mat,1024, 0,1024, 0,1024, grad_amp1,    phase_var,
s_angle_var, p_angle_var, r_angle_var );
    if ( ret ) {
        printf( "Failed to create aq matrix, return code = %d\n", ret );
        goto end;
    }
    delay( caldelay, us );
\\ DEFINE THE SECONDARY MATRIX
ret = MR3040_CreateMatrix( aq_sec,1024, 0,1024, 0,1024, grad_amp2, phase_var, s_angle_var,
p_angle_var, r_angle_var );
    if ( ret ) {
        printf( "Failed to create aq matrix, return code = %d\n", ret );
        goto end;
    }
    delay( caldelay, us );
```

Point 4:

```
clock = ramp/10 ;    /* 100 usec is the min tramp which sets the clock to 10, i.e., 10*100ns = 1 µs
*/
    MR3040_Clock(clock);
    hostrequest();
    resync();

\\ ***** SEQUENCE STARTS HERE *****
    MR3031_setup(pf1,"rf_analog_on",0,0,0,2000); /*Output a RF pulse so RF blanking can be
used as a trigger */

    rfampon(0);
    delay(20, us);
```

```
rfoff(1);  
MR3031_go();  
delay(100, us);  
rfoff();  
MR3031_setup(pf1,"rf_off",0,0,0,0);  
MR3031_go();  
    MR3040_SetList(read_list, CHANNEL_R);  
    MR3040_SelectMatrix( aq_mat);
```

Point 5:

```
MR3040_Start(CHANNEL_R);  
ret= tplat1+ramp;  
delay(ret,us);  
MR3040_continue(CHANNEL_R);  
ret = tplat2+ramp;  
delay(ret,us);  
MR3040_continue(CHANNEL_R);  
acquire( sample_period, no_samples );  
  
delay(tr,ms); /* Wait for tr */  
completed_av=completed_av+1;  
if(completed_av < no_averages)  
    goto averages_loop;  
  
end:  
}
```

- **Point 1 (DEMO_C.PPL)**

Primary matrices are identified within the range 0 ~ 255 and the associated secondary matrices are defined within the range 256 ~ 511. In this example the primary matrix "aq_mat" has the value of 3 and its associated secondary matrix "aq_sec" has the value 259, i.e. 3 + 256.

- **Point 2 (DEMO_C.PPL)**

The three waveform frames used in this program each have 100 points set at 1 μ s per point. The frame "0_max" is a linear ramp from zero to full maximum in the primary channel. The frame "max_p_s" contains a linear ramp from full maximum to zero in the primary channel and a ramp from zero to full maximum in the secondary channel. The frame "mx_0_sec" contains a linear ramp from maximum to zero in the secondary channel.

- **Point 3 (DEMO_C.PPL)**

Two calls to the create matrix functions are made one for the primary rotation matrix (aq_mat) and one for the secondary rotation matrix (aq_sec).

- **Point 4 (DEMO_C.PPL)**

The MR3040 master clock is set via the function MR3040_Clock(). The minimum setting of the clock is 1 μ s per point with a resolution of 100 ns. The minimum setting is 10, i.e., 10×100 ns = 1 μ s. In this program the ramp time is divided by 10 so if for example the ramp time selected was 150 μ s the clock setting would be 15, i.e. 1.5 μ s/point which for a frame 100 points has a total duration of 150 μ s.

- **Point 5 (DEMO_C.PPL)**

The matrix selected is the primary matrix aq_mat. However when the primary matrix is selected as the active matrix the associated secondary matrix is also selected to be active on the secondary channel. If the secondary matrix were selected as the active matrix then the associated primary matrix would be selected by default in the same manner for the primary channel.

7.1.1.4 DEMO_D.ppl

/*****

Sequence Name: DEMO_D.PPL

Author: MR Solutions Ltd.

Date: 08 February 2011

Note: This is the PPL DEMO file for testing all RF units.

This gradient controller demo program is a very simple example of the use of the MR3040 board. This demo is example of a loop structure call with the MR3040_Output function. A gradient pulse is output along the read channel n times, where n is the number of required loops. The RF blanking line may be used as an external trigger for an oscilloscope.

PLEASE NOTE THIS DEMO SEQUENCE IS ONLY INTENDED AS A TEACHING AID TO MR3040 PROGRAMMING AND AS SUCH SHOULD ONLY BE USED WITH THE RF AND GRADIENT HARDWARE TURNED OFF.

*****/

```
#use RF1 "c:\smis\seqlib\rfstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad
/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
SPECTRAL_WIDTH 1000, 400000, 25000, sample_period ;
GRADIENT_STRENGTH grad_var;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT " Repetition time","ms","%d",10,2000,600,1,tr;
EDITTEXT " First pulse duration","us","%d",15,4000,500,1,tplat1;
EDITTEXT " Number of loops =",",","%d",1,200,10,1,n_loop;
```



```
NO_AVERAGES 1, 10000, 2, no_averages ;
DSP_ROUTINE "c:\smis\dsp" ;
DISCARD 0, 10, 0, no_discard ;
DATA_TYPE 0x13;
END
*/
#include "stdfn_15.pph"
#include "std3040.pph"

main(){
/* constants */
const tramp                1000 ;  \\ ramp time (in us) at each gradient.
const aq_mat               3;
const calc_mat             255;      \\ Select an unused matrix during matrix
calculations
const caldelay             100;      \\ Give DSP 100us to finish new matrix
const CHANNEL_R            0x0002;  \\ For use in MR3040_Start etc.
const CHANNEL_P            0x0020;
const CHANNEL_S            0x0200;
const CHANNEL_PS           0x0220;
const ALL_CHANNELS         0x0222;
const NOLOOP               0;
const STARTLOOP            4;
const COUNTLOOP            8;
const JUMPBACK             12;
const max_waits            4095;

\\ looping control
int no_averages, completed_av, sample_period, no_samples, no_discard, phase_cycle, no_acq,
obs_power;

\\ timing variables
int tplat1, tplat2;
common int tr;

\\ frequency control
common int MHz, kHz, Hz, rx1MHz, rec_freq ;

\\ Variables for gradient timings
int ret, grad_var[4], n_loop, slice_list, read_list, read_lt1;

\\ orientation variables
common int phase_var, s_angle_var, p_angle_var, r_angle_var;

/*****/
ret = MR3040_CreateBaseMatrix( 0, 0, 0, grad_var[3], grad_var[0], grad_var[2],
grad_var[0], grad_var[1], grad_var[0] );
if( ret ) {
    printf( "Could not create base matrix, status = %d\n", ret);
    goto end;
}
delay(caldelay,us);
MR3040_SetListAddress(0)
read_list = MR3040_InitList();
```

Point 1:



```

MR3040_LoopCount( n_loop - 1 );
MR3040_Output( STARTLOOP, grad.address."ramp_zero_to_max_1ms",
grad.size."ramp_zero_to_max_1ms", grad.waits."ramp_zero_to_max_1ms");
MR3040_Output( NOLOOP, grad.address."ramp_max_to_zero_1ms",1, tplat1);
MR3040_Output( COUNTLOOP, grad.address."ramp_max_to_zero_1ms",
grad.size."ramp_max_to_zero_1ms",
grad.waits."ramp_max_to_zero_1ms");
sync(); \\ Synchronise to 100kHz.
\\ ***** MAIN ACQUISITION LOOP *****
\\ Looping structure follows that demanded by SCAN.
completed_av = 0; \\counter for averages loop
averages_loop:
/* Calculations supporting real-time adjustments. Check for overflow */
MR3040_SelectMatrix(calc_mat); /* Select unused matrix during calcs */
\\ DEFINE THE PRIMARY MATRIX
ret = MR3040_CreateMatrix( aq_mat, 1024, 0, 1024, 0, 1024, -512, phase_var, s_angle_var,
p_angle_var, r_angle_var);
if ( ret ) {
printf( "Failed to create aq matrix, return code = %d\n", ret );
goto end;
}
delay(caldelay, us);
hostrequest();
resync();

\\ ***** SEQUENCE STARTS HERE *****
MR3040_SetList(read_list, CHANNEL_R);
MR3040_SelectMatrix( aq_mat );
MR3031_setup(pf1,"rf_analog_on",0,0,0,2000); /*Output a RF pulse so that RF blanking can
be used as a trigger */
rfampon(0);
delay(20, us);
rfon(1);
MR3031_go();
delay(100, us);
rfoff();
MR3031_setup(pf1,"rf_off",0,0,0,0);
MR3031_go();
MR3040_Start(CHANNEL_R); /* Start 3040 channel _R */
ret= tplat1 + tplat2 + (tramp*4);
delay(ret,us);

```

Point 2:

```

acquire( sample_period, no_samples );
delay(tr,ms); /* Wait for tr */
completed_av=completed_av+1;
if(completed_av < no_averages)
goto averages_loop;
end:
}

```

• Point 1 (DEMO_D.PPL)

The number of loops "n_loop" is used to define the loop count with the function MR3040_Loopcount. The first output instruction is the start of the loop so the first argument is set to "STARTLOOP". The last MR3030_Output function does the loop count decrement and has a conditional return to the start of the loop so the argument "COUNTLOOP" is used. If an unconditional return was required then the argument "RETURN" would have been used thereby forcing a return to the start of the loop irrespective of the value of the loop count.

- **Point 2 (DEMO_D.PPL)**

The acquisition starts immediately after the gradient starts and then a delay of TR is executed. This minimum TR allowed is greater than the combination of maximum plateau duration and maximum number of loops.

7.1.1.5 GE_DEMO.PPL

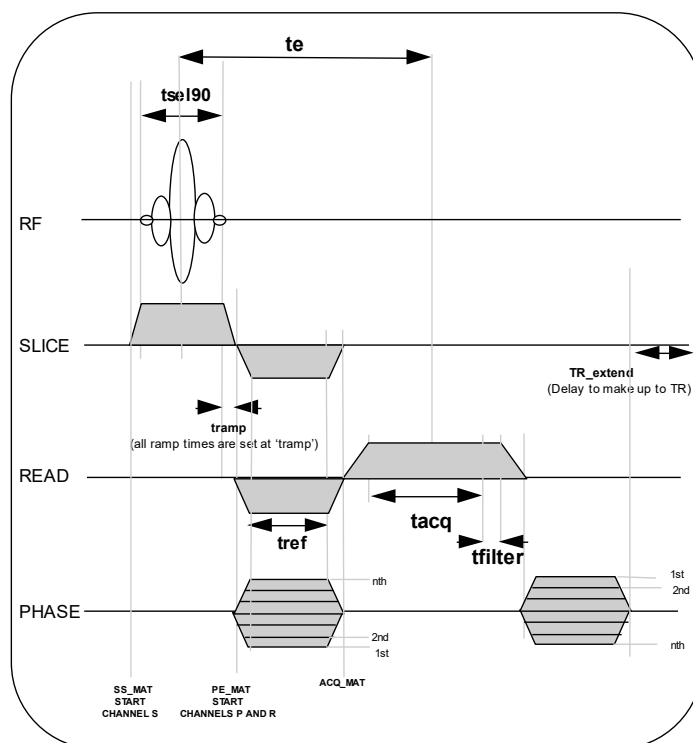


Fig 2: Timing diagram for GE_DEMO.PPL (4.5.6.5) and GEDEM_MC (4.5.6.6)

/*****

Sequence Name: GE_DEMO.PPL

Author: MR Solutions Ltd.

Date: 08 February 2011

Note: This is the PPL DEMO file for testing all RF units.

This is a very simple demonstration of the gradient echo sequence. The echo time is minimised by the selected bandwidth and spectral width. A 5 lobe sinc RF pulse (1500 Hz bandwidth) is used.

*****/

```
#use RF1 "c:\smis\seqlib\rfstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad

\\ Slice separation is centre to centre NOT slice gap.

/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
EDITTEXT "Receive Frequency ", "Hz", "%i", -32767, 32767, 0, 1, rec_freq;
SPECTRAL_WIDTH 1000, 400000, 25000, sample_period ;

Point 1:
GRADIENT_STRENGTH grad_var;
SLICE_THICKNESS 32767, 1500, gs_var ;
SLICE_SEPARATION -6000, 6000, 1500, slice_offset ;
NO_SLICES 1, 128, 1, no_slices ;
SLICE_INTERLEAVE 1, 3, 2, slice_interleave ;
EDITTEXT "Slice offset frequency ", "Hz", "%i", -32768, 32767, 0, 1, fov_slice_freq;
FOV 32767, 32767, 2.0e-3, gr_var, gp_init_var ;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
SCROLLBAR "Obs Mod. power level ", "(0 = off)", "%u", 0, 4, 1, 1, obs_mod_level;
SCROLLBAR "RF Pulse multiplier ", "%", "%.2f", 0, 2047, 512, 20.48, p90_mul ;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT " Repetition time", "ms", "%d", 1, 2000, 100, 1, tr;
EDITTEXT " Ramp time (multiple of 10 only)", "us", "%d", 100, 1000, 1000, 1, tramp;
NO_VIEWS 16, 512, 256, no_views ;
NO_AVERAGES 1, 10000, 2, no_averages ;
VIEW_BLOCK 1, 256, 2, view_block;
SLICE_BLOCK 1, 8, 1, slice_block;
PHASE_CYCLE 1, 2, 2, phase_cycle ;
SCROLLBAR "Read gradient compensation ", "DAC UNITS", "%d", -5000, 5000, 0, 1, gr_comp_scale ;
SCROLLBAR "Slice gradient compensation ", "DAC UNITS", "%d", -2000, 2000, 0, 1, gs_comp_scale ;
SCROLLBAR "Slice gradient ", "off/on", "%u", 0, 1, 1, 1, gs_on ;
SCROLLBAR "Read gradient ", "off/on", "%u", 0, 1, 1, 1, gr_on ;
SCROLLBAR "Phase gradient ", "off/on", "%u", 0, 1, 1, 1, gp_on ;
DSP_ROUTINE "c:\smis\dsp" ;
DISCARD 0, 10, 0, no_discard ;
DATA_TYPE 0x13;
END
*/
#include "stdfn_15.pph"
#include "std3040.pph"

main(){
```

Point 2:

```
/* constants */
const aq_mat      3;
const ss_mat      1;
const pe_mat      2;
```



```

const refocus_mat      4;
const calc_mat         255;          \\ Select an unused matrix during matrix calculations
const caldelay         100;          \\ Give DSP 100us to finish new matrix
const CHANNEL_R        0x0002;      \\ For use in MR3040_Start etc
const CHANNEL_P        0x0020;
const CHANNEL_S        0x0200;
const CHANNEL_PS       0x0220;
const ALL_CHANNELS     0x0222;
const NOLOOP           0;
const STARTLOOP        4;
const COUNTLOOP        8;
const JUMPBACK         12;
const max_waits        4095;
#include "var_15.pph"
discard(no_discard);
phase_increment(400);
tsel90 = 4000;          \\ Length of 90 pulse
/* The gradient strengths returned from PARSETUP from the FOV and SLICE THICKNESS keywords
are negative (gs_var, gr_var and gp_init_var). The phase encode time is set to 1500 usecs */
tacq = sample_period;
tacq = (tacq * (no_samples + no_discard))/10;          \\ acquire time in us.
tacq_2 = tacq/2;
tref = 1500 ;          /* Phase encode time set to 1500 us */
/* Calculate the phase encode increment(gp_inc). gp_init_var is the initial value passed over
from PARSETUP based on an effective phase encode time of 2 ms. Rescale for ramp time
selected. This avoids the gp_init_var being reset by PARSETUP to its initial value once the total
data has been collected when SCAN is in setup mode and the experiment is running repeatedly.
*/

```

Point 3:

```

templ1 = gp_init_var;
templ1 = templ1*2000L;          \\ templ1 is a temporary long for complicated calculations.
templ1 = templ1/(tref+tramp);
ret = dacmaxlong + templ1;
if (ret < 0 ){
    printf("FOV too small in phase encode direction");
    goto end;
}
gp_init_var_rescale = templ1;
gp_inc = scale(templ1, 2,no_views);          \\ gp_inc is now -ve

/* Calculate strengths of the slice and read refocussing lobes GR_COMP is the amplitude of the
read preparation pulse and GS_COMP is the amplitude of the slice refocussing pulse*/

templ1 = tacq_2 + (tramp/2);
templ2 = tref+tramp;
templ3 = gr_var;
/* If small bandwidths are used tacq_2 could be > 32k and the scale function will not work if one
argument is greater than 32k */

if (templ1 > dacmaxlong ) {
    templ3 = (templ3 * templ1)/ templ2;
}
else templ3 = scale( templ3, templ1 ,templ2);
ret = dacmaxlong + templ3;

```

```

if (ret < 0 ) {
    printf(" FOV too small for read preparation pulse ");
    goto end;
}
gr_comp = templ3;
templ1 = (tsel90 +tramp)/2;
templ2 = tref + tramp;
templ1 = gs_var * templ1 / templ2;
ret = dacmaxlong + templ1;
if (ret < 0 ){
    printf(" Slice refocussing lobe too large !\n");
    goto end;
}
gs_comp = templ1;

/* Report echo time for the parameters selected */

templ1 = tsel90/2 + 4*tramp +tacq_2;
printf(" The echo time of this sequence is : %d ms \n",templ1/1000);

/* Create the base matrix */

```

Point 4:

```

ret = MR3040_CreateBaseMatrix( 0, 0, 0,
grad_var[3],grad_var[0],grad_var[2],grad_var[0],grad_var[1],grad_var[0] );
if( ret ) {
    printf( "Could not create base matrix, status = %d\n", ret);
    goto end;
}
delay(100,us);

/* Create gradient lists we will be using for the slice, read and phase profiles. */

clock = tramp/10 ;           /* 100 usec is the min tramp which sets the clock to 10 ie 10*100ns =
1usec */

```

Point 5:

```
MR3040_Clock(clock);
```

Point 6:

```
MR3040_SetListAddress(0);
```

Point 7:

```

slice_list = MR3040_InitList();
MR3040_Output(NOLOOP,grad.address."0_max",grad.size."0_max",grad.waits."0_max");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."max_0",grad.size."max_0",grad.waits."max_0");
MR3040_Output(NOLOOP,grad.address."0_min",grad.size."0_min",grad.waits."0_min");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."min_0",grad.size."min_0",grad.waits."min_0");

```



```
read_list = MR3040_InitList();
MR3040_Output(NOLOOP,grad.address."0_min",grad.size."0_min",grad.waits."0_min");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."min_0",grad.size."min_0",grad.waits."min_0");
MR3040_Output(NOLOOP,grad.address."0_max",grad.size."0_max",grad.waits."0_max");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."max_0",grad.size."max_0",grad.waits."max_0");
```

```
phase_list = MR3040_InitList();
MR3040_Output(NOLOOP,grad.address."0_max",grad.size."0_max",grad.waits."0_max");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."max_0",grad.size."max_0",grad.waits."max_0");
```

```
refocus_list = MR3040_InitList();
MR3040_Output(NOLOOP,grad.address."0_max",grad.size."0_max",grad.waits."0_max");
MR3040_Hold(NOLOOP);
MR3040_Output( NOLOOP, grad.address."max_0",grad.size."max_0",grad.waits."max_0");
/* Set up frequency list for multislicing */
frequency_buffer(0);
frequency(MHz, kHz, Hz, rx1MHz);
multislice(slice_interleave, slice_offset, no_slices);
offset_frequency(fov_slice_freq);
sync(); /* Synchronise to 100kHz.*/
```

```
/****** MAIN ACQUISITION LOOP *****/
```

```
image_av = 0; //counter for averages loop
averages_loop:
    gp_var=gp_init_var_rescale; // Reset phase encode value for next image.
    current_view = 0; // Counter for P.E. (views) loop.
```

```
phase_encode_loop:
    current_view = current_view + 1;
    view_av = 0; // Counter for view block loop.
```

```
view_block_loop:
    current_slice=0; // Counter for multislice loop.
```

```
multislice_loop:
    current_slice=current_slice+1;
    slice_av=0; // Counter for slice block loop.
    no_acq = view_av + image_av; // no. of acq. of a given segment of data. (used to
control phase cycling)
```

```
slice_block_loop:
```

```
/* Calculate the make up time 'tr_extend' for repetition time. And report the echo time for the
parameters selected. */
```

```
tr_min = tsel90 + 8*tramp +2*tref +tacq +tfilter; // the value of 'tfilter' is defined in the variable
header file.
tr_extend = ((tr*1000L/no_slices) - tr_min)/1000L ;
if ( tr_extend < 0 ) {
```



```
printf(" TR too short extend by %d ms\n ",(((tr_min - (tr*1000L/no_slices))*no_slices/1000L) + 1));
    goto end ;
}

/* Calculations supporting real-time adjustments. Check for overflow */

MR3040_SelectMatrix( 255 );                /* Select unused matrix during calcs */
templ3 = gr_comp;
templ3 = templ3 + gr_comp_scale;
templ4 = gs_comp;
templ4 = templ4 + gs_comp_scale;
gr_comp_s = templ3;
gs_comp_s = templ4;
if (templ3 + dacmaxlong < 0) {
    printf("Reduce Read compensation\n");
    goto end;
}

if (templ4 + dacmaxlong < 0) {
    printf("Reduce Slice compensation\n");
    goto end;
}
```

Point 8:

```
MR3040_CreateMatrix( ss_mat, dacmax, gs_on*gs_var, dacmax, 0 ,dacmax, gr_on*gr_var,
    phase_var, s_angle_var, p_angle_var, r_angle_var );
delay(100,us);
MR3040_CreateMatrix( pe_mat, dacmax, gs_on*gs_comp_s, dacmax, -gp_on*gp_var ,dacmax,
gr_on*gr_comp_s,
    phase_var, s_angle_var, p_angle_var, r_angle_var );
delay(100,us);
MR3040_CreateMatrix( aq_mat, dacmax, 0, dacmax, 0 ,dacmax, gr_on*gr_var,
    phase_var, s_angle_var, p_angle_var, r_angle_var );
delay(100,us);
MR3040_CreateMatrix( refocus_mat, dacmax, 0, dacmax, gp_on*gp_var ,dacmax, gr_on*gr_var,
    phase_var, s_angle_var, p_angle_var, r_angle_var );
delay(100,us);
/* Set up receive frequency buffer and select the frequency for the current slice.*/

frequency_buffer(1);
frequency(MHz, kHz, Hz, rx1MHz);
offset_frequency(rec_freq);

frequency_buffer(0);
slice(current_slice);

/* Set phase according to PHASE_CYCLE keyword and acquisition number */

phase(aqphase(no_acq,phase_cycle));
hostrequest();
resync();                \\ Resynchronise to 100 kHz

/***** SEQUENCE STARTS HERE *****/
```

Point 9 :

```
MR3040_SetList( read_list, CHANNEL_R );
MR3040_SetList( phase_list, CHANNEL_P );
MR3040_SetList( slice_list, CHANNEL_S );
```

Point 10:

```
MR3040_SelectMatrix( ss_mat );
```

Point 11:

```
MR3040_Start(CHANNEL_S);
delay(tramp,us);
rfampon(0);
delay(warmup,us);          \\ 'Warmup' defined as 20 usecs in variable header 'var_15.pph'
rfon(obs_mod_level);
MR3031_setup(pf1,"5lobe_sinc_1500Hz",0,0,0,p90_mul);
MR3031_go();
delay(tsel90,us);
rfoff();
```

Point 12:

```
MR3040_Continue(CHANNEL_S);
delay(tramp,us);
MR3040_SelectMatrix( pe_mat );
MR3040_start(CHANNEL_R | CHANNEL_P);
delay(tramp,us);
delay(tref,us);
MR3040_Continue(ALL_CHANNELS);
delay(tramp,us);
```

```
frequency_buffer(1);          /* reset receive frequency for acquisition */
reset_frequency();
```

```
MR3040_SelectMatrix( aq_mat );
delay(tramp,us);
acquire( sample_period, no_samples );
MR3040_Continue(CHANNEL_R);
delay(tramp,us);
```

```
MR3040_SetList( refocus_list, CHANNEL_P );
MR3040_SelectMatrix( refocus_mat );
MR3040_Start( CHANNEL_P );
delay( tramp, us );
delay(tref,us);
MR3040_Continue( CHANNEL_P);
delay( tramp, us);
```

```
tr_extend = (tr/no_slices) - tr_min/1000L - time;          /* Execute extend time to make up to 'tr' */
if ( tr_extend > 0 ) delay(tr_extend,ms);
```

```
no_acq = no_acq + 1;          \\ acquisition number (for phase cycling)
slice_av = slice_av + 1;
if( slice_av < slice_block )          \\ Slice block
goto slice_block_loop;
```

```

if (current_slice < no_slices)                                \\ Multislicing
goto multislice_loop;
view_av = view_av + slice_block;
if (view_av < view_block)                                       \\ View block
goto view_block_loop;

gp_var=gp_var-gp_inc;

if (current_view < no_views)                                    \\ Phase encode
goto phase_encode_loop;

image_av=image_av+view_block;
if (image_av < no_averages)                                     \\ Averages
goto averages_loop;

end:
}

```

- **Point 1 (GE_DEMO.PPL)**

This is the gradient strength keyword that reads the gradient calibration values stored in c:\windows\parfilio.ini and places the result in the variable array 'grad_var' (see 2.2.4.1). These values are used by the PARSETUP application to calculate the gradient strengths in terms of PPL units for the imaging parameters selected. As an example consider the following gradient efficiency values and imaging parameters for this sequence as:

Gx = 600 Hz/mm, Gy = 500 Hz/mm and Gz = 700 Hz/mm.

The minimum value of these gradients, Gmin, is 500 Hz/mm

Spectral width (ΔB) = 25kHz.

Field Of View (FOV) = 300 mm

Number of Views (NV) = 256

Slice Thickness (ST) = 10mm.

The RF pulse bandwidth (RFBW) is 1500 Hz. This is defined in the SLICE THICKNESS keyword in the parameter list header.

The effective phase encode time (PET) is 2 ms. This is defined in the FOV keyword definition in the parameter list header. (Use 0.002 in the calculations below, i.e. seconds)

The values passed back to the variables gs_var, gr_var and gp_init_var (slice, read and phase) are :

$gr_var = (\Delta B / FOV) * (-32768 / Gmin) = - 5461$ PPL units.

$gp_init_var = (NV / (2 * PET * FOV)) * (-32768 / Gmin) = - 5033$ PPL units.

$gs_var = (RFBW / ST) * (-32768 / Gmin) = - 9830$ PPL units.

- **Point 2 (GE_DEMO.PPL)**

This section contains extra constant definitions for MR3040 control not included in the include file std3040.pph. Each gradient channel is identified as follows CHANNEL_R for the read channel, CHANNEL_P for the phase channel and CHANNEL_S for the slice channel. An additional include file 'var_15.pph' is inserted after these MR3040 definitions. This PPH (parameter header) file contains a set of standard definitions for gradient, RF, looping control etc. and is used by all the 3040 pulse sequences in the current release.

- **Point 3 (GE_DEMO.PPL)**

The effective phase-encode time defined in the FOV keyword in the parameter list header is 2 ms. This section re-calculates the phase encode gradient strength based on the ramp time selected.

- **Point 4 (GE_DEMO.PPL)**

The MR3040_CreateBaseMatrix function creates a base rotation matrix to which all subsequent rotations matrices are referred. The gradients are also scaled to be isotropic. The patient orientation may also be defined in this call. See Oblique angle transformations and **point 1** of DEMO_A.PPL.

- **Point 5 (GE_DEMO.PPL)**

The MR3040_Clock call sets the 3040 clock which has a minimum setting of 1 μ s and has a resolution of 100 ns. The waved frames used in this program use 100 points. For example, if the tramp selected was 120 μ s then the clock would be set to 12 i.e. $12 * 100$ ns = 1.2 μ s per point giving a total ramp time of 120 μ s.

- **Point 6 (GE_DEMO.PPL)**

The MR3040_SetAddress function sets the current address pointer to the supplied argument, which in this case is zero. This will be the starting address for all subsequent instructions written to the sequencer. The MR3040_InitList() function creates an address of the next available location in the sequencer memory and can be identified by a label, in this case 'slice_list'. See **point 2** of DEMO_A.PPL

- **Point 7 (GE_DEMO.PPL)**

The MR3040_Output function outputs a waveform from the waveform store. The first argument refers to the looping structure used, which in this case is no looping at all. (As an example of looping structure calls see DEMO_D.PPL). The next three arguments make use of the waveform alias for frame address, number of points in the frame and number of clock cycles per point. This function writes the output instruction to the sequencer.

Once the required sets of instructions have been written and identified by list labels (slice_list, read_list and phase_list), they can be assigned to a logical channel **point 9**. These instructions are then executed when a channel is issued with the start command, MR3040_StartChannel **point 11**. These are defined in c:\smis\include\m3040_15.pph. Compare this to **Point 4** in GEDEM_MC.PPL (below).

- **Point 8 (GE_DEMO.PPL)**

The MR3040_CreateMatrix function creates a rotation matrix identified by the first argument, in this example ss_mat. The next 3 pairs of arguments refer to the divisor and multiplier values for each logical channel (slice, read, phase). The argument phase_var is either 0 for vertical phase encoding or 1 for horizontal phase encoding. The last 3 arguments are the required rotations. This matrix becomes the active matrix once selected by the MR3040_SelectMatrix function **point 10**. These are, in general, defined in c:\smis\include\m3040_15.pph. Compare this to **Point 5** in GEDEM_MC.PPL (below).

- **Point 12 (GE_DEMO.PPL)**

The MR3040_Continue function instructs the sequencer to continue after an MR3040_Hold instruction. See DEMO_B.PPL as an example of this call.

7.1.1.6 GEDEM_MC.PPL

```

/*****

```

```

Sequence Name:      GEDEM_MC.PPL
Author:             MR Solutions Ltd.
Date:              08 February 2011
Note:              This is the PPL DEMO file for testing all RF units.

```

This is a very simple demonstration gradient echo sequence. The echo time is minimised for the bandwidth width and spectral width selected. The actual echo time is reported back to the 3020. A 5 lobe sinc RF pulse (1500 Hz bandwidth) is used.

```

*****/

```

```

#use RF1 "c:\smis\seqlib\rfstd.seq" pf1
#use GRAD "c:\smis\seqlib\g_demo.seq" grad

/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz ;
EDITTEXT "Receive Frequency ", "Hz ", "%i", -32767, 32767, 0, 1, rec_freq;
SPECTRAL_WIDTH 1000, 400000, 25000, sample_period ;
GRADIENT_STRENGTH grad_var;
SLICE_THICKNESS 32767, 1500, gs_var ;

```



```

SLICE_SEPARATION -6000, 6000, 1500, slice_offset ;
NO_SLICES 1, 128, 1, no_slices ;
SLICE_INTERLEAVE 1, 3, 2, slice_interleave ;
EDITTEXT "Slice offset frequency ", "Hz", "%i", -32768, 32767, 0, 1, fov_slice_freq;
FOV 32767, 32767, 2.0e-3, gr_var, gp_init_var ;
OBLIQUE_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
EDITTEXT "RF Pulse Shape = ", "(1-6)", "%d", 1, 6, 5, 1, rfnum;
SCROLLBAR "Obs Mod. power level ", "(0 = off)", "%u", 0, 4, 1, 1, obs_mod_level;
SCROLLBAR "RF Pulse multiplier ", "%", "%.2f", 0, 2047, 512, 20.48, p90_mul ;
NO_SAMPLES 64, 512, 256, no_samples ;
EDITTEXT " Repetition time", "ms", "%d", 1, 2000, 100, 1, tr;
EDITTEXT " Ramp time (multiple of 10 only)", "us", "%d", 100, 1000, 1000, 1, tramp;
NO_VIEWS 16, 512, 256, no_views ;
NO_AVERAGES 1, 10000, 2, no_averages ;
VIEW_BLOCK 1, 256, 2, view_block;
SLICE_BLOCK 1, 8, 1, slice_block;
PHASE_CYCLE 1, 2, 2, phase_cycle ;
SCROLLBAR "Read gradient compensation ", "DAC UNITS ", "%d", -5000, 5000, 0, 1, gr_comp_scale ;
SCROLLBAR "Slice gradient compensation ", "DAC UNITS ", "%d", -2000, 2000, 0, 1, gs_comp_scale ;
SCROLLBAR "Slice gradient ", "off/on", "%u", 0, 1, 1, 1, gs_on ;
SCROLLBAR "Read gradient ", "off/on", "%u", 0, 1, 1, 1, gr_on ;
SCROLLBAR "Phase gradient ", "off/on", "%u", 0, 1, 1, 1, gp_on ;
DSP_ROUTINE "c:\smis\dsp" ;
DISCARD 0, 10, 0, no_discard ;
DATA_TYPE 0x13;
END
*/

```

Point 1:

```

#include "stdfn_15.pph"
#include "var_15.pph"
#include "m3040_15.pph"
#include "m3031_15.pph"

main(){
#include "tstex_15.pph"

```

Point 2:

/* The newshape macro is used now used to assign each pulse to an index number. For example, '5' refers to the 5lobe_sinc_1500Hz, the default. The user only has to select an index number to select any one of the six following pulses. The arguments after each frame name are the pulse length (in usecs) and the pulse width (in Hz) which are written to the arrays 'rf_length' and 'rf_bwdth' respectively. The macro MR3031_RFSTART is now equivalent to the old MR3031_setup command but uses an index number instead of a frame label and is also more efficient in terms of 3020 memory */

```

NEWSHAPE_MAC(1, pf1, "3lobe_sinc_3kHz", 1332, 3000)
NEWSHAPE_MAC(2, pf1, "3lobe_sinc_1500Hz", 2664, 1500)
NEWSHAPE_MAC(3, pf1, "3lobe_sinc_750Hz", 5328, 750)
NEWSHAPE_MAC(4, pf1, "5lobe_sinc_3kHz", 2000, 3000)
NEWSHAPE_MAC(5, pf1, "5lobe_sinc_1500Hz", 4000, 1500)
NEWSHAPE_MAC(6, pf1, "5lobe_sinc_750Hz", 8000, 750)

```

```
discard(no_discard);
```



```
phase_increment(400);
```

```
tsetl90 = rf_length[rflnum];  
in these arrays
```

```
\\ length and bandwidth of pulse selected defined
```

```
pulse_bwdbth = rf_bwdbth[rflnum];
```

```
/* The gradient strengths returned from PARSETUP from the FOV and SLICE THICKNESS keywords  
are negative (gs_var, gr_var and gp_init_var). The phase encode time is set to 1500 usecs */
```

```
tacq = sample_period;
```

```
tacq = (tacq * (no_samples + no_discard))/10;    \\ acquire time in us.
```

```
tacq_2 = tacq/2;
```

```
tref = 1500 ;    /* Phase encode time set to 1500 uss */
```

```
/* Calculate the phase encode increment(gp_inc). gp_init_var is the initial value passed over  
from PARSETUP based on an effective phase encode time of 2 ms. Rescale for ramp time  
selected. This avoids the gp_init_var being reset by PARSETUP to its initial value once the total  
data has been collected when SCAN is in setup mode and the experiment is running repeatedly.  
*/
```

```
templ1 = gp_init_var;
```

```
templ1 = templ1*2000L;
```

```
\\ templ1 is a temporary long for complicated
```

```
calculations.
```

```
templ1 = templ1/(tref+tramp);
```

```
ret = dacmaxlong + templ1;
```

```
if (ret < 0 ){
```

```
    printf("FOV too small in phase encode direction");
```

```
    goto end;
```

```
}
```

```
gp_init_var_rescale = templ1;
```

```
gp_inc = scale(templ1, 2,no_views);    \\ gp_inc is now -ve
```

```
/* Calculate strengths of the slice and read refocussing lobes GR_COMP is the amplitude of the  
read preparation pulse and  
GS_COMP is the amplitude of the slice refocussing pulse*/
```

```
templ1 = tacq_2 + (tramp/2);
```

```
templ2 = tref+tramp;
```

```
templ3 = gr_var;
```

```
/* If small bandwidths are used, tacq_2 could be > 32k. The scale function won't work if one  
argument is greater than 32k */
```

```
if (templ1 > dacmaxlong ) {
```

```
    templ3 = (templ3 * templ1)/ templ2;
```

```
}
```

```
else templ3 = scale( templ3, templ1 ,templ2);
```

```
ret = dacmaxlong + templ3;
```

```
if (ret < 0 ) {
```

```
    printf(" FOV too small for read preparation pulse ");
```

```
    goto end;
```

```
}
```

```
gr_comp = templ3;
```

```
templ1 = (tsetl90 +tramp)/2;
```

```
templ2 = tref + tramp;
```

```
templ1 = gs_var * templ1/ templ2;
```



```
ret = dacmaxlong + templ1;
if (ret < 0 ){
    printf(" Slice refocussing lobe too large !\n");
    goto end;
}
gs_comp = templ1;

/* Report echo time for the parameters selected */

templ1 = tsel90/2 + 4*tramp + tref + tacq_2;
printf(" The echo time of this sequence is : %d ms \n",templ1/1000);
```

Point 3:

```
BASEMATRIX(0, 0 ,0)          /* Calculate Base matrix */
delay(caldelay,us);          /* Matrix calculation delay of 100 us */

/* Create gradient lists we will be using for the slice, read and phase profiles. */

clock = tramp/10 ;           /* 100 usec is the min tramp which sets the clock to 10 ie
10*100ns = 1usec */
MR3040_Clock(clock);

MR3040_SetListAddress(0);
```

Point 4:

```
slice_list = MR3040_InitList();
POSPULSE_HOLD()
NEGPULSE_HOLD()
```

```
read_list = MR3040_InitList();
NEGPULSE_HOLD()
POSPULSE_HOLD()
```

```
phase_list = MR3040_InitList();
POSPULSE_HOLD()
```

```
refocus_list = MR3040_InitList();
POSPULSE_HOLD()
```

```
/* Set up frequency list for multislicing */
```

```
frequency_buffer(0);
frequency(MHz, kHz, Hz, rx1MHz);
multislice(slice_interleave, slice_offset, no_slices);
offset_frequency(fov_slice_freq);
sync();          /* Synchronise to 100kHz.*/
```

```
/****** MAIN ACQUISITION LOOP *****/
```

```
image_av = 0;          \\counter for averages loop
```

```
averages_loop:
```

```
gp_var=gp_init_var_rescale;    \\ Reset phase encode value for next image.
current_view = 0;              \\ Counter for P.E. (views) loop.
```



phase_encode_loop:

```
current_view = current_view + 1;
view_av = 0;                \\ Counter for view block loop.
```

view_block_loop:

```
current_slice=0;            \\ Counter for multislice loop.
```

multislice_loop:

```
current_slice=current_slice+1;

slice_av=0;                \\ Counter for slice block loop.
no_acq = view_av + image_av; \\ no. of acquisition of a given segment of data.
                          \\ (used to control phase cycling)
```

slice_block_loop:

```
/* Calculate the make up time 'tr_extend' for repetition time. And report the echo time for the
parameters selected. */
tr_min = tsel90 + 8*tramp +2*tref +tacq +tfilter;    \\ the value of 'tfilter' is defined in the variable
header file.
```

```
tr_extend = ((tr*1000L/no_slices) - tr_min)/1000L ;
if ( tr_extend < 0 ) {
    printf(" TR too short extend by %d ms\n ",((tr_min - (tr*1000L/no_slices))*no_slices/1000L) +
1));
    goto end ;
}
```

```
/* Calculations supporting real-time adjustments. Check for overflow */
MR3040_SelectMatrix( 255 );          /* Select unused matrix during calcs */
templ3 = gr_comp;
templ3 = templ3 + gr_comp_scale;
templ4 = gs_comp;
templ4 = templ4 + gs_comp_scale;
gr_comp_s = templ3;
gs_comp_s = templ4;
if (templ3 + dacmaxlong < 0) {
    printf("Reduce Read compensation\n");
    goto end;
}
if (templ4 + dacmaxlong < 0) {
    printf("Reduce Slice compensation\n");
    goto end;
}
```

Point 5:

```
CREATE_MATRIX( ss_mat, gs_on*gs_var, 0 , 0 )
delay( caldelay, us );
CREATE_MATRIX(aq_mat, 0, 0, gr_on*gr_var)
delay( caldelay, us );
CREATE_MATRIX(pe_mat, gs_on*gs_comp_s, -gp_on*gp_var,gr_on*gr_comp_s)
delay( caldelay, us);
CREATE_MATRIX( refocus_mat, 0, gp_on*gp_var , gr_on*gr_var )
```



```
delay(caldelay,us);

/* Set up receive frequency buffer and select the frequency for the current slice.*/
frequency_buffer(1);
frequency(MHz, kHz, Hz, rx1MHz);
offset_frequency(rec_freq);

frequency_buffer(0);
slice(current_slice);

/* Set phase according to PHASE_CYCLE keyword and acquisition number */

phase(aqphase(no_acq,phase_cycle));

hostrequest();
resync();                \\ Resynchronise to 100 kHz

/***** SEQUENCE STARTS HERE *****/

MR3040_SetList( read_list, CHANNEL_R );
MR3040_SetList( phase_list, CHANNEL_P );
MR3040_SetList( slice_list, CHANNEL_S );
MR3040_SelectMatrix( ss_mat );
MR3040_Start(CHANNEL_S);
MR3031_RFSTART(rfnum,pf1,tse190,p90_mul,tramp)
MR3040_Continue(CHANNEL_S);
delay(tramp,us);
MR3040_SelectMatrix( pe_mat );
MR3040_start(CHANNEL_R | CHANNEL_P);
delay(tramp,us);
delay(tref,us);
MR3040_Continue(ALL_CHANNELS);
delay(tramp,us);

/* reset receive frequency for acquisition */
frequency_buffer(1);
reset_frequency();

MR3040_SelectMatrix( aq_mat );
delay(tramp,us);
acquire( sample_period, no_samples );
MR3040_Continue(CHANNEL_R);
delay(tramp,us);

MR3040_SetList( refocus_list, CHANNEL_P );
MR3040_SelectMatrix( refocus_mat );
MR3040_Start( CHANNEL_P );
delay( tramp, us );
delay(tref,us);
MR3040_Continue( CHANNEL_P);
delay( tramp, us);

/* Execute extend time to make up to 'tr' */
tr_extend = (tr/no_slices) - tr_min/1000L - time;
if ( tr_extend > 0 ) delay(tr_extend,ms);
```

```

no_acq = no_acq + 1;           \\ acquisition number (for phase cycling)
slice_av = slice_av + 1;
if( slice_av < slice_block )   \\ Slice block
goto slice_block_loop;
if (current_slice < no_slices)  \\ Multislicing
goto multislice_loop;

view_av = view_av + slice_block;
if (view_av < view_block)      \\ View block
goto view_block_loop;
gp_var=gp_var-gp_inc;

if (current_view < no_views)    \\ Phase encode
goto phase_encode_loop;

image_av=image_av+view_block;
if (image_av < no_averages)     \\ Averages
goto averages_loop;

end:
}

```

- **Point 1 (GEDEM_MC.PPL)**

M3040_15.PPH, TSTEX_15.PPH and M3031_15.PPH are extra “include” files that are not in the original GE_DEMO.PPL program. M3040_15.PPH contains macro definitions for 3040 control. M3031_15.PPH contains macro definitions for 3031 control. TSTEX_15.PPH is several lines of code for testing of allowed combination of no of averages and phase cycling. The full contents description of these files may be found in appendix A (section 5.6).

- **Point 2 (GEDEM_MC.PPL)**

The NEWSHAPE macro is used to assign each RF pulse to an index number. The user simply selects an index number to select a required pulse. If it is required to use different RF pulses not in this list it is a straightforward matter to change the program to user different RF pulses. Simply change the frame label in each NEWSHAPE macro to the desired RF pulse, ensuring that the correct pulse length and pulse bandwidth are supplied (the last two arguments). It is also important to ensure that the frames exist in the RF waveform editor file. To output one of these waveforms the MR3031_RFSTART macro is used (**point 6**). The arguments supplied to this macro are:

- the index number of the RF pulse;
- the alias name for the RF file;
- the length of the pulse;
- the pulse multiplier; and
- a pre delay before output of the pulse.

This macro takes into account turning on/off the RF amplifier and allows a warm-up time defined by the variable warmup in VAR_15.PPH. The macro guarantees that the RF pulse

This macro creates a base matrix. Only the rotations of the base matrix need be specified as the rescaling of the gradients for isotropic output is taken care of automatically.

The POSPULSE_HOLD() macro creates a set of MR3040_Output functions to define a gradient trapezoid with a MR3040_Hold function for the gradient plateau of positive amplitude. The NEGPULSE_HOLD() macro defines a negative gradient trapezoid.

The CREATE_MATRIX macro calls the MR3040_Creatematrix function, however this macro only requires a matrix label, and the amplitude of the logical slice, phase and read gradients. These arguments are the multiplier values in the gradient scaling ratios. The divisors have been set at 32767, i.e. the DAC maximum. This means the gradient values passed back from PARSETUP can be used immediately in this macro.

The diagram illustrates the timing of various signals in a 2D MRI sequence over multiple repetitions (n times). The signals are plotted against time, with key intervals and events labeled:

- RF (Radio Frequency):** Shows two RF pulses. The first pulse is followed by a delay of $te/2$. The second pulse is followed by a delay of $te/2$. The time between the start of the two RF pulses is $tsel180$. The time between the start of the first RF pulse and the start of the first SLICE ramp is $tsel90$.
- SLICE:** Shows two trapezoidal ramps. The first ramp is followed by a delay of $tramp$ (all ramp times are set at $tramp$). The second ramp is followed by a delay of $trush$. The time between the start of the two SLICE ramps is $tsel180$.
- READ:** Shows two trapezoidal ramps. The first ramp is followed by a delay of $tacq$. The second ramp is followed by a delay of $tacq$. The time between the start of the two READ ramps is $tsel180$.
- PHASE:** Shows two trapezoidal ramps. The first ramp is followed by a delay of trf . The second ramp is followed by a delay of trf . The time between the start of the two PHASE ramps is $tsel180$.
- Timing Intervals:**
 - $tsel90$: Time between the start of the first RF pulse and the start of the first SLICE ramp.
 - $te/2$: Time between the end of the first RF pulse and the start of the second RF pulse.
 - $tsel180$: Time between the start of the two RF pulses, SLICE ramps, READ ramps, and PHASE ramps.
 - $tramp$: Time between the end of the first SLICE ramp and the start of the first READ ramp.
 - $trush$: Time between the end of the second SLICE ramp and the start of the second READ ramp.
 - $tacq$: Time between the end of the first READ ramp and the start of the second READ ramp.
 - trf : Time between the end of the first PHASE ramp and the start of the second PHASE ramp.
 - trf_filter : Time between the end of the second PHASE ramp and the start of the second SLICE ramp.
 - TR_extend : Time between the end of the second READ ramp and the start of the second SLICE ramp.

Sequence Name: FSE_PF.PPL

Author: MR Solutions Ltd.

Date: 09 February 2011

Fast Spin Echo Sequence with Driven Equilibrium and Partial Fourier Options.

***** /

```
#use RF1 "c:\smis\seqlib\RFstd.seq" pf1
#use RF1 "c:\smis\seqlib\hsc_1500.seq" pf7
#use RF1 "c:\smis\seqlib\presata.seq" pf8
#use GRAD "c:\smis\seqlib\g3040_15.seq" grad

/* PARAMLIST
OBSERVE_FREQUENCY "1H", -20000, 20000, 0, MHz, kHz, Hz, rx1MHz;
EDITTEXT "Receive Frequency ", "Hz ", "%d", -32000, 32000, 0, 1, rec_freq;
SPECTRAL_WIDTH 1000, 100000, 25000, sample_period;
GRADIENT_STRENGTH grad_var;

NO_SAMPLES 64, 1024, 256, no_samples;
NO_VIEWS 16, 512, 256, no_views;
VIEWS_PER_SEGMENT 2, 64, 4, views_per_seg;
SCROLLBAR "Freq (under/over)sampling ", "under/none/over", "%d", -1, 1, 0, 1, oversample;
SCROLLBAR "Phase (under/over)sampling ", "under/none/over", "%d", -1, 1, 0, 1, oversample2;
SCROLLBAR "Navigator?", "1=yes", "%d", 0, 1, 0, 1, nav_on;

DISCARD 0, 128, 6, no_discard ;
NO_AVERAGES 1, 128, 1, no_averages ;
VIEW_BLOCK 1, 128, 1, view_block;
SLICE_BLOCK 1, 1, 1, slice_block;
PHASE_CYCLE 1, 2, 1, phase_cycle ;
EXPERIMENT_ARRAY 1, 128, 1, no_experiments;

EDITTEXT "Patient X angle", "degrees", "%.1f", -1800, 1800, 0, 10, subj_angle_x;
EDITTEXT "Patient Y angle", "degrees", "%.1f", -900, 900, 0, 10, subj_angle_y;
EDITTEXT "Patient Z angle", "degrees", "%.1f", -1800, 1800, 0, 10, subj_angle_z;

SLICE_THICKNESS 32767, 1070, gs_var ;
BATCH_SLICES 0, 128, 0, batch_slices ;
BATCH_INTERLEAVE 0, 5, 1, batch_interleave ;
SLICE_SEPARATION -6000, 6000, 1070, slice_offset ;
NO_SLICES 1, 128, 1, no_slices ;
SLICE_INTERLEAVE 1, 5, 2, slice_interleave ;
FOV 32767, 32767, 3.0e-3, gr_var, gp_init_var ;
MULTI_ORIENTATION 0, 3, 0, phase_var, r_angle_var, p_angle_var, s_angle_var;
FOV_SLICE_OFF -32000, 32000, 0, 400, fov_slice_off;
FOV_PHASE_OFF -32000, 32000, 0, 4000, fov_phase_off;
FOV_READ_OFF -32000, 32000, 0, 4000, fov_read_off;
```

Point 1:

```
EDITTEXT "RF pulse shape ", "1-6", "%d", 1, 6, 2, 1, rfnum;
SCROLLBAR "Obs. Mod. Level ", "(0 = off)", "%u", 0, 1, 1, 0.25, obs_mod_level;
SCROLLBAR "RF Calibration value ", "%", "%.2f", 0, 2047, 256, 20.48, rfcal ;
SCROLLBAR "90 deg scale factor ", "rfcal", "%.2f", 0, 180, 90, 90, alpha ;
```

```
SCROLLBAR "180 deg scale factor ", "rfcal", "%.2f", 0, 300, 200, 100, p180_scale ;
SCROLLBAR "RF Spoil ", "off/on", "%u", 0, 1, 0, 1, rfsp_on ;
```

Point 2:

```
EDITTEXT "Repetition time between shots", "ms", "%d", 13, 32767, 2000, 1, tr;
EDITTEXT "Base echo time", "ms", "%d", 5, 64, 20, 1, te;
EDITTEXT "Effective echo time", "ms", "%d", 5, 4096, 80, 1, te_2;
EDITTEXT "PF effective number of echoes", "", "%d", 0, 64, 0, 1, PF_echoes;
EDITTEXT "Eddy Current Delay", "us", "%d", 0, 5000, 0, 1, eddy_delay;
SCROLLBAR "Report Min TE and TR times ?", "1=yes", "%d", 0, 1, 0, 1, report_on;
```

Point 3:

```
EDITTEXT "Ramp time", "us, multiple of 10 only", "%d", 100, 1000, 500, 1, tramp;
EDITTEXT "Rephase lobe duration", "us", "%d", 500, 5000, 1500, 1, tref;
```

Point 4:

```
SCROLLBAR "Slice refocus lobe post 180", "%", "%d", 0, 300, 100, 1, gsp_lobe;
SCROLLBAR "Slice refocus adjustment", "%", "%1f", -100, 100, 7, 10, gs_comp_scale;
SCROLLBAR "Read dephase lobe pre 180", "%", "%d", 0, 300, 0, 1, grp_lobe;
SCROLLBAR "Read dephase adjustment", "%", "%1f", -100, 100, 0, 10, gr_comp_scale;
EDITTEXT "180 Crusher", "us", "%d", 10, 5000, 10, 1, tcrush;
SCROLLBAR "Mains gating ON", "1=yes", "%u", 0, 1, 0, 1, mgating;
SCROLLBAR "Slice Frequency ", "Hz ", "%i", -3200, 3200, 0, 0, 1, slice_freq_var;
SCROLLBAR "Driven Equilibrium ON", "1=yes", "%d", 0, 1, 1, 1, de_on;
```

Point 5:

```
SCROLLBAR "Pre-Sat. Pulse ", "off/on", "%u", 0, 1, 0, 1, satgs_on;
SCROLLBAR "Pre-Sat. Scale Factor ", "rfcal", "%.2f", 0, 500, 100, 100, p_sat_scale;
SCROLLBAR "Pre-Sat. Orientation: Phase/Read", "Phase/Read", "%u", 1, 2, 1, 1, sat_orient;
SCROLLBAR "Pre-Sat. Slab Thickness ", "mm", "%d", 20, 500, 50, 1, sat_fov_scale;
EDITTEXT "Pre-Sat. Slab offset ", "mm", "%d", -200, 200, 0, 1, sat_offset;
EDITTEXT "Pre-Sat. Crusher Duration", "us", "%d", 10, 10000, 1000, 1, sat_tcrush;
EDITTEXT "Pre-Sat. Crusher Amplitude", "DAC UNITS", "%d", -32767, 32767, 0, 1, sat_gr_amp;
SCROLLBAR "Pre-Sat. RF pulse shape", "HypSec/Sinc-Cos", "%u", 7, 8, 7, 1, rfnum_sat;
```

Point 6:

```
SCROLLBAR "Slice gradient ON", "1=yes", "%d", 0, 1, 1, 1, gs_on;
SCROLLBAR "Read gradient ON", "1=yes", "%d", 0, 1, 1, 1, gr_on;
SCROLLBAR "Phase gradient ON", "1=yes", "%d", 0, 1, 1, 1, gp_on;
DSP_ROUTINE "c:\smis\dsp";
DATA_TYPE 0x13;
SCROLLBAR "Discard Acquisitions ", "", "%d", 0, 20, 0, 1, no_disacq;
END
*/
```

```
#define MAO
#define MAX_SLICE          128

#include "stdfn_15.pph"
```

```
#include "var_15.pph"
#include "offst_15.pph"
#include "m3040_15.pph"
#include "m3031_15.pph"
```

```
main(){
```

Point 7:

```
\\ gradient variables
int gs_rp, gr_dp, gp_dp, gp_store, gp_var_calc, gp_var_calc_rescale;
int gp_cnt, gp_loc, gp_mul, echo_centre[256], echo_location[256], gp_order[640];
int views_per_echo, array_count, view_shift, PF_echoes;
int slice_list_rp, read_list_dp, phase_list_dp;
int grp_dp, gsp_rp, sl_thickness, sat_thickness;
int fse_gs_comp, fde_gs_comp, fir_gs_comp;
common int grphase_on, grp_lobe, gsp_lobe, slice_freq_var;

\\ timing variables
long te_a, te_b, te_c, sample_period_l;
int te_extend, tdp, tacq_int, del, mgating;
int te_balance_a, te_balance_b, te_balance_c, eddy_delay;

\\ misc
int slice_180_refocus, stim_tcrush, temp, nav_on, nav_cnt, no_views_eff;
int sat_offset, de_on;
int fov_phase_off_sat, fov_read_off_sat, im_FOV;
int disacq_cnt, no_disacq;
int slice_batch_start, no_experiments, completed_ex;

#include "tstex_15.pph"

\\Definitions in addition to var_15.pph
```

Point 8:

```
#define slice_crush          21
#define slice_crush_sec     277
#define fse_mat              23
#define fse_mat_sec         279
#define tacq_phase           16
#define post_90_delay        -278
#define post_180_delay_1     7
#define post_180_delay       -280
#define post_acq_delay       200
```

/* The newshape macro is used now used to assign each pulse to an index number. For example '5' refers to the 5lobe_sinc_1500Hz, the default. The user only has to select an index number to select any one of the six following pulses. The arguments after each frame name are the pulse length (in usecs) and the pulse width (in Hz) which are written to the arrays 'rf_length' and 'rf_bwidth' respectively. The macro MR3031_RFSTART is now equivalent to the old MR3031_setup command but uses an index number instead of a frame label and is also more efficient in terms of 3020 memory */

```
NEWSHAPE_MAC(1, pf1, "3lobe_sinc_3kHz", 1332, 2140)
NEWSHAPE_MAC(2, pf1, "3lobe_sinc_1500Hz", 2664, 1070)
```



```

NEWSHAPE_MAC(3, pf1,"3lobe_sinc_750Hz" , 5328, 535)
NEWSHAPE_MAC(4, pf1,"5lobe_sinc_3kHz" , 2000, 2140)
NEWSHAPE_MAC(5, pf1,"5lobe_sinc_1500Hz" , 4000, 1070)
NEWSHAPE_MAC(6, pf1,"5lobe_sinc_750Hz" , 8000, 535)
NEWSHAPE_MAC(7, pf7,"hypsec_1500Hz" , 10000, 1500)
NEWSHAPE_MAC(8, pf8,"5sinc_7_5cos_6ms" , 6000, 1500)

```

\\ Establish Overhead time for bandwidths selected. This is used in phase angle correction

```
overhead = 264;
```

Point 9:

```
tfilter = acqpad(sample_period)/10;
```

\\ ***** PHASE ENCODE MATRIX CALCULATION *****

Point 10:

\\ Calculate the effective number of views if navigators are used

```

no_views_eff = no_views;
if(nav_on==1) no_views_eff = no_views - views_per_seg;
if (PF_echoes == 0) PF_echoes = views_per_seg;

```

\\ Check that phase encode and echo parameters are compatible

```

if (no_views_eff%(views_per_seg*2) != 0 ) {
    printf("Number of views / (2*views per segment) must be an integer \n");
    if(nav_on==1) printf("Navigator is enabled \n");
    if(nav_on==0) printf("Navigator is disabled \n");
    goto end;
}

```

```

if (PF_echoes < views_per_seg ) {
    printf("PF effective echoes must be >= views per segment \n");
    goto end;
}

```

```

if (PF_echoes >= 2*views_per_seg ) {
    printf("PF effective echoes must be < 2*views per segment \n");
    goto end;
}

```

\\ Check that the base echo time is a multiple of the effective echo time

```

if (te_2%te != 0 ) {
    printf("The base echo time must be a multiple of the effective echo time \n");
    goto end;
}

```

```
te_eff = scale(te_2, 1, te);
```

```

if (te_eff > views_per_seg ) {
    printf("The effective echo time is too large\n");
    goto end;
}

```

\\ Set up the echo_centre array

```
views_per_echo = no_views_eff/(2*views_per_seg);
view_shift = (PF_echoes - views_per_seg) * views_per_echo;
array_count = 0;
echo_centre_loop:
array_count = array_count + 1;

gp_mul = (array_count-te_eff)*views_per_echo;
if(gp_mul < (-no_views_eff/2) - view_shift) gp_mul = gp_mul + no_views_eff;
if(gp_mul >= (no_views_eff/2) - view_shift) gp_mul = gp_mul - no_views_eff;
echo_centre[array_count] = gp_mul;

gp_mul = (1-te_eff-array_count)*views_per_echo;
if(gp_mul < (-no_views_eff/2) - view_shift) gp_mul = gp_mul + no_views_eff;
if(gp_mul >= (no_views_eff/2) - view_shift) gp_mul = gp_mul - no_views_eff;
echo_centre[array_count + views_per_seg] = gp_mul;

if(array_count < views_per_seg) goto echo_centre_loop;
```

\\ Set up the echo_location array

```
array_count = 0;

echo_location_loop:
array_count = array_count + 1;
echo_location[array_count] = (views_per_echo/2) - array_count + 1;
echo_location[array_count+views_per_echo] = echo_location[array_count];
if(array_count < views_per_echo) goto echo_location_loop;
```

\\ Set up the gp_order array

```
array_count = 0;
gp_loc = 0;
gp_store = 0;
if (nav_on) {
nav_loop:
gp_order[array_count] = 0;
array_count = array_count + 1;
if (array_count < views_per_seg) goto nav_loop;
}
gp_order_loop1:
gp_loc = gp_loc + 1;
if (gp_loc > views_per_echo) gp_store = views_per_seg;
gp_cnt = 0;
gp_order_loop2:
gp_cnt = gp_cnt + 1;
gp_mul = echo_centre[gp_cnt + gp_store] + echo_location[gp_loc];
if(gp_mul < (-no_views_eff/2)-view_shift) gp_mul = gp_mul + no_views_eff;
if(gp_mul >= (no_views_eff/2)-view_shift) gp_mul = gp_mul - no_views_eff;
gp_order[array_count] = gp_mul;
array_count = array_count + 1;
if (gp_cnt < views_per_seg) goto gp_order_loop2;
if (gp_loc < 2*views_per_echo) goto gp_order_loop1;
```

\\ ***** SET UP IMAGING PARAMETERS *****

Point 11:

```
discard(no_discard);
phase_increment(1);
deg_90 = scale(90, 1000, phase_res);
deg_360 = scale(360, 1000, phase_res);
sample_period_l = sample_period ;
tacq = (sample_period_l * (no_samples+no_discard))/10;
templ1 = tacq - dacmaxlong;
if (templ1 > 0 ) {
printf("Acquistion time too long");
goto end;
}
tacq_int = tacq;
```

\\ Define the length of the RF pulse and its associated bandwidth

```
tse190 = rf_length[rnum];
tse180 = tse190;
pulse_bwath = rf_bwath[rnum];
tse_sat = rf_length[rnum_sat];
sat_pulse_bwath = rf_bwath[rnum_sat];
tdp = tref;
gr_oversample = 3 + oversample;
gr_undersample = 3 - oversample;
gp_oversample = 3 + oversample2;
gp_undersample = 3 - oversample2;
gp_mul = 0;
current_slice = 1;
if (batch_slices == 0) batch_slices = no_slices;
if (eddy_delay < 10) eddy_delay = 10;
```

/* These are the scale factors defined in the PARAMLIST header which must be a factor of ten for the following macros to work correctly */

```
scale_read_off = scale(4000, gr_oversample, gr_undersample);
scale_slice_off = 400;
scale_phase_off = scale(4000, gp_oversample, gp_undersample);
```

/* gp_var_calc is the initial phase encode value passed from PARSETUP based on an effective phase encode delay of 4 ms. It must be rescaled for the selected ramp time. The value is reassigned to prevent reset by PARSETUP to the initial value during setup */

```
templ1 = gp_init_var;
templ2 = gp_undersample;
templ3 = gp_oversample;
templ1 = (templ1*templ2*3000L)/templ3;
templ1 = templ1/(tdp+tramp);
ret = dacmaxlong + templ1;
if (ret<0) {
printf("FOV too small in phase encode direction");
goto end;
}
gp_inc = scale(templ1,2,no_views);
```

\\ gs_var must also be rescaled for the RF pulse selected.

\\ gs_var was calculated by PARSETUP assuming a 1500Hz bandwidth.

```
templ1 = gs_var;
templ1 = templ1 * pulse_bwdth;
templ1 = templ1/1070L;
ret = dacmaxlong + templ1;
if (ret < 0){
printf("Slice thickness is too small for selected pulse");
goto end;
}
gs_var_rescale = templ1;
```

\\ Switched gradient dephasing lobe calculation

```
templ1 = scale(gr_var, tacq/2 + tramp/2, tdp + tramp);
templ2 = gr_undersample;
templ3 = gr_oversample;
templ4 = (templ1*templ2)/templ3;
ret = dacmaxlong + templ4;
if (ret < 0){
printf("FOV too small for read compensation");
goto end;
}
gr_comp = templ4;
```

\\ Slice gradient refocussing lobe calculation

```
templ1 = (tsel90 + tramp)/2;
templ2 = tref + tramp;
templ1 = gs_var_rescale * templ1/templ2;
ret = dacmaxlong + templ1;
if (ret < 0){
printf("Slice refocussing lobe is excessive\n");
printf("Decrease bandwidth or select a longer pulse\n");
goto end;
}
gs_comp = templ1;
```

\\ Pre-sat calculations

```
templ1 = 1L*sample_period*gr_var;           \\ Calculate FOV
templ2 = 10000000L/grad_var[0];
templ3 = (-32768L*templ2)/templ1;           \\ Image FOV in mm
im_FOV = templ3 + 1L;
```

```
if (sat_orient == 1 && satgs_on == 1) {
sat_channel = 0x0020;           \\ Address of phase channel on 3040 board
templ1 = -32768L*sat_pulse_bwdth; \\ Calculate pre-sat gradient strength
templ2 = sat_fov_scale*1L;
templ2 = templ2*grad_var[0];
templ4 = templ1/templ2;
ret = dacmaxlong + templ4;

if (ret < 0 ) {
printf(" Pre-sat slice thickness too small ");
goto end;
}
```



```

gs_sat = templ4;                                \\ Scale the gradient strength

SELECTSLICE( current_slice )
SLICEOFF(fov_phase_off,sat_pulse_bwidth,scale_phase_off,sat_slice_freq_long,fov_sat_slice_freq,k
Hz_new3,HZ_new3)
templ1 = 1L*sat_offset*sat_pulse_bwidth;
templ2 = sat_slice_freq_long*im_FOV;
sat_slice_freq_long = (templ2 + templ1)/sat_fov_scale;
fov_sat_slice_freq = sat_slice_freq_long;
kHz_new3 = 0;
Hz_new3 = 0;
if (sat_slice_freq_long < -32767L | sat_slice_freq_long > 32767L) {
fov_sat_slice_freq = sat_slice_freq_long%32767L;
kHz_new3 = 32*(sat_slice_freq_long/32767);
Hz_new3 = 767*(sat_slice_freq_long/32767);}
}

if (sat_orient== 2 && satgs_on == 1) {
sat_channel = 0x0002;                          \\ Address of read channel on 3040 board
templ1 = -32768L*sat_pulse_bwidth;              \\ Calculate pre-sat gradient strength
templ2 = sat_fov_scale*1L;
templ2 = templ2*grad_var[0];
templ4 = templ1/templ2;
ret = dacmaxlong + templ4;

if (ret < 0 ) {
printf(" Pre-sat slice thickness too small ");
goto end;
}
gs_sat = templ4;                                \\ Scale the gradient strength

SELECTSLICE( current_slice )
SLICEOFF(fov_read_off,sat_pulse_bwidth,scale_read_off,sat_slice_freq_long,fov_sat_slice_freq,kHz_
new3,HZ_new3)
templ1 = 1L*sat_offset*sat_pulse_bwidth;
templ2 = sat_slice_freq_long*im_FOV;
sat_slice_freq_long = (templ2 + templ1)/sat_fov_scale;
fov_sat_slice_freq = sat_slice_freq_long;
kHz_new3 = 0;
Hz_new3 = 0;
if (sat_slice_freq_long < -32767L | sat_slice_freq_long > 32767L) {
fov_sat_slice_freq = sat_slice_freq_long%32767L;
kHz_new3 = 32*(sat_slice_freq_long/32767);
Hz_new3 = 767*(sat_slice_freq_long/32767);}
}
if (report_on) printf("gs_sat = %d sat_freq = %d FOV = %d \n",gs_sat,fov_sat_slice_freq,im_FOV);

```

Point 12:

```

BASEMATRIX( subj_angle_z, subj_angle_y, subj_angle_x )          \\ Calcluate Base
matrix
delay(caldelay,us);

\\ macro call to remove initial glitch from MR3031 and MR3040

MR3031_DEGLITCH
MR3040_DEGLITCH

```

\\ Create gradient lists to be used for the slice, read, and phase profiles

```
/* 100 us is the minimum tramp permitted by PARSETUP. This sets the argument 'clock' to 20 and
the sequencer timebase to 2 us. (the argument multiplies the duration of the sequencer master
clock timebase by 100 ns units). A tramp of 1 ms sets the argument to 200 and the timebase to 20
us.*/
```

```
clock = tramp/5;
MR3040_Clock(clock);
```

```
MR3040_SetListAddress(0);
```

```
slice_list = MR3040_InitList();
POSPULSE_HOLD()
```

```
MR3040_Output(NOLOOP,grad.address."0_mn_sec",grad.size."0_mn_sec",grad.waits."0_mn_sec");
DELAY(tref, clock, "mn_0_sec")
MR3040_Output( NOLOOP,
grad.address."mn_0_sec",grad.size."mn_0_sec",grad.waits."mn_0_sec");
```

```
slice_180_refocus = MR3040_InitList();
POSPULSE_HOLD()
```

```
sat_list = MR3040_InitList();
POSPULSE_2_HOLD()
```

```
read_list = MR3040_InitList();
```

```
MR3040_Output(NOLOOP,grad.address."0_mn_sec",grad.size."0_mn_sec",grad.waits."0_mn_sec");
DELAY(tdp, clock, "mn_0_sec")
MR3040_Output( NOLOOP,
grad.address."mn_0_sec",grad.size."mn_0_sec",grad.waits."mn_0_sec");
DELAY(eddy_delay, clock, "0_max")
MR3040_Output(NOLOOP,grad.address."0_max",grad.size."0_max",grad.waits."0_max");
DELAY(tacq_int, clock, "max_0")
MR3040_Output( NOLOOP, grad.address."max_0",grad.size."max_0",grad.waits."max_0");
DELAY(eddy_delay, clock, "0_max")
```

```
MR3040_Output(NOLOOP,grad.address."0_mn_sec",grad.size."0_mn_sec",grad.waits."0_mn_sec");
DELAY(tdp, clock, "mn_0_sec")
MR3040_Output( NOLOOP,
grad.address."mn_0_sec",grad.size."mn_0_sec",grad.waits."mn_0_sec");
```

```
phase_list = MR3040_InitList();
```

```
MR3040_Output(NOLOOP,grad.address."0_mn_sec",grad.size."0_mn_sec",grad.waits."0_mn_sec");
DELAY(tdp, clock, "mn_0_sec")
MR3040_Output( NOLOOP,
grad.address."mn_0_sec",grad.size."mn_0_sec",grad.waits."mn_0_sec");
DELAY(eddy_delay, clock, "0_max")
DELAY(tramp, clock, "0_max")
DELAY(tacq_int, clock, "0_max")
DELAY(tramp, clock, "0_max")
DELAY(eddy_delay, clock, "0_max")
```

```
MR3040_Output(NOLOOP,grad.address."0_mx_sec",grad.size."0_mx_sec",grad.waits."0_mx_sec");
```



```

    DELAY(tdp, clock, "mx_0_sec")
    MR3040_Output( NOLOOP,
grad.address."mx_0_sec",grad.size."mx_0_sec",grad.waits."mx_0_sec");

    slice_list_rp = MR3040_InitList();

MR3040_Output(NOLOOP,grad.address."0_mx_sec",grad.size."0_mx_sec",grad.waits."0_mx_sec");
    DELAY(tdp, clock, "mx_0_sec")
    MR3040_Output( NOLOOP,
grad.address."mx_0_sec",grad.size."mx_0_sec",grad.waits."mx_0_sec");
    DELAY(eddy_delay, clock, "0_max")
    DELAY(tramp, clock, "0_max")
    DELAY(tacq_int, clock, "0_max")
    DELAY(tramp, clock, "0_max")
    DELAY(eddy_delay, clock, "0_max")

MR3040_Output(NOLOOP,grad.address."0_mx_sec",grad.size."0_mx_sec",grad.waits."0_mx_sec");
    DELAY(tdp, clock, "mx_0_sec")
    MR3040_Output( NOLOOP,
grad.address."mx_0_sec",grad.size."mx_0_sec",grad.waits."mx_0_sec");

    CREATE_MATRIX(ss_mat,0,0,0)
    delay(caldelay,us);                \\ Create a zero matrix to select during matrix
recalculation

    phase_rec = 0;                    \\ reset receiver phase to zero
    sync();                          \\ Synchronise to 100kHz
    starttimer();                    \\ mark time for first acquisition, initiated after each
profile

\\ ***** MAIN ACQUISITION LOOP *****

```

Point 13:

```

    image_av = 0;
averages_loop:

    completed_ex = 0;
experiments_loop:

    slice_batch_start = 0;
slice_batch_loop:

    disacq_cnt = 0;
    current_view = 0;
phase_encode_loop:

    view_av = 0;
view_block_loop:

discard_loop:

    current_slice = slice_batch_start;
multislice_loop:
    current_slice = current_slice + 1;

```



```
slice_av = 0;
no_acq = view_av + image_av;
slice_block_loop:

/* Now use the offset macros to calculate the offsets in terms of frequency and phase increments.
They work as follows:-
1.READOFF returns the value of the read offset in Hz to
read_freq_long and a 16 bit conversion to fov_read_freq plus kHz_new2 and Hz_new2
2.PHASEOFF returns the number of phase increments to be added to the
receiver phase each time a new line of k space is acquired. This is
passed back to the variable fov_phase_deg.
3. SLICEOFF returns the value of the slice offset in Hz to the variable
slice_freq_long and a 16 bit conversion to fov_slice_freq plus kHz_new and Hz_new */

    BATCHSLICE( current_slice )

READOFF(fov_read_off,sample_period,scale_read_off,read_freq_long,fov_read_freq,kHz_new2,HZ_new2)
    PHASEOFF(fov_phase_off,phase_res,scale_phase_off,fov_phase_deg)

SLICEOFF(fov_slice_off,pulse_bw, scale_slice_off,slice_freq_long,fov_slice_freq,kHz_new,HZ_new
)

/* Set phase according to PHASE_CYCLE keyword and acquisition number transmitter phase of 90
is alternated. So if phase cycling is selected the DC offset is eliminated but so also is the spurious
signal from an imperfect 180 deg pulse. */

    phase_90 = aqphase(no_acq,phase_cycle)*deg_90;
    phase_180 = aqphase(no_acq,phase_cycle)*deg_90 + deg_90;

/* Set up frequency offsets for current slice. */

    templ3 = fov_read_freq + rec_freq;
    if (templ3 + dacmaxlong < 0 | templ3 - dacmaxlong > 0) {
        printf("Reduce receive frequency\n");
        goto end;
    }
    frequency_buffer(1);
    frequency(MHz, kHz+kHz_new2, Hz+Hz_new2, rx1MHz);
    offset_frequency(fov_read_freq+rec_freq);

    if(satgs_on){
        frequency_buffer(3);
        frequency(MHz, kHz+kHz_new3, Hz+Hz_new3, rx1MHz);
        offset_frequency(fov_sat_slice_freq);
        reset_frequency();
    }

    frequency_buffer(0);          // buffer 0 = tx freq
    frequency(MHz, kHz+kHz_new, Hz+Hz_new, rx1MHz);
    offset_frequency(fov_slice_freq+(slice_freq_var*10));
    reset_frequency();

    p90_mul = scale(rfcal,alpha,90);
    p180_mul = scale(rfcal,p180_scale,100);
    p_sat_mul = scale(rfcal,p_sat_scale,100);
    if (p_sat_mul > 2047) p_sat_mul = 2047;
```

```
if (p90_mul > 2047) p90_mul = 2047;
if (p180_mul > 2047) p180_mul = 2047;
```

\\ Calculation of te extend time

Point 14:

```
te_a = tsel90 + tcrush + tref + 4*tramp;
te_b = tsel90/2 + tcrush + tref + eddy_delay + 4*tramp + tacq/2;
te_c = tsel90 + tcrush + 2*tramp;
temp = te_a;
if (te_b > temp) temp = te_b;
if (report_on == 1) printf("\nMinimum TE = %d ms\n", (temp+800)/500 + 1);
if (te*500 - te_a - 800 < 0) {
    printf("Echo time too short, extend by %d ms\n", (te_a+800)/500-te+1);
    goto end;
}
if (te*500 - te_b - 800 < 0) {
    printf("Echo time too short, extend by %d ms\n", (te_b+800)/500-te+1);
    goto end;
}
te_balance_a = te*500 - te_a;
te_balance_b = te*500 - te_b;
te_balance_c = te*500 - te_c;
```

\\ Calculate phase increment following each 180 degree pulse

```
templ1 = slice_freq_long + (slice_freq_var*10L) - read_freq_long - (rec_freq*1L);
templ2 = (((tacq*1L)+(tramp*1L))*(tacq_phase*1L))/10L;
templ3 = overhead*1L;
templ4 = ((templ2+templ3)*templ1)/1000L;
phase_ang = templ4%(deg_360*1L);
templ5 = ((templ2+templ3)*templ1)%1000L;
remainder_phase = templ5;
phase_correction = 0;
```

\\ tr_min includes additional 3 ms to allow for matrix calculations and overheads

Point 15:

```
tr_min = (tramp + tsel90/2 + satgs_on*(sat_tcrush + tsel_sat + 4*tramp + 100) + de_on*(2*tramp +
tcrush + tsel90))/1000 + views_per_seg*te + te/2 + de_on*(te/2) + 3;
if (report_on == 1) printf("Minimum TR = %d ms\n", tr_min*batch_slices);
tr_extend = ((tr*1000L/batch_slices) - tr_min*1000L)/1000L;
if (tr_extend < 0) {
    printf("TR too short extend by %d ms\n", (((tr_min*1000L -
(tr*1000L/batch_slices))*batch_slices/1000L) + 1));
    if(batch_slices > 1) printf("or decrease number of slices \n");
    goto end;
}
```

\\ Real time adjustment of slice and read compensation gradients

Point 16:

```
templ1 = gs_comp;
```



```

templ1 = templ1 + (templ1*(gs_comp_scale))/1000L;           \\ Manual override of slice
rephase lobe
templ2 = (templ1*gsp_lobe)/100L;
templ3 = templ1 - templ2;
if (templ2 + dacmaxlong < 0 | templ3 + dacmaxlong < 0){
    printf("Reduce slice gradient compensation\n");
    goto end;
}
gs_rp = templ3;
gsp_rp = templ2;

templ1 = gr_comp;
templ1 = templ1 + (templ1*gr_comp_scale)/1000L;           \\ Manual override of
switching dephase lobe
templ2 = (templ1*grp_lobe)/100L;
templ3 = templ1 - templ2;
if (templ2 + dacmaxlong < 0 | templ3 + dacmaxlong < 0){
    printf("Reduce read gradient compensation\n");
    goto end;
}
gr_dp = templ3;
grp_dp = -templ2;

\\ Calculate the matrices for this phase-encode step
\\ Phase-encode and refocus matrices only need be rescaled, but recalculate all for real time
update of angulations

```

Point 17:

```

MR3040_SelectMatrix( ss_mat );           \\ Select a zero matrix during matrix re-
calculation

CREATE_MATRIX(aq_mat,gs_on*gs_var_rescale,gp_on*gp_var,(gr_on*gr_undersample)*(gr_var/gr
_oversample))
delay(caldelay,us);
CREATE_MATRIX(aq_mat_sec,gs_on*gsp_rp,gp_on*gp_var,gr_on*gr_dp)
delay(caldelay,us);

CREATE_MATRIX(fse_mat,gs_on*gs_var_rescale,0,0)
delay(caldelay,us);
CREATE_MATRIX(fse_mat_sec,gs_on*gs_rp,0,gr_on*grp_dp)
delay(caldelay,us);

CREATE_MATRIX(slice_crush,gs_on*gs_var_rescale,0,0)
delay(caldelay,us);
CREATE_MATRIX(slice_crush_sec, 0, 0, 0)
delay(caldelay,us);

CREATE_MATRIX(chess_mat, satgs_on*gs_sat, satgs_on*gs_sat, satgs_on*gs_sat)
delay(caldelay,us);

CREATE_MATRIX(chess_mat_sec, satgs_on*sat_gr_amp, satgs_on*sat_gr_amp,
satgs_on*sat_gr_amp)
delay(caldelay,us);

```

/* For narrow band systems the observe modulator level can be 0 or 4 ie on or off full controll is provided in the instrument panel 0db to - 40db . So if the RF is on set the observe modulator level to 4 */

```
rf_on = 0 ;
if ( obs_mod_level > 0 ) rf_on = 4 ;
```

\\ If rf spoiling selected then use rf spoiling macro

```
if (rfsp_on > 0 ) {
    rf_inc = rf_inc + 1L;
    ret = 0;
    RFSPoil_SE(rf_inc, deg_90, deg_117, phase_90, phase_180, ret)
}
```

\\ Discard acquisitions

```
if(disacq_cnt < no_disacq){
    Dummy_Cycles(1);
}
else {
    Dummy_Cycles(0);
}
```

Point 18:

```
waittimer(30000); \\ Recovery delay
time = hostrequest();
resync();
\\ *****SEQUENCE STARTS HERE *****
\\ Mains gating

if (mgating){
    output(27);
    do{
        noop;
    } until input(8)!=0;
    do{
        noop;
    } until input(8)==0;
}

\\ Setup gradients and rf phase
```

Point 19:

```
echo_cnt = 0;
resync();
MR3040_Clock(clock);
phase(phase_90);           \\ 90x pulse
```

Point 20:

```
if (satgs_on) {
    starttimer();
    MR3040_SetList( sat_list, sat_channel );
```

```

MR3040_SelectMatrix( chess_mat );
MR3040_Clock(clock);
frequency_buffer(3);
reset_frequency();
waittimer(1000);

MR3040_Start(sat_channel);
MR3031_RFSTART(rfnum_sat,tset_sat,p_sat_mul,tramp,rf_on);
MR3040_CONTINUE(sat_channel);
delay(tramp,us);
delay(tramp,us);
delay(sat_tcrush,us);
MR3040_CONTINUE( sat_channel );

starttimer();
ret = tramp*10;
frequency_buffer(0);
reset_frequency();
waittimer(ret);
}

```

\\ Selective 90 degree pulse in presence of slice select gradient

Point 21:

```

MR3040_SetList( slice_list, CHANNEL_S | CHANNEL_R );
MR3040_SelectMatrix( fse_mat );

MR3040_Start( CHANNEL_S | CHANNEL_R );
MR3031_RFSTART(rfnum,tset90,p90_mul,tramp,rf_on)
MR3040_CONTINUE( CHANNEL_S | CHANNEL_R );
starttimer();
ret = tramp*10;
resync();
phase(phase_180);           \\ 180 y pulse
waittimer(ret);
delay(tramp,us);
delay(tref,us);
delay(tramp,us);

ret = te_balance_a - 200;
delay(ret,us);
starttimer();
MR3040_SetList( slice_180_refocus, CHANNEL_S);
MR3040_SelectMatrix(slice_crush);
waittimer(2000 + post_90_delay);

```

Point 22:

```

echo_loop:
MR3040_Start( CHANNEL_S );
MR3031_RFSTART(rfnum,tset90,p180_mul,tramp+tcrush,rf_on)
delay(tcrush,us);
MR3040_CONTINUE( CHANNEL_S );
delay(tramp,us);

```

\\ Recalculate phase encode matrices

```
starttimer();
ret = te_balance_b - 800;
```

```
gp_mul = gp_order[current_view + echo_cnt];
gp_var = (-gp_inc*gp_mul);
```

```
MR3040_SelectMatrix( ss_mat );
```

```
CREATE_MATRIX(aq_mat,gs_on*gs_var_rescale,gp_on*gp_var,(gr_on*gr_undersample)*(gr_var/gr_oversample))
    delay(caldelay,us);
CREATE_MATRIX(aq_mat_sec,gs_on*gsp_rp,gp_on*gp_var,gr_on*gr_dp)
    delay(caldelay,us);
```

\\ Calculate increment in rec phase for offset in pe direction . This is necessary for each PE step

```
templ1 = IntToLong(gp_mul) * IntToLong(fov_phase_deg);
phase_rec_cal2 = scale(360,1000,phase_res);
phase_rec = templ1%phase_rec_cal2;
if (gp_on == 0 ) phase_rec = 0;
if (rfsp_on > 0 ) {temp_mac = phase_rec + rf_sp_angle;
temp_mac = temp_mac%(deg_90*4);
phase_rec = temp_mac;}
temp = 0;
if(echo_cnt == 1) temp = post_180_delay_1;
```

```
MR3040_SelectMatrix( aq_mat );
MR3040_SetList( read_list, CHANNEL_R );
MR3040_SetList( phase_list, CHANNEL_P );
MR3040_SetList( slice_list_rp, CHANNEL_S );
waittimer(8000 + post_180_delay + temp + post_acq_delay);
```

```
delay(ret,us);
```

```
MR3040_Start( ALL_CHANNELS );
delay(tramp,us);
starttimer();
ret=tref*10;
rphase(phase_rec+phase_correction);
templ1 = phase_ang;
templ2 = echo_cnt + 1;
templ3 = templ1*templ2;
phase_correction_0 = templ3%deg_360*1L;
templ4 = templ2*(remainder_phase*1L)/1000L;
phase_correction_1 = templ4;
```

```
templ4 = templ2*(remainder_phase*1L)%1000L;
phase_correction_2 = 0;
if (templ4 > 500 ) phase_correction_2 = 1;
if (templ4 < -500 ) phase_correction_2 = -1;
phase_correction = (phase_correction_0 + phase_correction_1 +
phase_correction_2)%deg_360;
waittimer(ret);
delay(tramp,us);
delay(eddy_delay,us);
```

```

starttimer();
ret = tramp*10;
resync();
frequency_buffer(1);           // buffer 1 = rx freq
reset_frequency();
waittimer(ret);

```

```

acquire( sample_period, no_samples );

```

```

starttimer();
resync();
frequency_buffer(0);           // buffer 0 = tx freq
reset_frequency();
phase(phase_180 + phase_correction);
ret = tramp*20;

```

```

echo_cnt = echo_cnt + 1;
del = tref - tfilter;
waittimer(ret);
delay(eddy_delay,us);
delay(del,us);
delay(tramp,us);

```

```

ret = te_balance_b - 800;
delay(ret,us);

```

```

starttimer();
MR3040_SetList( slice_180_refocus, CHANNEL_S);
if ( echo_cnt < views_per_seg ) {
waittimer(8000 - post_acq_delay);
goto echo_loop;
}

```

\\ Driven Equilibrium Pulses

```

if (de_on == 1){
waittimer(8000 - post_acq_delay);
MR3040_Start( CHANNEL_S );
MR3031_RFSTART(rfnum,tse190,p180_mul,tramp+tcrush,rf_on)
delay(tcush,us);
MR3040_CONTINUE( CHANNEL_S );
delay(tramp,us);

```

```

starttimer();
ret = te_balance_c - 200;
phase(phase_90 + 2*deg_90 + phase_correction);    \\ 90-x pulse
MR3040_SetList( slice_180_refocus, CHANNEL_S);
MR3040_SelectMatrix(slice_crush);
waittimer(2000);
delay(ret,us);

```

```

MR3040_Start( CHANNEL_S );
MR3031_RFSTART(rfnum,tse190,p90_mul,tramp,rf_on)
MR3040_CONTINUE( CHANNEL_S );
delay(tramp,us);
}

```

\\ Repetition delays

```
if(time > 10)time = 0;
tr_extend = (tr/batch_slices) - tr_min - time;
if(tr_extend > 0)delay(tr_extend, ms);
```

starttimer();

\\ matrix calculations and overheads

Point 23:

```
no_acq = no_acq + 1;                \\ acquisition number
slice_av = slice_av + 1;
if( slice_av < slice_block )        \\ Slice block
goto slice_block_loop;

if (current_slice < slice_batch_start + batch_slices)
goto multislice_loop;

disacq_cnt = disacq_cnt + 1;
if (disacq_cnt <= no_disacq)        \\ discarded acquisitions
goto discard_loop;

view_av = view_av + slice_block;
if (view_av < view_block)           \\ View block
goto view_block_loop;

current_view = current_view + views_per_seg;
if (current_view < no_views )       \\ Phase encode
goto phase_encode_loop;
slice_batch_start = slice_batch_start + batch_slices;
if (slice_batch_start < no_slices)
goto slice_batch_loop;

completed_ex = completed_ex + 1;
if (completed_ex < no_experiments)
goto experiments_loop;

image_av = image_av + view_block;
if (image_av < no_averages)         \\ Averages
goto averages_loop;

end:

}
```

At the top of the program is a comment which gives a brief description of the sequence. The #use statements are declared next indicating the location of pulse shapes that will be used by the sequence. The PARAMLIST keywords are set out within a comment statement so as not to interfere with the PPL compiler. See Chapter 2 for description of keywords in PARAMLIST header. Several features are defined using SCROLLBAR and EDITTEXT keywords. Among those are phase/frequency over/under-sampling options, enabling/disabling the phase navigator, control over patient angles.

- **Point 1**

Variables for RF pulse control are defined. While pulse shape has to be selected prior to running the sequence, RF calibration value, scale factor for $90^\circ/180^\circ$ pulse, observation mode (on/off) and RF spoiling (on/off) can be edited in real-time.

- **Point 2**

Real-time control over repetition and echo times and Partial Fourier (PF) acquisition option. 0 under "PF effective number of echoes" mean that the PF option is disabled. In order to use the PF option one has to work the PF effective number of echoes and phase encode steps (NO_VIEWS). This is illustrated using an example below. The reconstruction parameters for PF will be covered later. For example: we would like to acquire 256×256 matrix using 8 echo train and shorten the acquisition time as much as possible using the PF option. What are the NO_VIEWS and PF effective number of echoes? Note that the frequency encode is not affected by PF thus NO_SAMPLES is 256 in this case.

- ✓ The acquired data has to cover just over half of the k-space. The recommendation is to acquire at least 55%. For our example we have to acquire at least $256 \times 0.55 = (\text{round to nearest integer}) 141$ phase encode steps.
- ✓ Next we need to work out the phase encode steps per echo that fit into FSE constraints: PF phase encode steps/echo train length $\rightarrow$ round to the nearest even number. In our example: $141/8 = 17.6 \rightarrow 18$.
- ✓ Using the phase encode steps per echo we need to calculate the phase encode steps that will actually be acquired using PF: phase encode steps per echo * echo train length. For example: $18 \times 8 = 144$. This is the number that has to be entered under NO_VIEWS. Note: if phase navigator is enabled the echo train length has to be added to NO_VIEWS. In the example above the NO_VIEWS with enabled phase navigator would become $144 + 8 = 152$.
- ✓ The number of the PF effective echoes is calculated as follows: The desired matrix size(full k-space)/phase encode steps per echo $\rightarrow$ round to the nearest integer. Our example: $256/18 = 14.2 \rightarrow 14$. This is the number that has to be entered under "PF effective number of echoes".

- **Point 3**

Ramp time and rephase lobe duration have to be defined (in us) prior to running the sequence.

- **Point 4**

Real-time parameters for image quality optimization.

- **Point 5**

Parameters for pre-saturation band control.

- **Point 6**

The gradients can be switched on/off in real time on individual channels.

- **Point 7**

Declaration of various variables.

- **Point 8**

Matrices are defined using #define keyword. The primary matrices are given arbitrary, but unique number. Secondary matrix must be defined using primary matrix number + 256. For example:

```
#define slice_crush          21    //arbitrary unique number
#define slice_crush_sec     277    //21+256=277
```

- **Point 9**

tfilter is calculation time for the digital filters which depends on bandwidth and dsp routine used. acqpad is the function that does this calculation.

- **Point 10**

Setting the optimum phase encode order for FSE.

- **Point 11**

Setting up various imaging parameters, including user-friendly error messages.

- **Point 12**

Calculations for MR3040 Oblique Gradient waveform generator. See 4.2 and 4.3.

- **Point 13**

The main acquisition loop starts here. The looping structure is as shown in diagram 4.2. BATCHSLICE is defined in offset_15.pph

- **Point 14**

Minimum TE is calculated including time overhead for crusher, ramp time and rephase lobe duration.

- **Point 15**

Minimum TR is calculated including the overhead if pre-saturation pulse or/and driven equilibrium options are enabled.

- **Point 16**

This bit of code enables the user to adjust slice and read gradient compensations in real time for easier image quality optimization.

- **Point 17**

Matrix calculations, see Sections 4.2 and 4.3.

- **Point 18**

A time delay is added to allow all the calculations to complete. **hostrequest()** checks the MR3020 input buffer and adds extra time delay if needed. Then the sequence is synchronized to AC with the **resync()** function.

- **Point 19**

Gradients and RF pulse preparation.

- **Point 20**

Pre-saturation pulse.

- **Point 21**

90° pulse. Note that the execution time is controlled with **starttimer** (starts an internal clock for timing sections), **waittimer** (waits the specified amount of time starting from starttimer point) and **delay** (requires time units to be specified as well) functions. If the time

specified in waittimer is not enough to execute the statements in that timing block, a time delay of 5 ms will be added.

- **Point 22**

Echo loop.

- **Point 23**

The loop counters are incremented and the pulse sequence is terminated with the end statement.

[Reconstruction of data using PF option]

Consider the same example where 256x256 data matrix was acquired with 8 echo train using 14 PF effective echoes (144 actual phase encode steps). The lookup table has to be generated using **pfgen** tool with the following syntax:

```
pfgen outfile te_eff echoes pf_echoes views nav_on
```

Outfile: output file name, maximum 8 characters

te_eff: which echo is at the centre of k-space (Effective echo time/Base echo time)

echoes: echo train length (8 in the above example)

pf_echoes: Pf effective number of echoes (14 in the above example)

views: acquired number of phase encode steps including the navigator echoes (in the above example 144 if the phase navigator is disabled, 152 if the navigator is enabled)

nav_on: 1 if phase navigator is enabled, 0 if it is disabled.

The changes that are needed to the RPR file if PF option is used:

- ✓ Change the lookup table VIEW_TABLE file name into the one generated by pfgen.
- ✓ Change the phase encode steps SAMPLE DIM2 to the actual phase encode steps acquired (NO_VIEWS) including the number of phase navigator echoes
- ✓ Edit the Fourier Transform size if required (would be forced to 256 in the example above)
- ✓ Output size should stay as the number of phase encode steps in the full k-space (256 in the example above)

Add three user-adjustable parameters for Partial Fourier: Kernel Offset, Iterations and Smoothing. The Kernel is calculated as follows: Total K space/2 - NO_VIEWS(without navigator echoes). In the example above it is $256/2 - 144 = -26$. The Smoothing defines the

number of points over which the two data sets are combined (linearly) to provide a smooth transition and minimise ringing. A typical value would be 10% of the total k-space (26 in the example above). Iterations: this specifies the number of loops the algorithm executes trying to iterate on the correct answer. The recommended number is 5.

Chapter 8. FINAL WORDS

Should you encounter any issue or require technical support, please contact **MRS** Support in the United Kingdom.

Users finding errors or omissions in this document are requested to advise **MRS**.

Email: support@mrsolutions.com

Write or telephone:

*MR Solutions Limited,
Ashbourne House, The Guildway
Old Portsmouth Road, Artington
Guildford,
Surrey GU3 1LR, UK
Tel: +44 (0) 1483 906 308
Fax: +44 (0) 1483 375 990*

When contacting **MRS**, please have the following information available:

- Product name and version number
- Operating system and version number
- As many pictures / information about your issue
- What has been done so far to address the issue
- The TeamViewer ID and password to your machine (make sure remote access is permitted at your site)